PHÂN HIỆU TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI
TẠI TP. HỒ CHÍ MINH

BỘ MÔN CÔNG NGHỆ THÔNG TIN

KHAI PHÁ DỮ LIỆU

BÁO CÁO THỰC HÀNH BUỔI 2

Phạm Hải Yến - 6451071091

CQ.CN.64.CNTT

GVHD: Ths. Nguyễn Thiện Dương

TP. Hồ Chí Minh - 2026

Lưu ý:

1. *Số trang trong tài liệu thực hành tính theo số trang ứng dụng đọc PDF đếm*
2. *SV bố trí mỗi câu ở dưới sẽ là 1 file mã nguồn riêng (file.ipynb)*
3. *Những câu nào có yêu cầu lập trình viết mã nguồn thì SV phải in ra họ tên – MSSV (lệnh print) trong câu đó*
4. ***Hình chụp kết quả phải có chứa thanh taskbar của máy tính (có kèm theo thời gian và ngày làm bài)***
5. *SV đặt tên cho file mã nguồn mỗi câu như sau:* ***<MSSV>_LAB2_Bai<X>***
6. *Tất cả các câu sau đó sẽ tổng hợp lại và push lên 1 repository duy nhất trên Github*
7. *Đặt tên cho repository theo cú pháp:* ***<MSSV>_Lab2***

BÁO CÁO THỰC HÀNH

SV paste link dẫn đến 1 repo duy nhất của tất cả các câu bên dưới tại dây (lưu ý để ở chế độ public)

https://github.com/25phamhaiyen/6451071091_Lab2

Bài 2 (Trang 15-16)

Câu 1:

//dán mã nguồn

```
# BÀI 2 - CÂU 1
# Nhập dữ liệu Adult Income và xác định số lượng mẫu train, test

from google.colab import files
import pandas as pd
import io

print("Họ tên: Phạm Hải Yến")
print("MSSV: 6451071091")
print("Bài 2 - Câu 1")
print("-" * 50)

# Upload 2 file từ máy tính
uploaded = files.upload()

# Tên các thuộc tính
columns = [
    "age", "workclass", "fnlwgt", "education",
    "education-num", "marital-status", "occupation",
    "relationship", "race", "sex", "capital-gain",
    "capital-loss", "hours-per-week", "native-country",
    "income"
]
```

```
# Tự động tìm tên file đã upload
train_file = next(
    name for name in uploaded.keys()
    if name.startswith("adult.data")
)

test_file = next(
    name for name in uploaded.keys()
    if name.startswith("adult.test")
)

# Đọc dữ liệu huấn luyện
train_df = pd.read_csv(
    io.BytesIO(uploaded[train_file]),
    header=None,
    names=columns,
    skipinitialspace=True
)

# Đọc dữ liệu kiểm thử
test_df = pd.read_csv(
    io.BytesIO(uploaded[test_file]),
    header=None,
    names=columns,
    skipinitialspace=True,
    comment="|"
)

# In số lượng dữ liệu
print("\nTên file huấn luyện:", train_file)
print("Tên file kiểm thử:", test_file)

print("\nSố lượng dữ liệu huấn luyện:",
```

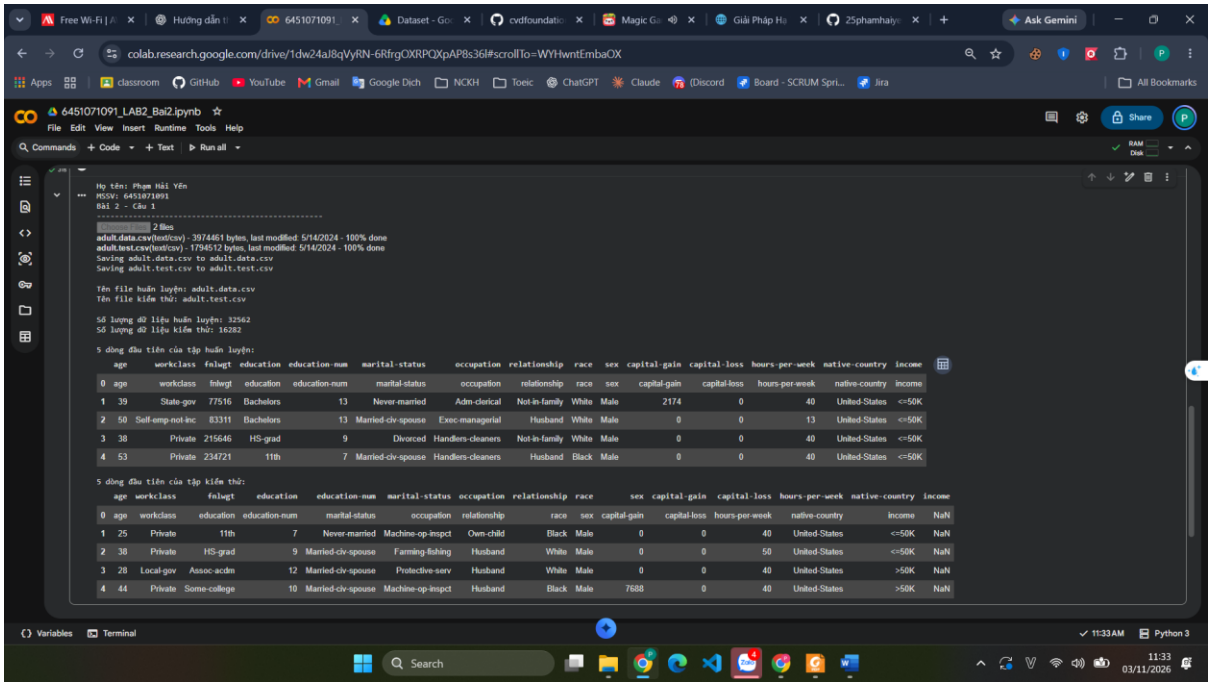
```
      train_df.shape[0])


print("Số lượng dữ liệu kiểm thử:",
      test_df.shape[0])


# Hiển thị dữ liệu
print("\n5 dòng đầu tiên của tập huấn luyện:")
display(train_df.head())


print("\n5 dòng đầu tiên của tập kiểm thử:")
display(test_df.head())
```

//Chụp hình kết quả



Câu 2:

//dán mã nguồn

```
# BÀI 2 - CÂU 2
# TIẾN HÀNH TIỀN XỬ LÝ DỮ LIỆU


import pandas as pd
```

```
import numpy as np


print("Họ tên: Phạm Hải Yến")
print("MSSV: 6451071091")
print("Bài 2 - Câu 2")
print("-" * 50)


# 1. Đọc lại dữ liệu gốc
# File adult.test.csv của bạn có dòng tiêu đề (header)


train_df = pd.read_csv(
    "/content/adult.data.csv",
    skipinitialspace=True
)


test_df = pd.read_csv(
    "/content/adult.test.csv",
    header=0,
    skipinitialspace=True
)


# Xóa khoảng trắng ở tên cột
train_df.columns = train_df.columns.str.strip()
test_df.columns = test_df.columns.str.strip()


# Xóa khoảng trắng ở dữ liệu dạng chuỗi
for df in [train_df, test_df]:
    for col in df.select_dtypes(include="object").columns:
        df[col] = df[col].str.strip()


# 2. Thay ký hiệu '?' bằng NaN
train_df = train_df.replace("?", np.nan)
test_df = test_df.replace("?", np.nan)
```

```
# 3. Xóa các dòng chứa dữ liệu thiếu
train_df = train_df.dropna().copy()
test_df = test_df.dropna().copy()

print("Số dòng train sau khi xóa dữ liệu thiếu:",
      train_df.shape[0])

print("Số dòng test sau khi xóa dữ liệu thiếu:",
      test_df.shape[0])

# 4. Xóa cột fnlwgt nếu tồn tại
train_df = train_df.drop(columns=["fnlwgt"], errors="ignore")
test_df = test_df.drop(columns=["fnlwgt"], errors="ignore")

print("\nCác thuộc tính sau khi xóa fnlwgt:")
print(train_df.columns.tolist())

# 5. Nối dữ liệu huấn luyện và kiểm thử
all_data = pd.concat(
    [train_df, test_df],
    ignore_index=True
)

print("\nKích thước dữ liệu huấn luyện:",
      train_df.shape)

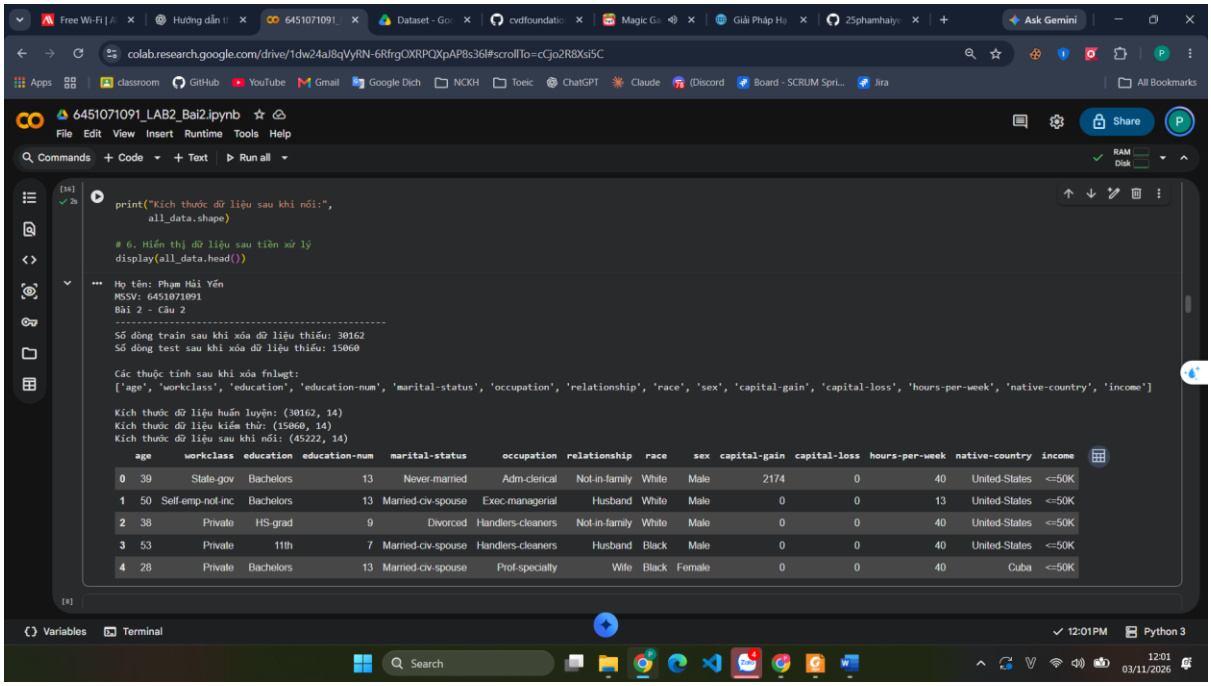
print("Kích thước dữ liệu kiểm thử:",
      test_df.shape)

print("Kích thước dữ liệu sau khi nối:",
      all_data.shape)
```

```
# 6. Hiển thị dữ liệu sau tiền xử lý

display(all_data.head())
```

//Chụp hình kết quả



Câu 3:

//dán mã nguồn

```
# BÀI 2 - CÂU 3

# KHẢO SÁT ĐỘ TƯƠNG ĐỒNG BẰNG PEARSON


import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns


print("Họ tên: Phạm Hải Yến")

print("MSSV: 6451071091")

print("Bài 2 - Câu 3")

print("-" * 50)


# 1. Chuyển các thuộc tính số về đúng kiểu dữ liệu

numeric_columns = [
```

```
    "age",
    "education-num",
    "capital-gain",
    "capital-loss",
    "hours-per-week"
]

for col in numeric_columns:
    if col in all_data.columns:
        all_data[col] = pd.to_numeric(
            all_data[col],
            errors="coerce"
        )

# 2. Lấy các thuộc tính dạng số
numeric_data = all_data.select_dtypes(
    include=[np.number]
)

print("Các thuộc tính số:")
print(numeric_data.columns.tolist())

# 3. Tính ma trận tương quan Pearson
corr_matrix = numeric_data.corr(
    method="pearson"
)

print("\nMa trận tương quan Pearson:")
display(corr_matrix)

# 4. Vẽ biểu đồ Heatmap
plt.figure(figsize=(12, 8))
```

```
sns.heatmap(
    corr_matrix,
    annot=True,
    cmap="coolwarm",
    fmt=".2f",
    center=0
)

plt.title("Correlation Heatmap - Adult Income")
plt.show()

# 5. Xác định các cặp thuộc tính tương quan cao
threshold = 0.8

upper_triangle = corr_matrix.abs().where(
    np.triu(
        np.ones(corr_matrix.shape),
        k=1
    ).astype(bool)
)

high_corr_pairs = []

for col in upper_triangle.columns:
    for row in upper_triangle.index:
        value = upper_triangle.loc[row, col]

        if pd.notna(value) and value >= threshold:
            high_corr_pairs.append(
                (row, col, value)
            )

print("\nCác cặp thuộc tính tương quan cao:")
```

```
for col1, col2, value in high_corr_pairs:
    print(f"{col1} - {col2}: {value:.3f}")

# 6. Loại bỏ thuộc tính tương quan cao
drop_columns = list(
    upper_triangle.columns[
        (upper_triangle >= threshold).any()
    ]
)

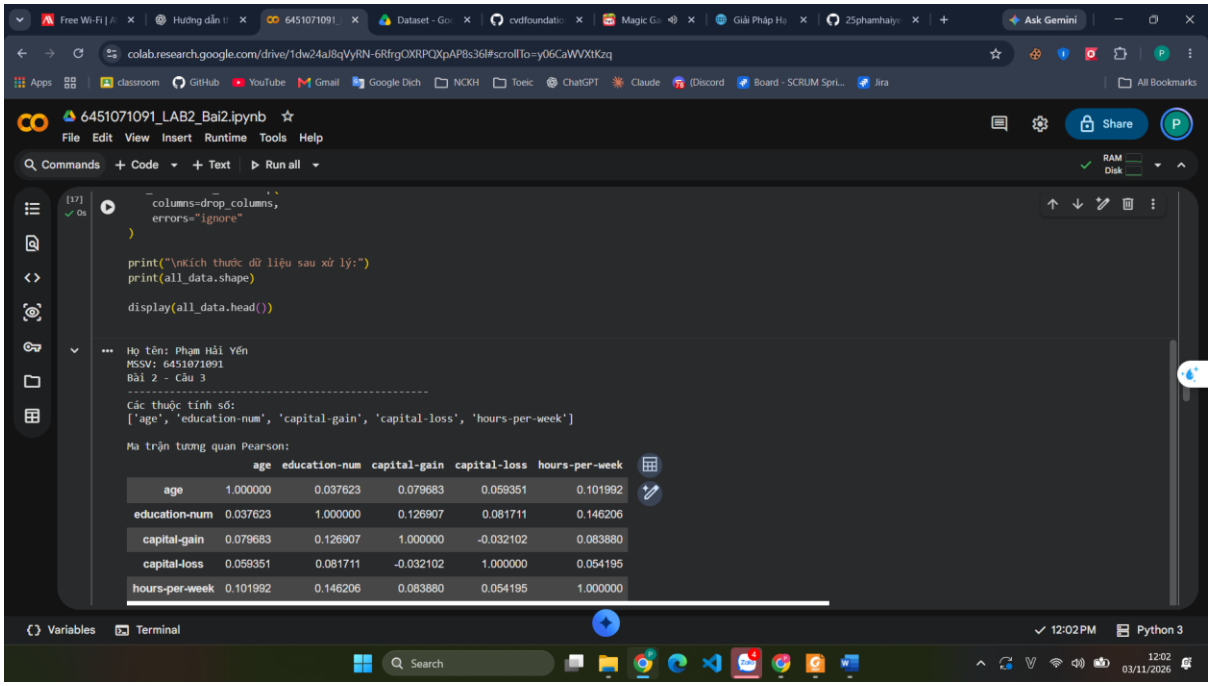
print("\nCác cột bị loại bỏ:")
print(drop_columns)

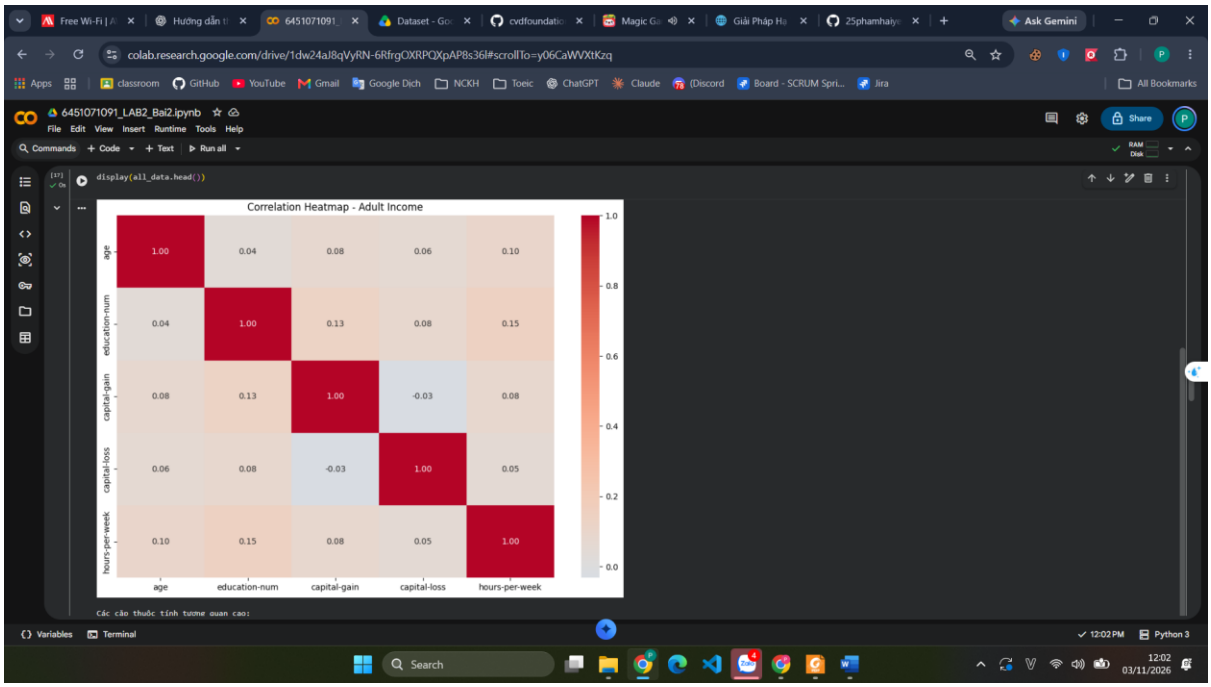
all_data = all_data.drop(
    columns=drop_columns,
    errors="ignore"
)

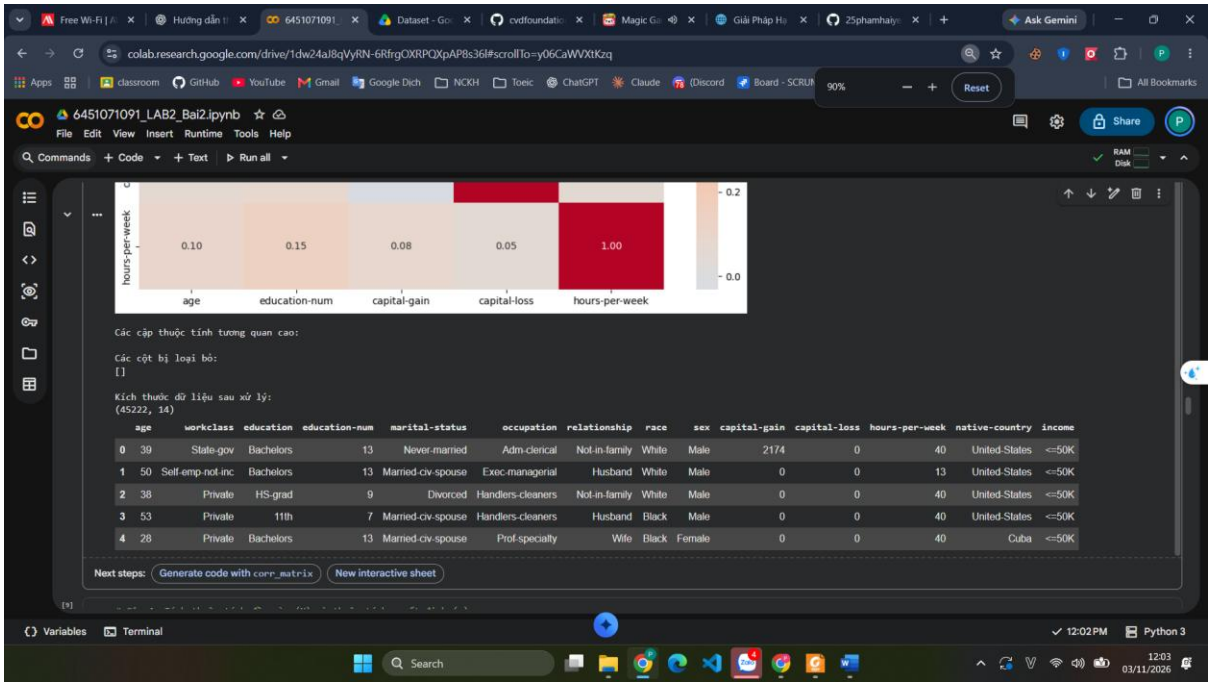
print("\nKích thước dữ liệu sau xử lý:")
print(all_data.shape)

display(all_data.head())
```

//Chụp hình kết quả


6451071091_LAB2_Bai2.ipynb
File Edit View Insert Runtime Tools Help
Share
columns=drop_columns,
errors="ignore"
)
print("\nKích thước dữ liệu sau xử lý:")
print(all_data.shape)
display(all_data.head())
Họ tên: Phạm Hải Yến
MSSV: 6451071091
Bài 2 - Câu 3
Các thuộc tính số:
['age', 'education-num', 'capital-gain', 'capital-loss', 'hours-per-week']
Ma trận tương quan Pearson:
age education-num capital-gain capital-loss hours-per-week
age 1.000000 0.037623 0.079683 0.059351 0.101992
education-num 0.037623 1.000000 0.126907 0.081711 0.146206
capital-gain 0.079683 0.126907 1.000000 -0.032102 0.083880
capital-loss 0.059351 0.081711 -0.032102 1.000000 0.054195
hours-per-week 0.101992 0.146206 0.083880 0.054195 1.000000
Variables Terminal
12:02 PM Python 3


Correlation Heatmap - Adult Income
age
education-num
capital-gain
capital-loss
hours-per-week
1.00 0.04 0.08 0.06 0.10
0.04 1.00 0.13 0.08 0.15
0.08 0.13 1.00 -0.03 0.08
0.06 0.08 -0.03 1.00 0.05
0.10 0.15 0.08 0.05 1.00
Các cặp thuộc tính tương quan cao:
Variables Terminal
12:02 PM Python 3



Câu 4:

//dán mã nguồn

```
# Câu 4: Tách thuộc tính đầu vào (X) và thuộc tính quyết định (y)

print("Họ tên: Phạm Hải Yến")

print("MSSV: 6451071091")

print("Bài 2 - Câu 4")


target_column = "income"


# X: các thuộc tính dùng để dự đoán

X = all_data.drop(columns=[target_column])


# y: thuộc tính cần dự đoán

y = all_data[target_column]


print("Kích thước X:", X.shape)

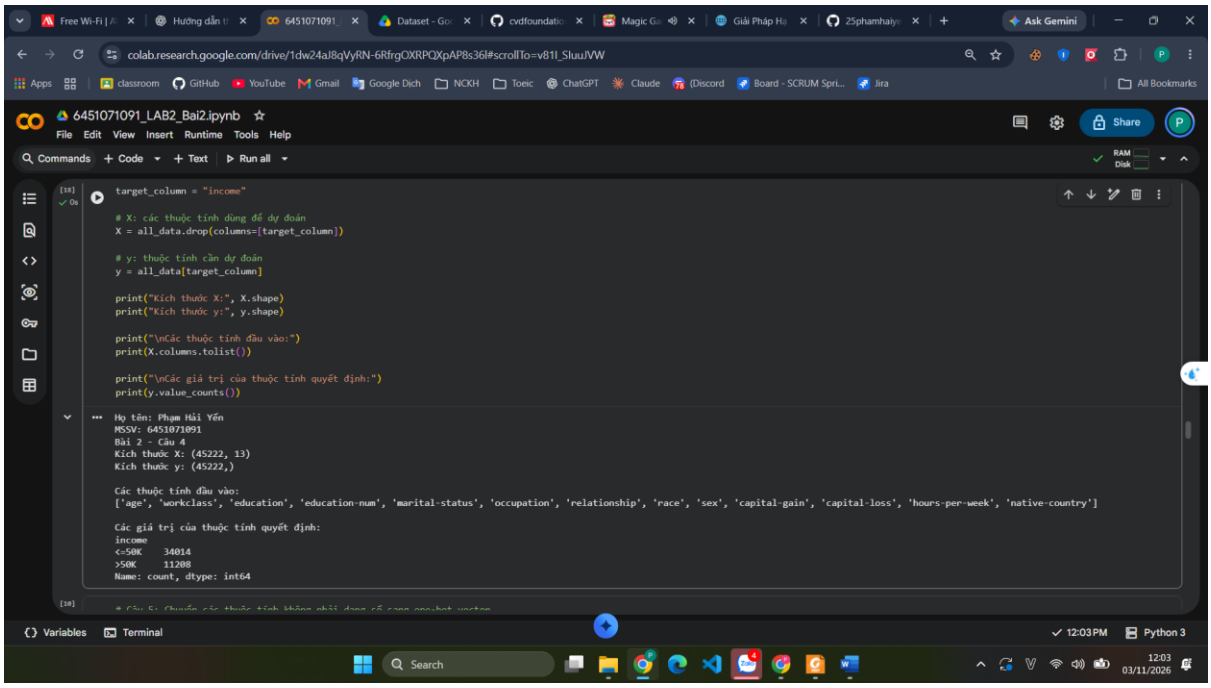
print("Kích thước y:", y.shape)


print("\nCác thuộc tính đầu vào:")

print(X.columns.tolist())
```

```
print("\nCác giá trị của thuộc tính quyết định:")
print(y.value_counts())
```

//Chụp hình kết quả



Câu 5:

//dán mã nguồn

```
# Câu 5: Chuyển các thuộc tính không phải dạng số sang one-hot vector


print("Họ tên: Phạm Hải Yến")
print("MSSV: 6451071091")
print("Bài 2 - Câu 5")


import pandas as pd


# X là dữ liệu đầu vào đã tách ở Câu 4
# Tìm các cột có kiểu dữ liệu không phải số
categorical_columns = X.select_dtypes(
    include=["object", "category", "bool"]
).columns.tolist()
```

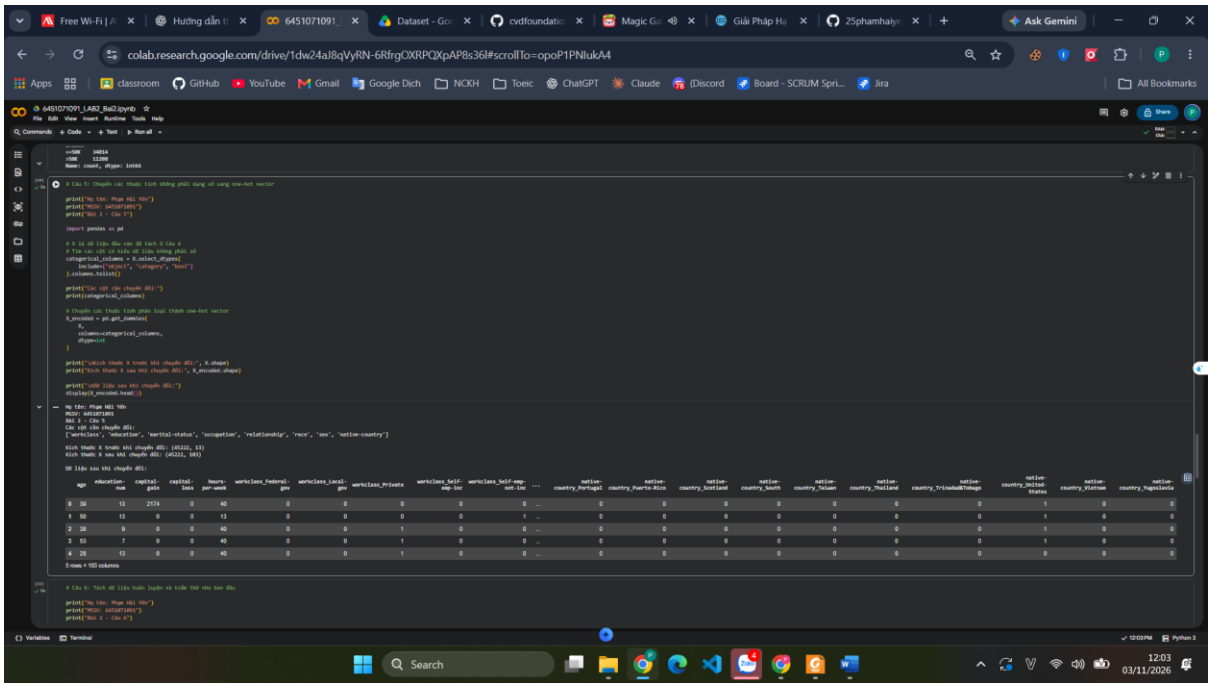
```
print("Các cột cần chuyển đổi:")
print(categorical_columns)


# Chuyển các thuộc tính phân loại thành one-hot vector
X_encoded = pd.get_dummies(
    X,
    columns=categorical_columns,
    dtype=int
)


print("\nKích thước X trước khi chuyển đổi:", X.shape)
print("Kích thước X sau khi chuyển đổi:", X_encoded.shape)


print("\nDữ liệu sau khi chuyển đổi:")
display(X_encoded.head())
```

//Chụp hình kết quả



Câu 6:

//dán mã nguồn

```
# Câu 6: Tách dữ liệu huấn luyện và kiểm thử như ban đầu
```

```
print("Họ tên: Phạm Hải Yến")
print("MSSV: 6451071091")
print("Bài 2 - Câu 6")

# Lấy X và y từ dữ liệu huấn luyện
X_train = train_df.drop(columns=["income"])
y_train = train_df["income"]

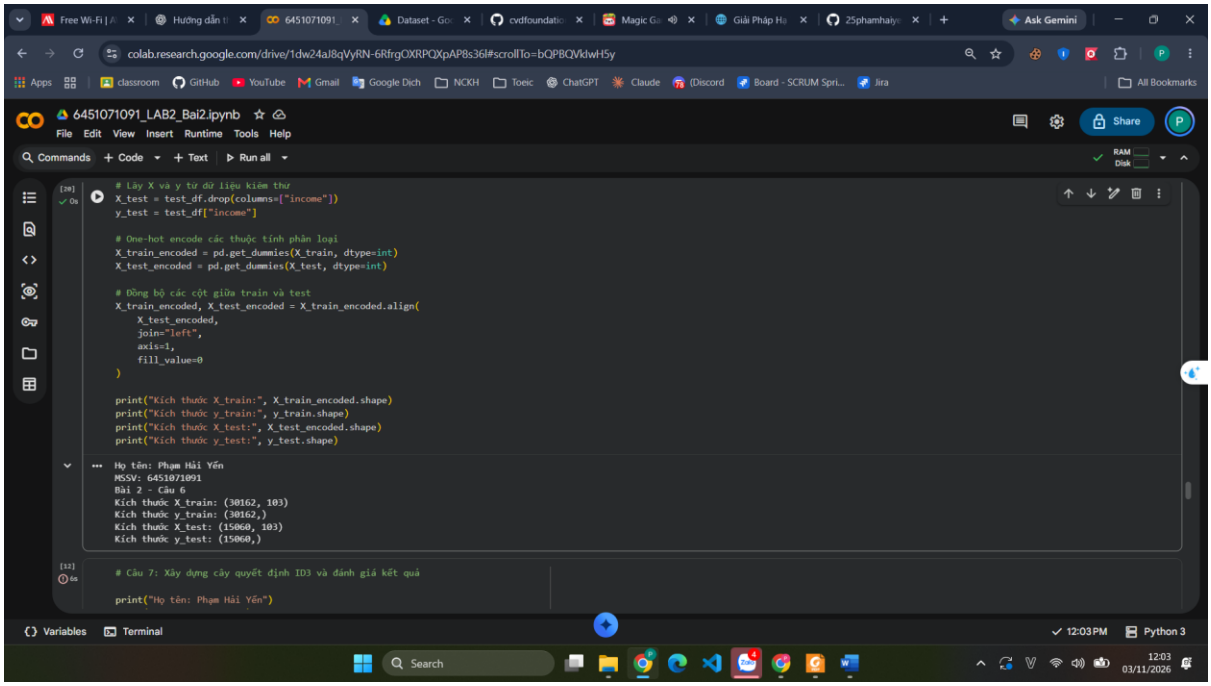
# Lấy X và y từ dữ liệu kiểm thử
X_test = test_df.drop(columns=["income"])
y_test = test_df["income"]

# One-hot encode các thuộc tính phân loại
X_train_encoded = pd.get_dummies(X_train, dtype=int)
X_test_encoded = pd.get_dummies(X_test, dtype=int)

# Đồng bộ các cột giữa train và test
X_train_encoded, X_test_encoded = X_train_encoded.align(
    X_test_encoded,
    join="left",
    axis=1,
    fill_value=0
)

print("Kích thước X_train:", X_train_encoded.shape)
print("Kích thước y_train:", y_train.shape)
print("Kích thước X_test:", X_test_encoded.shape)
print("Kích thước y_test:", y_test.shape)
```

//Chụp hình kết quả



Câu 7:

//dán mã nguồn

```
# Câu 7: Xây dựng cây quyết định ID3 và đánh giá kết quả


print("Họ tên: Phạm Hải Yến")

print("MSSV: 6451071091")

print("Bài 2 - Câu 7")


from sklearn.tree import DecisionTreeClassifier, plot_tree

from sklearn.metrics import confusion_matrix, classification_report,
accuracy_score

import matplotlib.pyplot as plt


# Chuẩn hóa nhãn income:

# adult.test.csv có thể có dấu chấm ở cuối nhãn, ví dụ ">50K."

y_train_id3 = y_train.astype(str).str.strip().str.rstrip(".")

y_test_id3 = y_test.astype(str).str.strip().str.rstrip(".")


# Xây dựng cây quyết định theo Entropy (Information Gain)

id3_model = DecisionTreeClassifier(

    criterion="entropy",
```

```
    random_state=42
)

# Huấn luyện mô hình
id3_model.fit(X_train_encoded, y_train_id3)

# Dự đoán trên tập kiểm thử
y_pred_id3 = id3_model.predict(X_test_encoded)

# Đánh giá mô hình
print("Accuracy:", accuracy_score(y_test_id3, y_pred_id3))

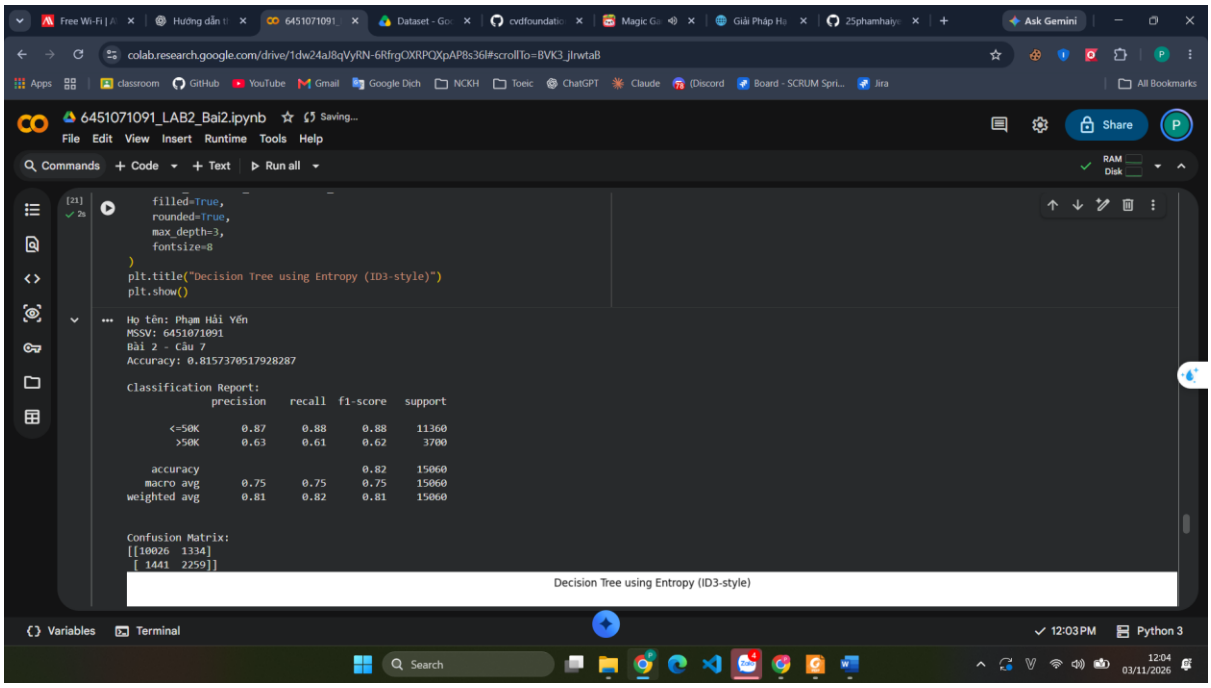
print("\nClassification Report:")
print(classification_report(y_test_id3, y_pred_id3))

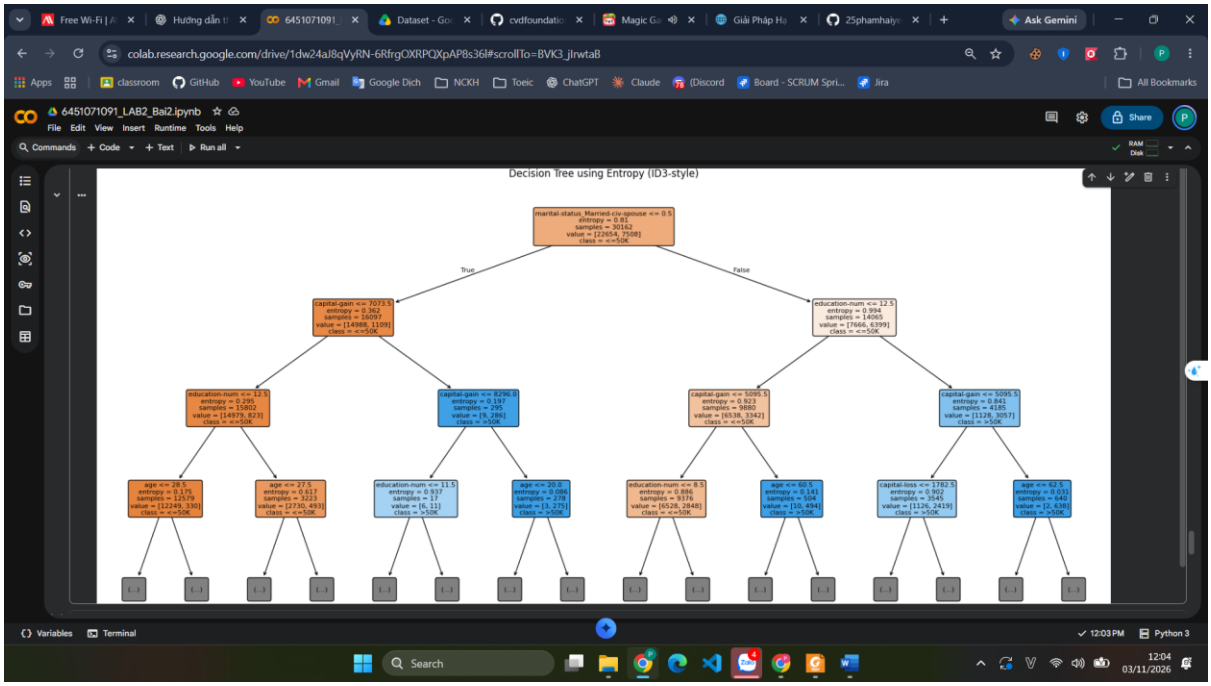
print("\nConfusion Matrix:")
cm = confusion_matrix(
    y_test_id3,
    y_pred_id3,
    labels=["<=50K", ">50K"]
)
print(cm)

# Biểu diễn cây quyết định
plt.figure(figsize=(22, 10))
plot_tree(
    id3_model,
    feature_names=X_train_encoded.columns,
    class_names=id3_model.classes_,
    filled=True,
    rounded=True,
    max_depth=3,
    fontsize=8
```

```
)
plt.title("Decision Tree using Entropy (ID3-style)")
plt.show()
```

//Chụp hình kết quả





Câu 8:

//dán mã nguồn

```
# BÀI 2 - CÂU 8
```

```
# XÂY DỰNG CÂY CART VÀ KIỂM THỬ


import matplotlib.pyplot as plt

from sklearn.tree import DecisionTreeClassifier, plot_tree

from sklearn.metrics import confusion_matrix, classification_report,
accuracy_score


print("Họ tên: Phạm Hải Yến")

print("MSSV: 6451071091")

print("Bài 2 - Câu 8 - Cây CART")

print("-" * 50)


# 1. Xây dựng mô hình CART

# CART sử dụng Gini làm tiêu chí phân chia mặc định


cart_model = DecisionTreeClassifier(

    criterion="gini",

    random_state=42

)


# 2. Huấn luyện mô hình

cart_model.fit(X_train_encoded, y_train)


# 3. Dự đoán trên tập kiểm thử

y_pred_cart = cart_model.predict(X_test_encoded)


# 4. Đánh giá mô hình

print("Độ chính xác CART:",

      accuracy_score(y_test, y_pred_cart))


print("\nMa trận nhầm lẫn:")

print(confusion_matrix(y_test, y_pred_cart))
```

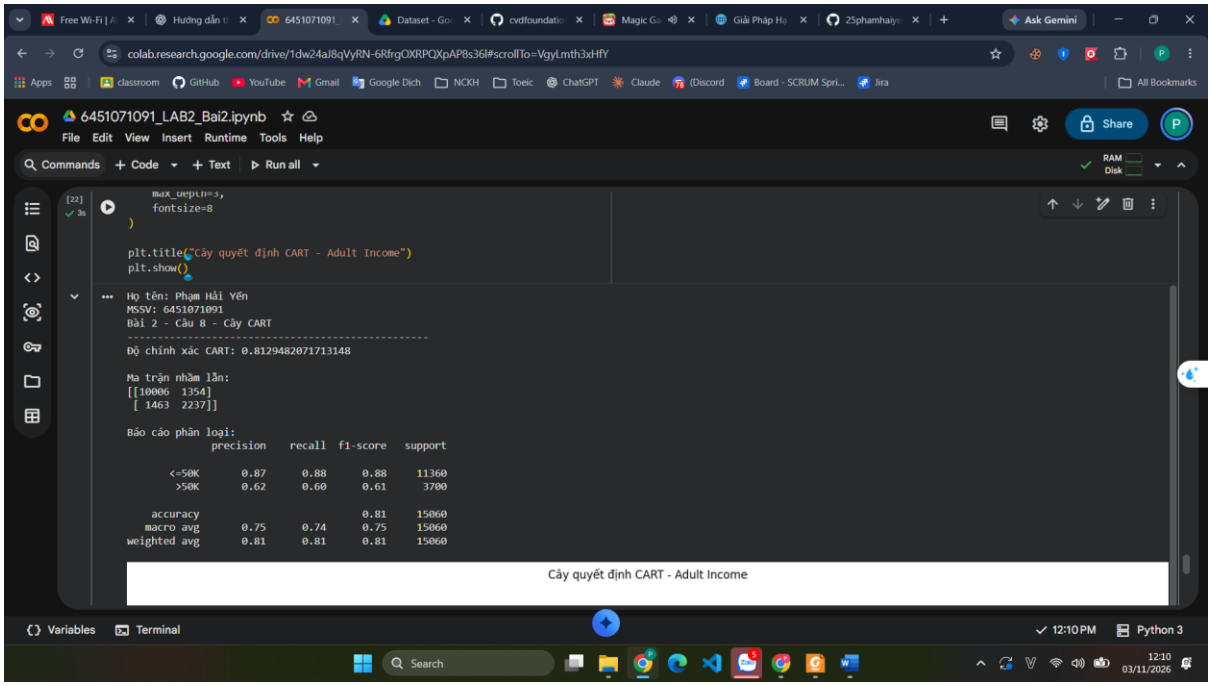
```
print("\nBáo cáo phân loại:")
print(classification_report(y_test, y_pred_cart))

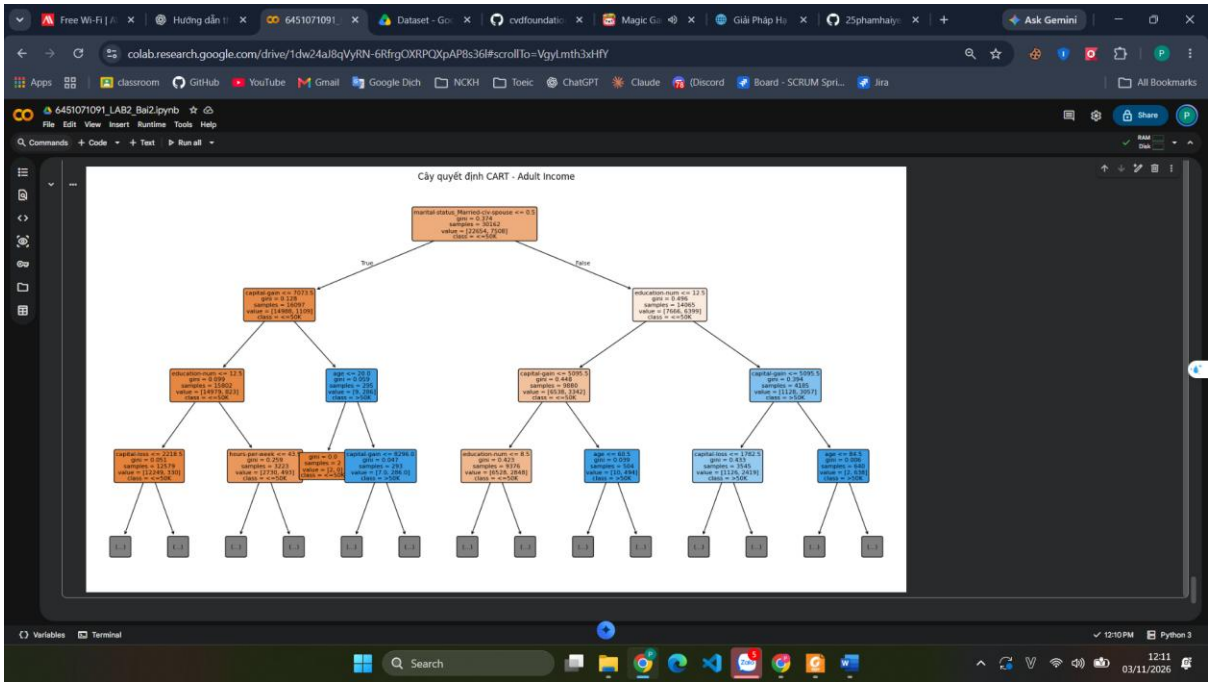
# 5. Biểu diễn cây CART
plt.figure(figsize=(20, 10))

plot_tree(
    cart_model,
    feature_names=X_train_encoded.columns,
    class_names=cart_model.classes_,
    filled=True,
    rounded=True,
    max_depth=3,
    fontsize=8
)

plt.title("Cây quyết định CART - Adult Income")
plt.show()
```

//Chụp hình kết quả





Câu 9:

//dán mã nguồn

```
# BÀI 2 - CÂU 9
# XÂY DỰNG MÔ HÌNH NAIVE BAYES

import matplotlib.pyplot as plt
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report,
    ConfusionMatrixDisplay
)

print("Họ tên: Phạm Hải Yến")
print("MSSV: 6451071091")
print("Bài 2 - Câu 9 - Naive Bayes")
print("-" * 50)

# 1. Khởi tạo mô hình Naive Bayes
nb_model = GaussianNB()
```

```
# 2. Huấn luyện mô hình
nb_model.fit(X_train_encoded, y_train)

# 3. Dự đoán trên tập kiểm thử
y_pred_nb = nb_model.predict(X_test_encoded)

# 4. Đánh giá mô hình
accuracy_nb = accuracy_score(y_test, y_pred_nb)

print("Độ chính xác Naive Bayes:",
      accuracy_nb)

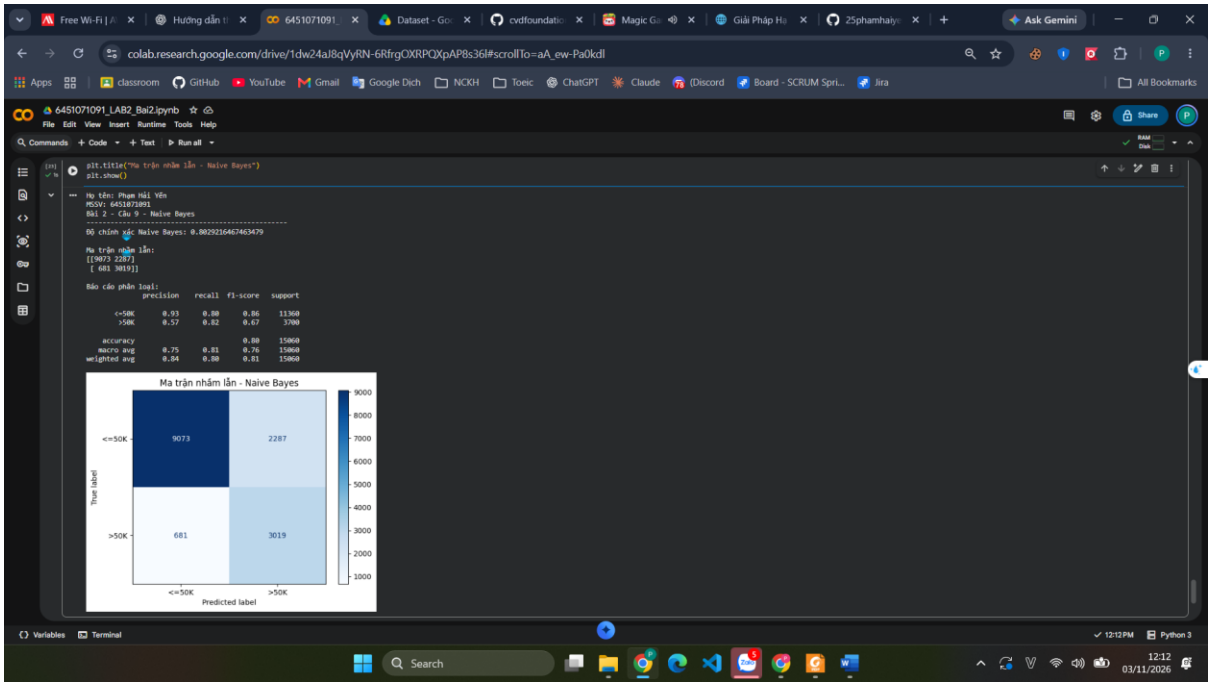
print("\nMa trận nhầm lẫn:")
cm_nb = confusion_matrix(y_test, y_pred_nb)
print(cm_nb)

print("\nBáo cáo phân loại:")
print(classification_report(y_test, y_pred_nb))

# 5. Biểu diễn ma trận nhầm lẫn
disp = ConfusionMatrixDisplay(
    confusion_matrix=cm_nb,
    display_labels=nb_model.classes_
)

disp.plot(cmap="Blues")
plt.title("Ma trận nhầm lẫn - Naive Bayes")
plt.show()
```

//Chụp hình kết quả



Câu 10:

//dán mã nguồn

```
# BÀI 2 - CÂU 10
# SO SÁNH KẾT QUẢ CÁC MÔ HÌNH

import pandas as pd
import matplotlib.pyplot as plt
from sklearn.metrics import accuracy_score

print("Họ tên: Phạm Hải Yến")
print("MSSV: 6451071091")
print("Bài 2 - Câu 10 - So sánh mô hình")
print("-" * 50)

# 1. Tính độ chính xác của từng mô hình
accuracy_id3 = accuracy_score(
    y_test,
    y_pred_id3
)

accuracy_cart = accuracy_score(
```

```
    y_test,
    y_pred_cart
)

accuracy_nb = accuracy_score(
    y_test,
    y_pred_nb
)

# 2. Tạo bảng so sánh
comparison = pd.DataFrame({
    "Mô hình": [
        "ID3",
        "CART",
        "Naive Bayes"
    ],
    "Độ chính xác": [
        accuracy_id3,
        accuracy_cart,
        accuracy_nb
    ]
})

print("Bảng so sánh độ chính xác:")
display(comparison)

# 3. Vẽ biểu đồ so sánh
plt.figure(figsize=(8, 5))

plt.bar(
    comparison["Mô hình"],
    comparison["Độ chính xác"]
)
```

```
plt.title("So sánh độ chính xác các mô hình")
plt.xlabel("Mô hình")
plt.ylabel("Accuracy")
plt.ylim(0, 1)

for i, value in enumerate(comparison["Độ chính xác"]):
    plt.text(
        i,
        value + 0.01,
        f"{value:.4f}",
        ha="center"
    )

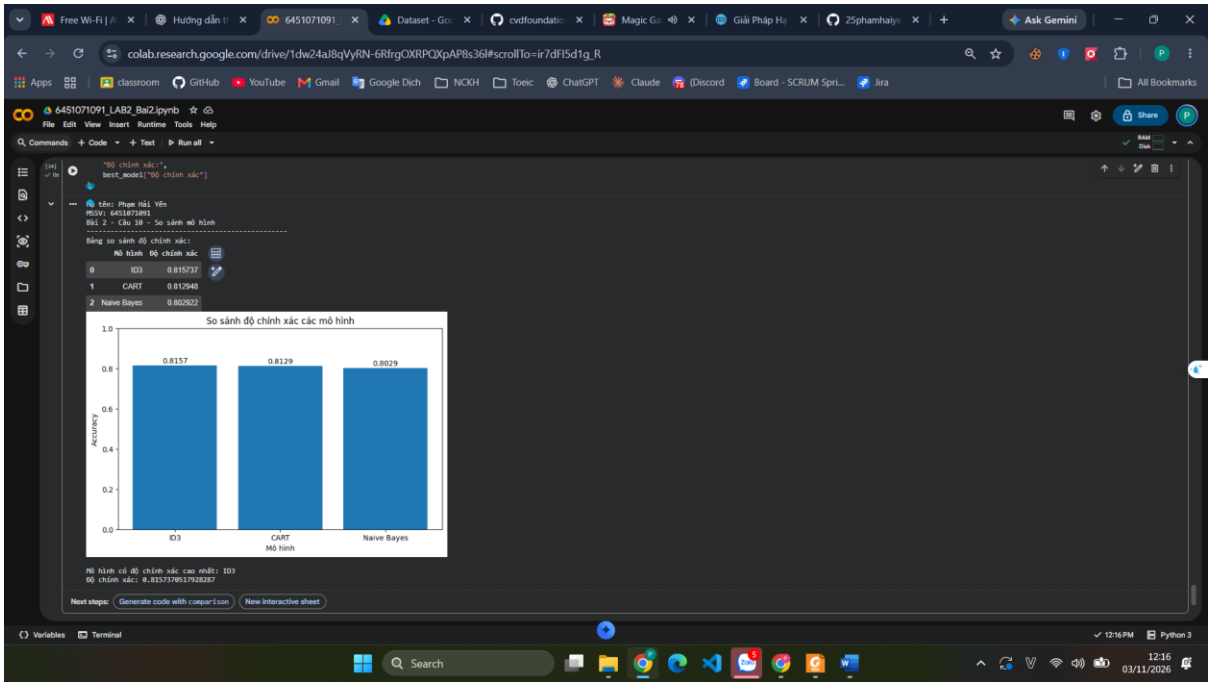
plt.show()

# 4. Xác định mô hình có độ chính xác cao nhất
best_model = comparison.loc[
    comparison["Độ chính xác"].idxmax()
]

print(
    "\nMô hình có độ chính xác cao nhất:",
    best_model["Mô hình"]
)

print(
    "Độ chính xác:",
    best_model["Độ chính xác"]
)
```

//Chụp hình kết quả



Bài 4 (Trang 26)

Câu 3:

//dán mã nguồn

```
# BÀI 4 - CÂU 3
# UPLOAD, ĐỌC VÀ KHẢO SÁT DỮ LIỆU RED WINE QUALITY

import pandas as pd
import numpy as np
from google.colab import files

print("Họ tên: Phạm Hải Yến")
print("MSSV: 6451071091")
print("Bài 4 - Câu 3")
print("-" * 50)

# 1. Upload file dữ liệu
uploaded = files.upload()

file_name = list(uploaded.keys())[0]
```

```
print("\nFile đã upload:", file_name)

# 2. Đọc dữ liệu
# File hiện tại sử dụng dấu phẩy để phân cách
wine_df = pd.read_csv(
    file_name,
    sep=","
)

# 3. Khảo sát dữ liệu
print("\nKích thước dữ liệu:", wine_df.shape)

print("\nTên các thuộc tính:")
print(wine_df.columns.tolist())

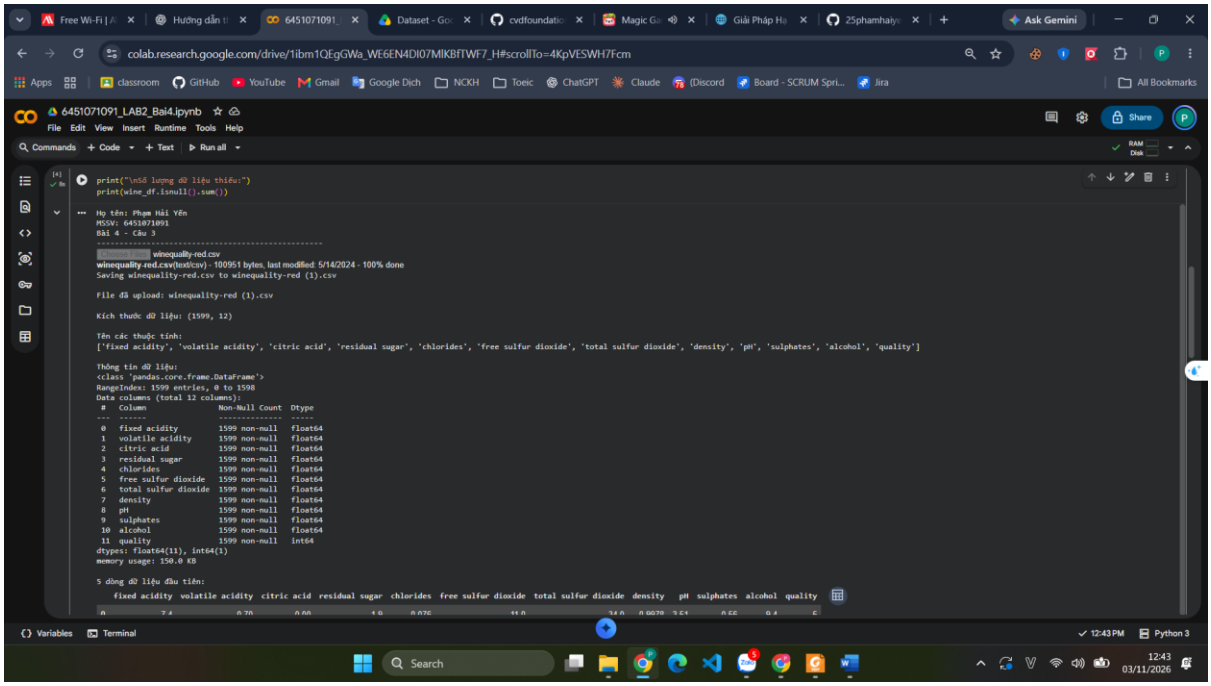
print("\nThông tin dữ liệu:")
wine_df.info()

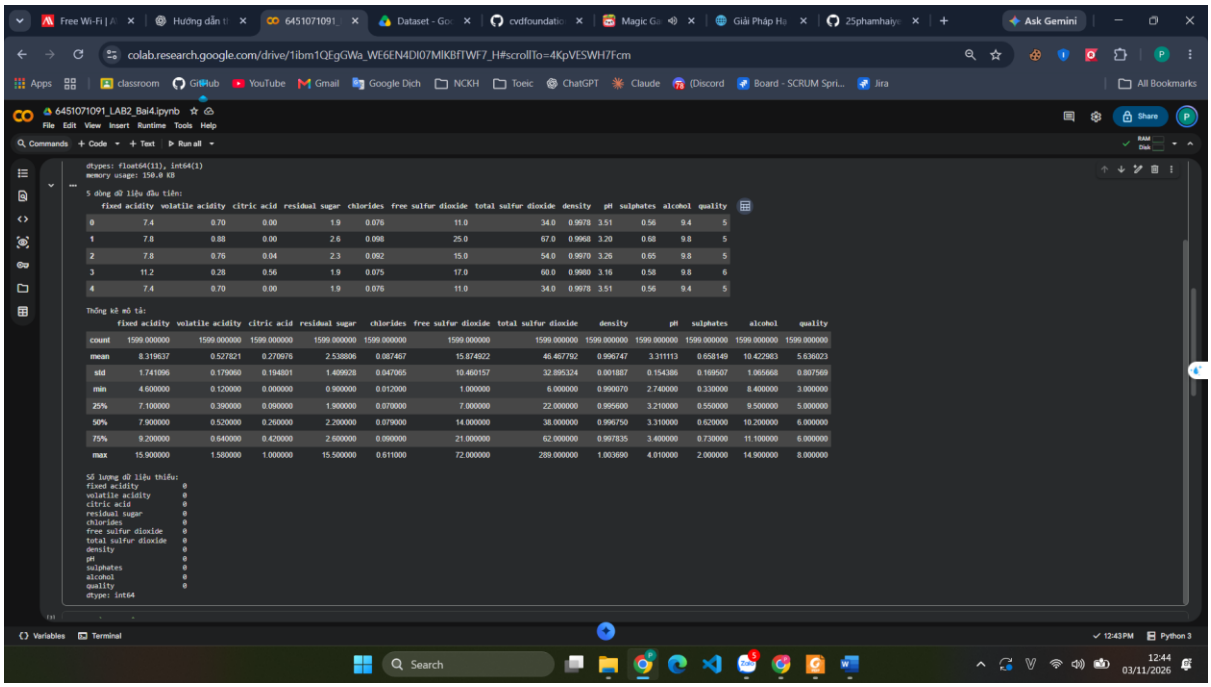
print("\n5 dòng dữ liệu đầu tiên:")
display(wine_df.head())

print("\nThống kê mô tả:")
display(wine_df.describe())

print("\nSố lượng dữ liệu thiếu:")
print(wine_df.isnull().sum())
```

//Chụp hình kết quả





Câu 4:

//dán mã nguồn

```
# BÀI 4 - CÂU 4

# TIỀN XỬ LÝ VÀ KHẢO SÁT TƯƠNG QUAN


import matplotlib.pyplot as plt

import seaborn as sns
```

```
print("Họ tên: Phạm Hải Yến")
print("MSSV: 6451071091")
print("Bài 4 - Câu 4")
print("-" * 50)

# Xóa dòng có dữ liệu thiếu
wine_df = wine_df.dropna().copy()

# Kiểm tra dữ liệu trùng lặp
print("Số dòng trùng lặp:",
      wine_df.duplicated().sum())

# Tính tương quan Pearson
corr_matrix = wine_df.corr(method="pearson")

print("\nMa trận tương quan:")
display(corr_matrix)

# Vẽ heatmap
plt.figure(figsize=(12, 8))

sns.heatmap(
    corr_matrix,
    annot=True,
    cmap="coolwarm",
    fmt=".2f"
)

plt.title("Ma trận tương quan Pearson - Red Wine Quality")
plt.show()

# Tìm các cặp thuộc tính có tương quan cao
threshold = 0.8
```

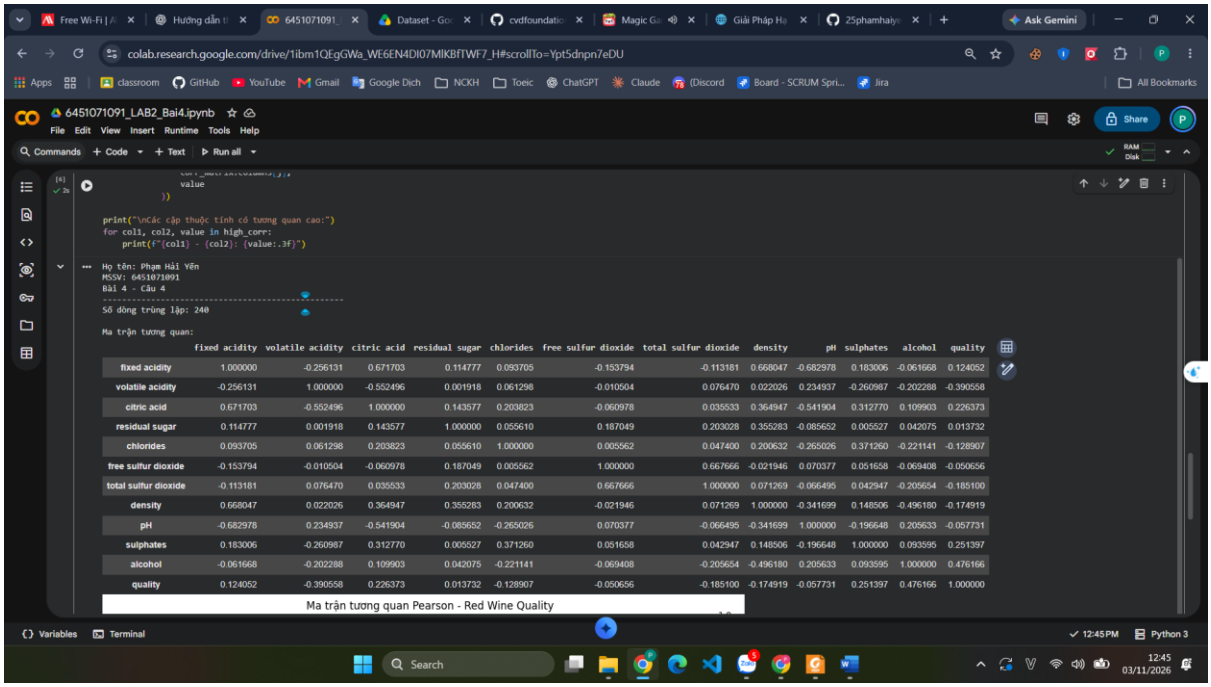
```
high_corr = []

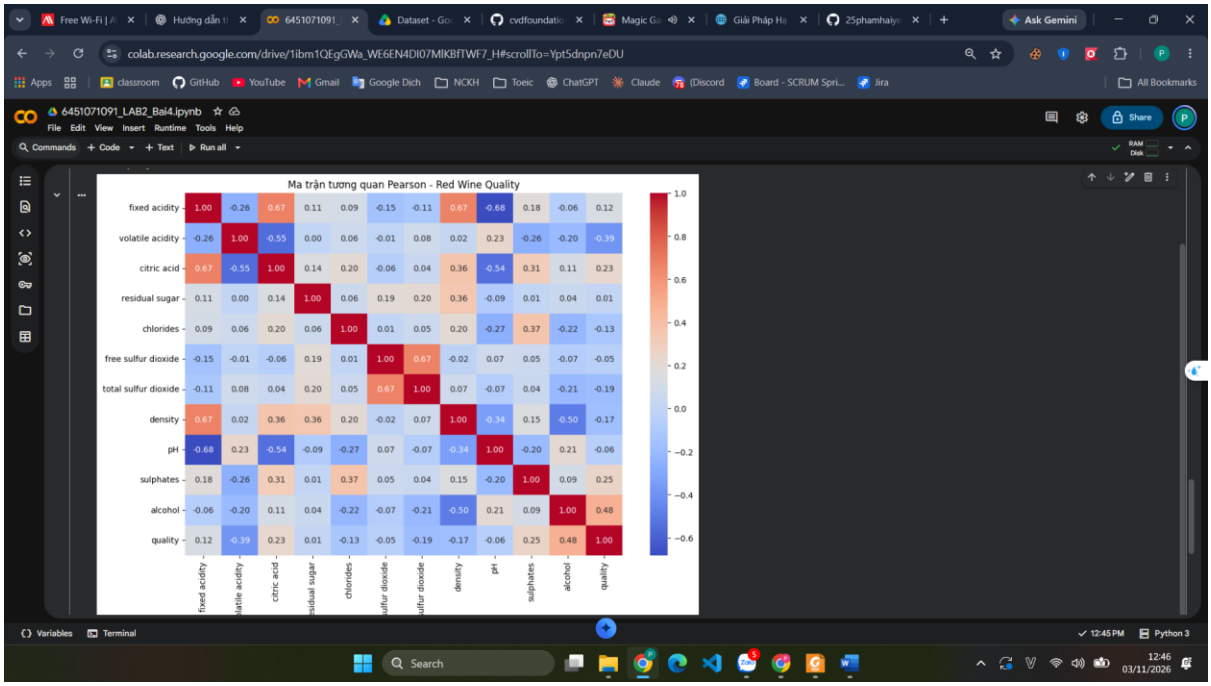
for i in range(len(corr_matrix.columns)):
    for j in range(i + 1, len(corr_matrix.columns)):
        value = corr_matrix.iloc[i, j]

        if abs(value) >= threshold:
            high_corr.append((
                corr_matrix.columns[i],
                corr_matrix.columns[j],
                value
            ))

print("\nCác cặp thuộc tính có tương quan cao:")
for col1, col2, value in high_corr:
    print(f"{col1} - {col2}: {value:.3f}")
```

//Chụp hình kết quả





Câu 5:

//dán mã nguồn

```
# BÀI 4 - CÂU 5

# TÁCH X VÀ y


print("Họ tên: Phạm Hải Yến")

print("MSSV: 6451071091")

print("Bài 4 - Câu 5")

print("-" * 50)


# Thuộc tính quyết định

target_column = "quality"


# X: các thuộc tính đầu vào

X = wine_df.drop(columns=[target_column])


# y: nhãn cần dự đoán

y = wine_df[target_column]


print("Kích thước X:", X.shape)
```

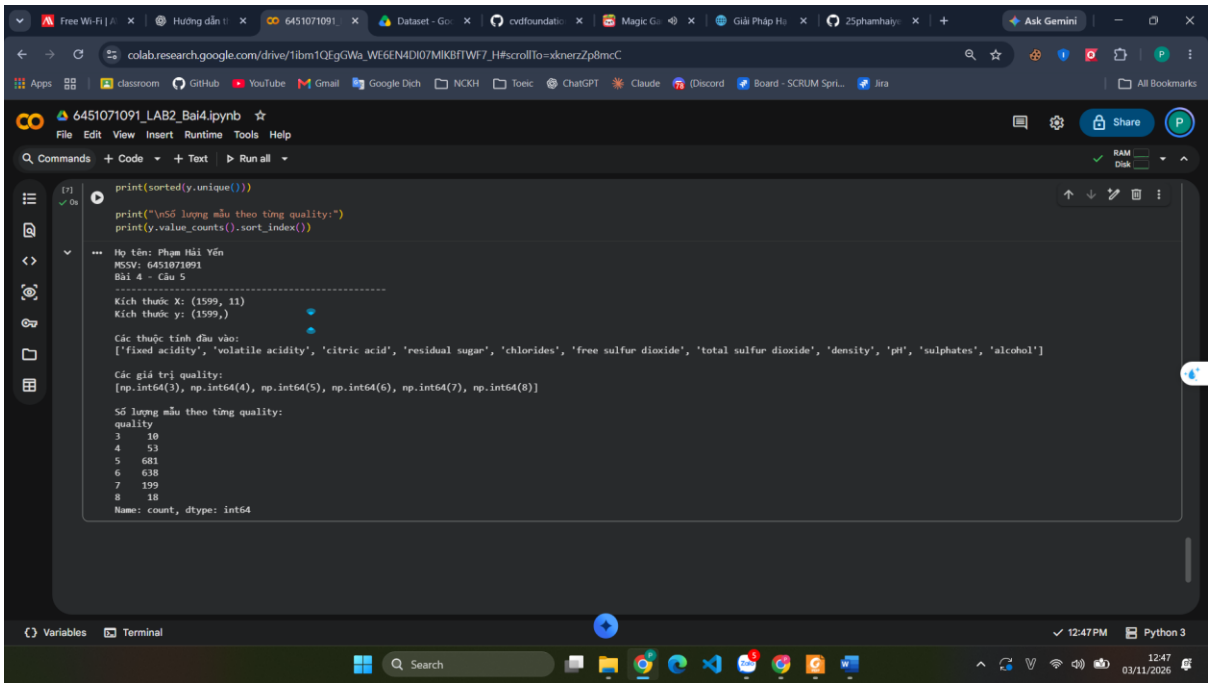
```
print("Kích thước y:", y.shape)


print("\nCác thuộc tính đầu vào:")
print(X.columns.tolist())


print("\nCác giá trị quality:")
print(sorted(y.unique()))


print("\nSố lượng mẫu theo từng quality:")
print(y.value_counts().sort_index())
```

//Chụp hình kết quả



Câu 6:

//dán mã nguồn

```
# BÀI 4 - CÂU 6

# CHIA DỮ LIỆU TRAIN VÀ TEST


from sklearn.model_selection import train_test_split


print("Họ tên: Phạm Hải Yến")
```

```
print("MSSV: 6451071091")
print("Bài 4 - Câu 6")
print("-" * 50)

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.3,
    random_state=42,
    stratify=y
)

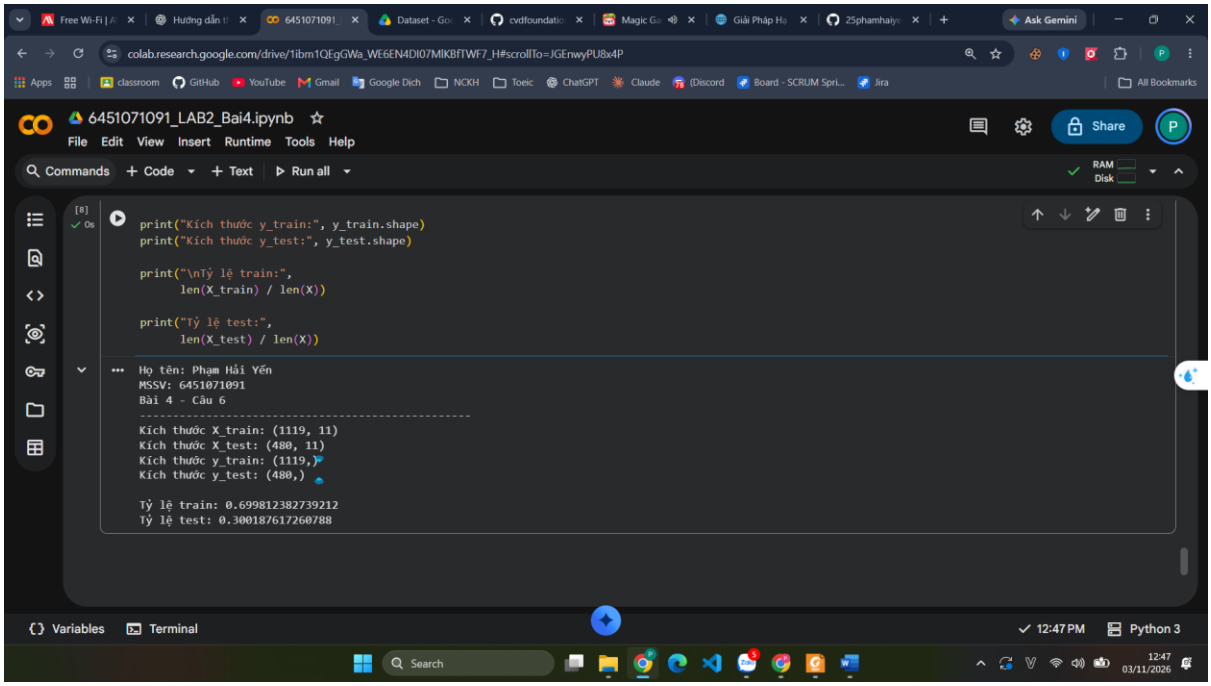
print("Kích thước X_train:", X_train.shape)
print("Kích thước X_test:", X_test.shape)

print("Kích thước y_train:", y_train.shape)
print("Kích thước y_test:", y_test.shape)

print("\nTỷ lệ train:",
      len(X_train) / len(X))

print("Tỷ lệ test:",
      len(X_test) / len(X))
```

//Chụp hình kết quả



Câu 7:

//dán mã nguồn

```
# BÀI 4 - CÂU 7
# XÂY DỰNG CÂY QUYẾT ĐỊNH ID3

import matplotlib.pyplot as plt

from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report
)

print("Họ tên: Phạm Hải Yến")
print("MSSV: 6451071091")
print("Bài 4 - Câu 7 - ID3")
print("-" * 50)

# Khởi tạo mô hình ID3
id3_model = DecisionTreeClassifier(
```

```
    criterion="entropy",
    random_state=42
)

# Huấn luyện
id3_model.fit(X_train, y_train)

# Dự đoán
y_pred_id3 = id3_model.predict(X_test)

# Độ chính xác
accuracy_id3 = accuracy_score(
    y_test,
    y_pred_id3
)

print("Độ chính xác ID3:", accuracy_id3)

print("\nMa trận nhầm lẫn:")
print(confusion_matrix(y_test, y_pred_id3))

print("\nBáo cáo phân loại:")
print(classification_report(
    y_test,
    y_pred_id3,
    zero_division=0
))

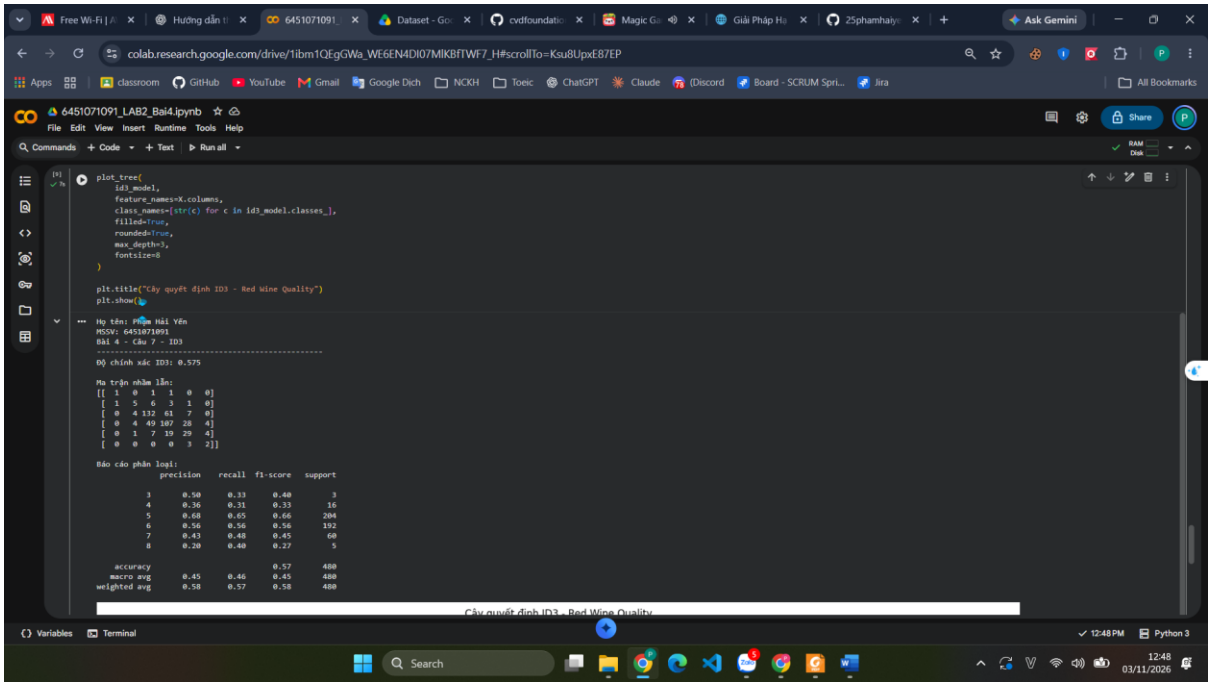
# Vẽ cây quyết định
plt.figure(figsize=(20, 10))

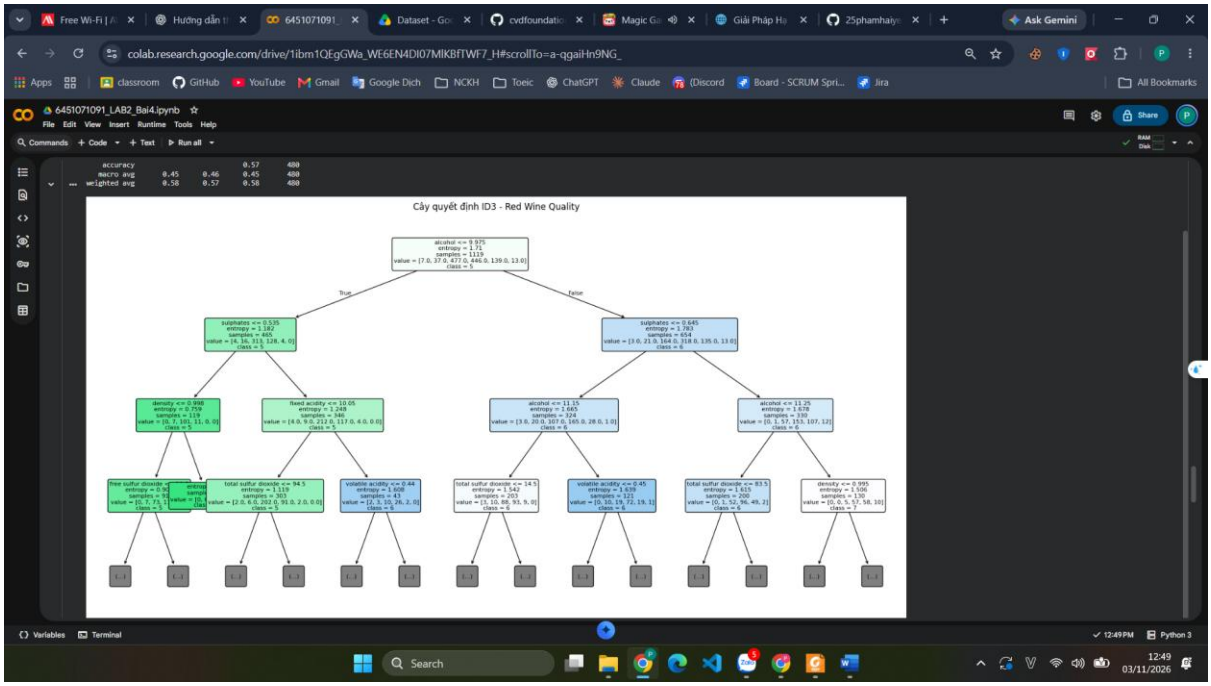
plot_tree(
    id3_model,
```

```
    feature_names=X.columns,
    class_names=[str(c) for c in id3_model.classes_],
    filled=True,
    rounded=True,
    max_depth=3,
    fontsize=8
)

plt.title("Cây quyết định ID3 - Red Wine Quality")
plt.show()
```

//Chụp hình kết quả

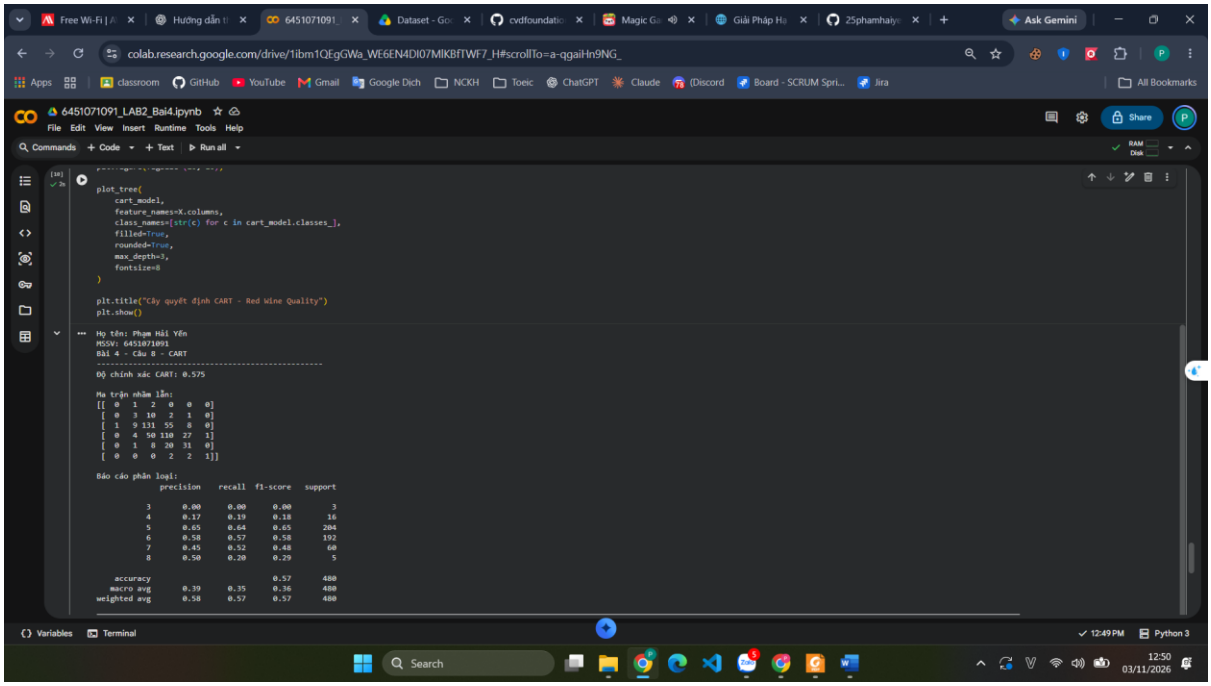


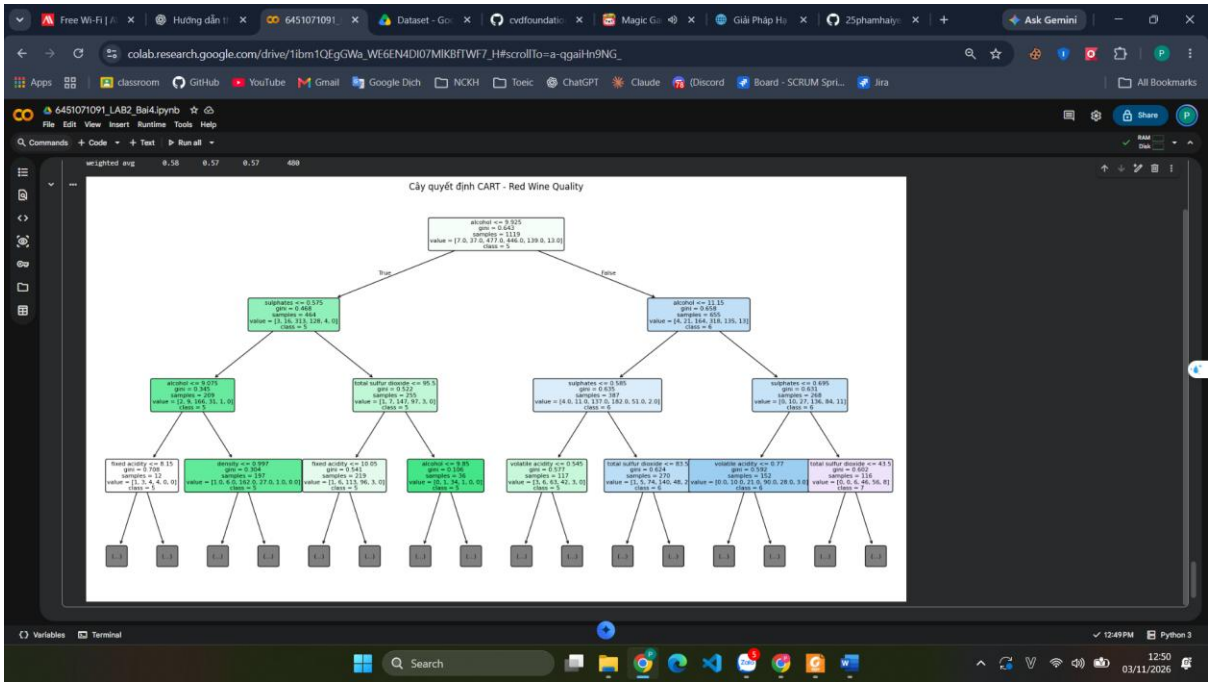


Câu 8:

//dán mã nguồn

//Chụp hình kết quả





Câu 9:

//dán mã nguồn

```
# BÀI 4 - CÂU 9
# XÂY DỰNG MÔ HÌNH NAIVE BAYES

from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report,
    ConfusionMatrixDisplay
)

import matplotlib.pyplot as plt

print("Họ tên: Phạm Hải Yến")
print("MSSV: 6451071091")
print("Bài 4 - Câu 9 - Naive Bayes")
print("-" * 50)
```

```
# Khởi tạo mô hình
nb_model = GaussianNB()

# Huấn luyện
nb_model.fit(X_train, y_train)

# Dự đoán
y_pred_nb = nb_model.predict(X_test)

# Đánh giá
accuracy_nb = accuracy_score(
    y_test,
    y_pred_nb
)

print("Độ chính xác Naive Bayes:", accuracy_nb)

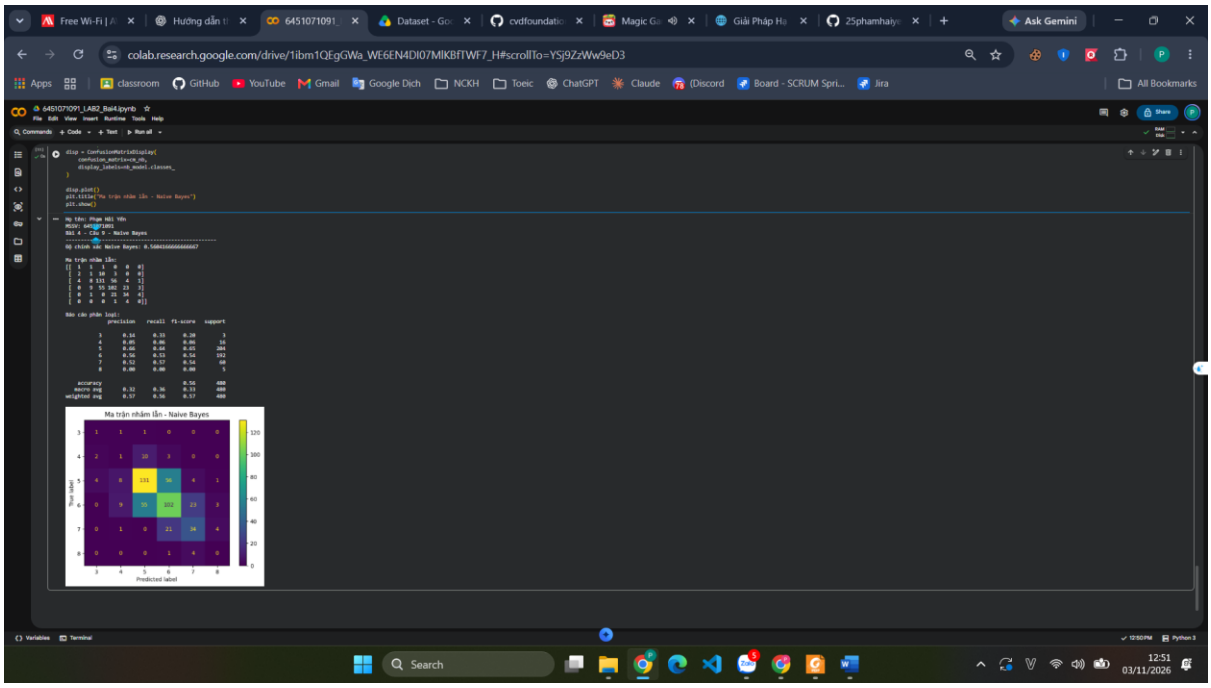
print("\nMa trận nhầm lẫn:")
cm_nb = confusion_matrix(y_test, y_pred_nb)
print(cm_nb)

print("\nBáo cáo phân loại:")
print(classification_report(
    y_test,
    y_pred_nb,
    zero_division=0
))

# Vẽ ma trận nhầm lẫn
disp = ConfusionMatrixDisplay(
    confusion_matrix=cm_nb,
    display_labels=nb_model.classes_
)
```

```
disp.plot()
plt.title("Ma trận nhầm lẫn - Naive Bayes")
plt.show()
```

//Chụp hình kết quả



Câu 10:

//dán mã nguồn

```
# BÀI 4 - CÂU 10
# SO SÁNH CÁC MÔ HÌNH

import pandas as pd
import matplotlib.pyplot as plt

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score
)
```

```
print("Họ tên: Phạm Hải Yến")
print("MSSV: 6451071091")
print("Bài 4 - Câu 10")
print("-" * 50)

# Danh sách mô hình và kết quả dự đoán
models = {
    "ID3": y_pred_id3,
    "CART": y_pred_cart,
    "Naive Bayes": y_pred_nb
}

# Tính các chỉ số đánh giá
results = []

for model_name, y_pred in models.items():
    results.append({
        "Mô hình": model_name,
        "Accuracy": accuracy_score(y_test, y_pred),
        "Precision": precision_score(
            y_test,
            y_pred,
            average="weighted",
            zero_division=0
        ),
        "Recall": recall_score(
            y_test,
            y_pred,
            average="weighted",
            zero_division=0
        ),
        "F1-score": f1_score(
            y_test,
```

```
            y_pred,
            average="weighted",
            zero_division=0
        )
    })

comparison = pd.DataFrame(results)

print("Bảng so sánh kết quả:")
display(comparison)

# Vẽ biểu đồ so sánh Accuracy
plt.figure(figsize=(8, 5))

plt.bar(
    comparison["Mô hình"],
    comparison["Accuracy"]
)

plt.title("So sánh Accuracy các mô hình")
plt.xlabel("Mô hình")
plt.ylabel("Accuracy")
plt.ylim(0, 1)

for i, value in enumerate(comparison["Accuracy"]):
    plt.text(
        i,
        value + 0.01,
        f"{value:.4f}",
        ha="center"
    )

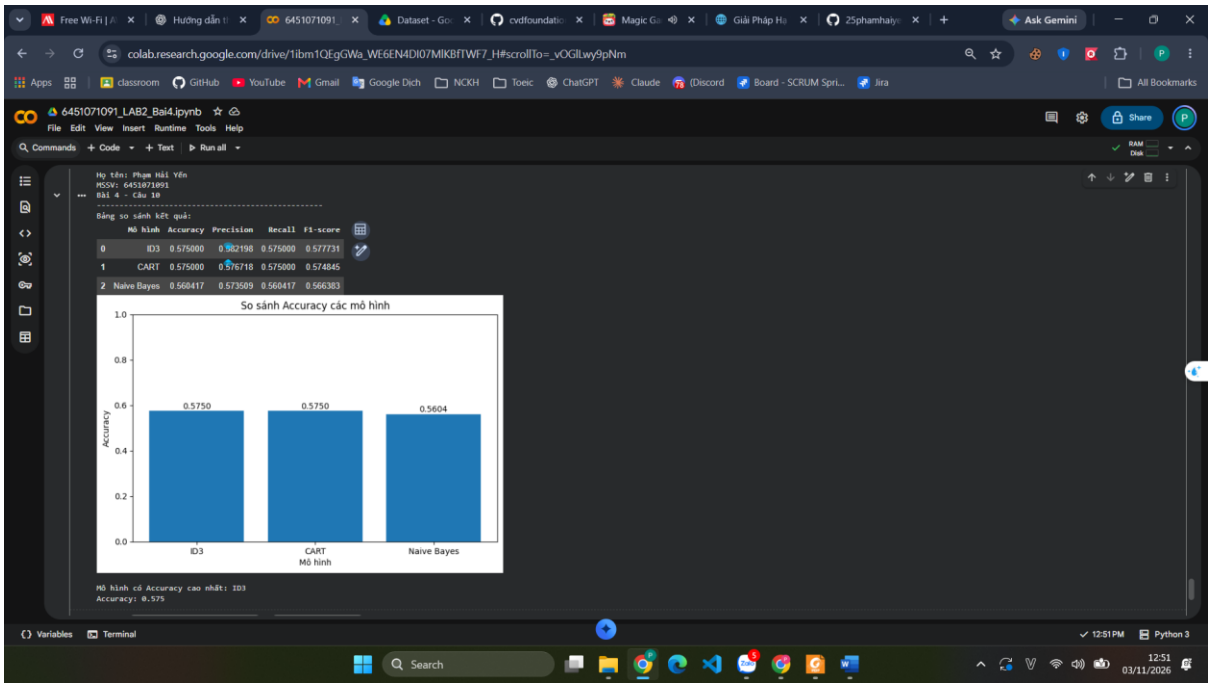
plt.show()
```

```
# Xác định mô hình tốt nhất theo Accuracy
best_model = comparison.loc[
    comparison["Accuracy"].idxmax()
]

print("\nMô hình có Accuracy cao nhất:",
      best_model["Mô hình"])

print("Accuracy:",
      best_model["Accuracy"])
```

//Chụp hình kết quả



Bài 5 (Trang 26, 27)

Câu a)

//dán mã nguồn

```
# BÀI 5 - CÂU a
# ĐỌC VÀ TIỀN XỬ LÝ DỮ LIỆU HEART DISEASE

import pandas as pd
import numpy as np
```

```
from google.colab import files


print("Họ tên: Phạm Hải Yến")
print("MSSV: 6451071091")
print("Bài 5 - Câu a")
print("-" * 50)

# 1. Upload file dữ liệu
uploaded = files.upload()
file_name = list(uploaded.keys())[0]

print("File đã upload:", file_name)

# 2. Đọc dữ liệu
heart_df = pd.read_csv(
    file_name,
    sep=None,
    engine="python"
)

# 3. Danh sách tên cột theo đề
column_names = [
    "age",
    "sex",
    "chest_pain_type",
    "resting_blood_pressure",
    "cholesterol",
    "fasting_blood_sugar",
    "rest_ecg",
    "max_heart_rate_achieved",
    "exercise_induced_angina",
    "st_depression",
    "st_slope",
```

```
    "num_major_vessels",
    "thalassemia",
    "target"
]

# 4. Chuẩn hóa tên cột
heart_df.columns = heart_df.columns.astype(str).str.strip()

# Nếu file chưa có tên cột theo đề và có đúng 14 cột,
# gán tên cột theo tài liệu
if "target" not in heart_df.columns and heart_df.shape[1] == 14:
    heart_df.columns = column_names

# 5. Xử lý dữ liệu thiếu
heart_df = heart_df.replace("?", np.nan)
heart_df = heart_df.dropna().copy()

# 6. Chuyển dữ liệu sang kiểu số
for col in heart_df.columns:
    heart_df[col] = pd.to_numeric(
        heart_df[col],
        errors="coerce"
    )

heart_df = heart_df.dropna().copy()

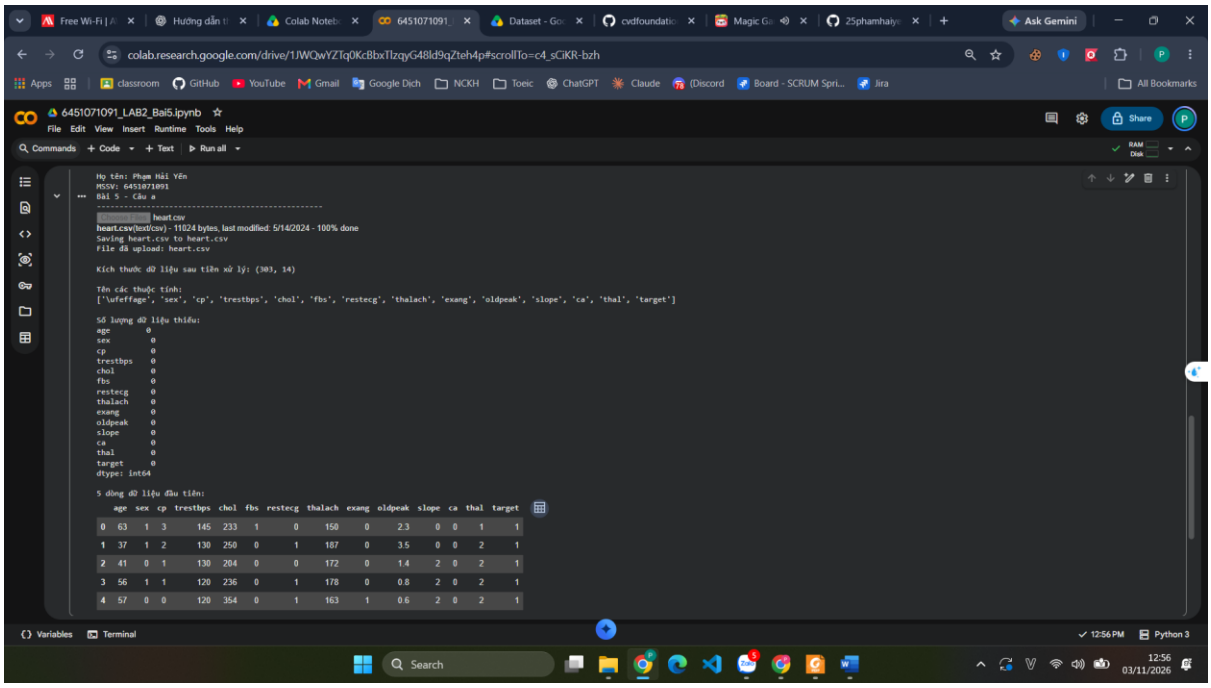
# 7. Hiển thị kết quả
print("\nKích thước dữ liệu sau tiền xử lý:",
      heart_df.shape)

print("\nTên các thuộc tính:")
print(heart_df.columns.tolist())
```

```
print("\nSố lượng dữ liệu thiếu:")
print(heart_df.isnull().sum())


print("\n5 dòng dữ liệu đầu tiên:")
display(heart_df.head())
```

//Chụp hình kết quả



Câu 3:

//dán mã nguồn

```
# BÀI 5 - CÂU 3
# KHẢO SÁT DỮ LIỆU HEART DISEASE


import pandas as pd
import matplotlib.pyplot as plt


print("Họ tên: Phạm Hải Yến")
print("MSSV: 6451071091")
print("Bài 5 - Câu 3")
print("-" * 50)


# Kích thước dữ liệu
```

```
print("Kích thước dữ liệu:", heart_df.shape)


# Thông tin dữ liệu
print("\nThông tin dữ liệu:")
heart_df.info()

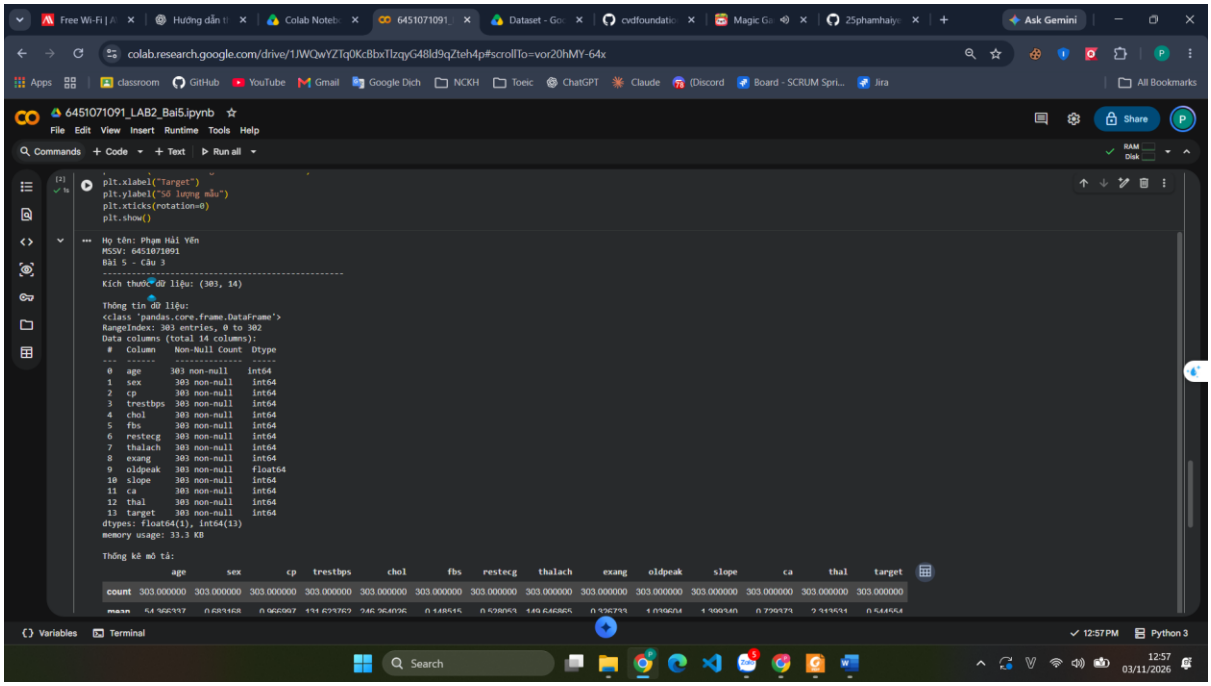

# Thống kê mô tả
print("\nThống kê mô tả:")
display(heart_df.describe())

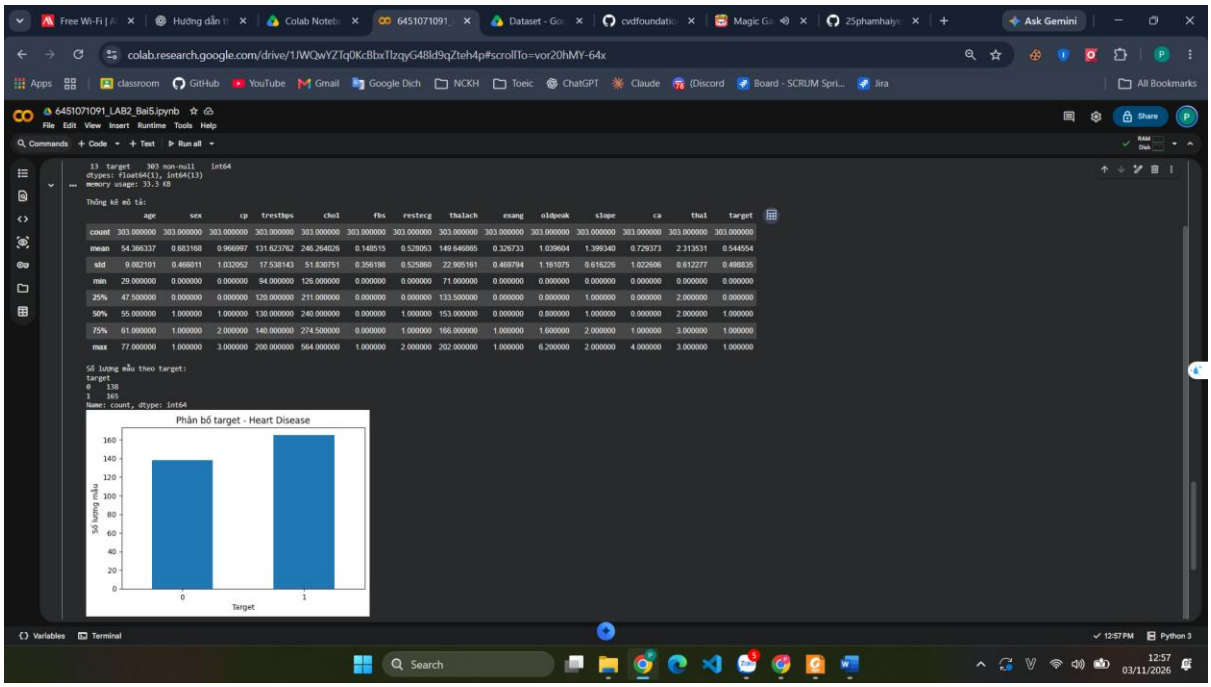

# Số lượng mẫu theo từng lớp target
print("\nSố lượng mẫu theo target:")
print(heart_df["target"].value_counts().sort_index())


# Biểu đồ phân bố target
heart_df["target"].value_counts().sort_index().plot(
    kind="bar",
    figsize=(6, 4)
)


plt.title("Phân bố target - Heart Disease")
plt.xlabel("Target")
plt.ylabel("Số lượng mẫu")
plt.xticks(rotation=0)
plt.show()
```

//Chụp hình kết quả





Câu 4:

//dán mã nguồn

```
# BÀI 5 - CÂU 4

# KHẢO SÁT TƯƠNG QUAN PEARSON


import matplotlib.pyplot as plt

import seaborn as sns
```

```
print("Họ tên: Phạm Hải Yến")
print("MSSV: 6451071091")
print("Bài 5 - Câu 4")
print("-" * 50)

# Tính ma trận tương quan
corr_matrix = heart_df.corr(method="pearson")

print("Ma trận tương quan:")
display(corr_matrix)

# Vẽ heatmap
plt.figure(figsize=(14, 10))

sns.heatmap(
    corr_matrix,
    annot=True,
    cmap="coolwarm",
    fmt=".2f",
    linewidths=0.5
)

plt.title("Ma trận tương quan Pearson - Heart Disease")
plt.show()

# Tìm các cặp thuộc tính có tương quan cao
threshold = 0.8
high_corr = []

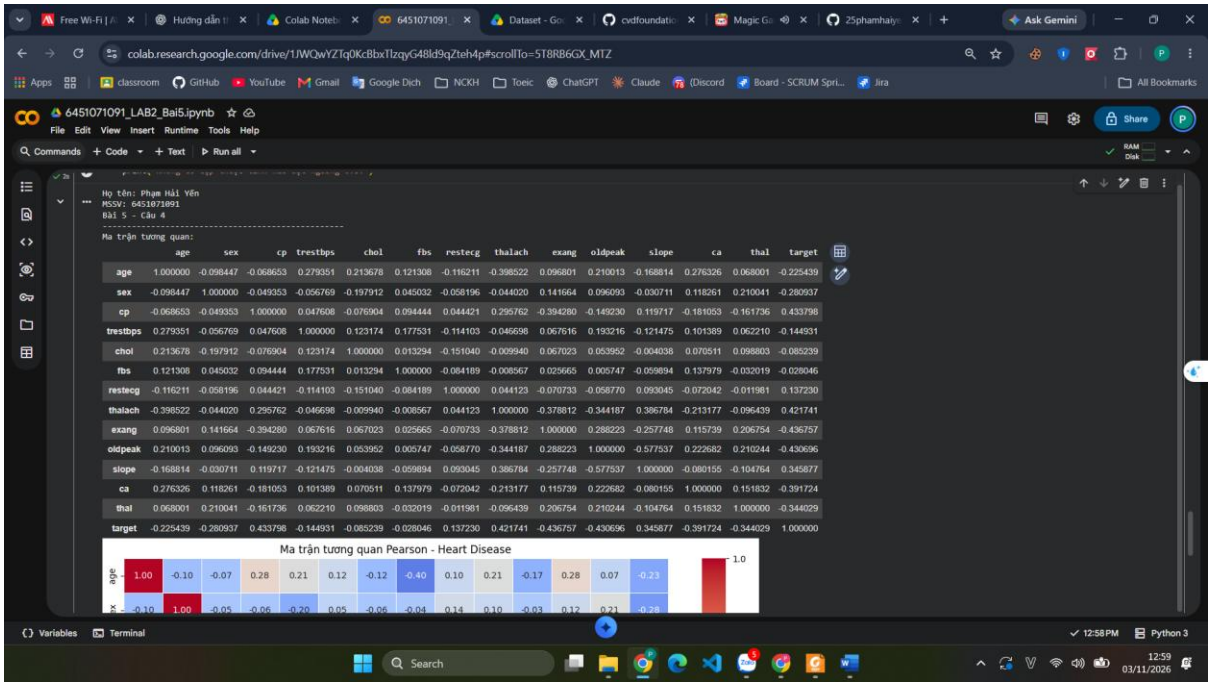
for i in range(len(corr_matrix.columns)):
    for j in range(i + 1, len(corr_matrix.columns)):
        value = corr_matrix.iloc[i, j]
```

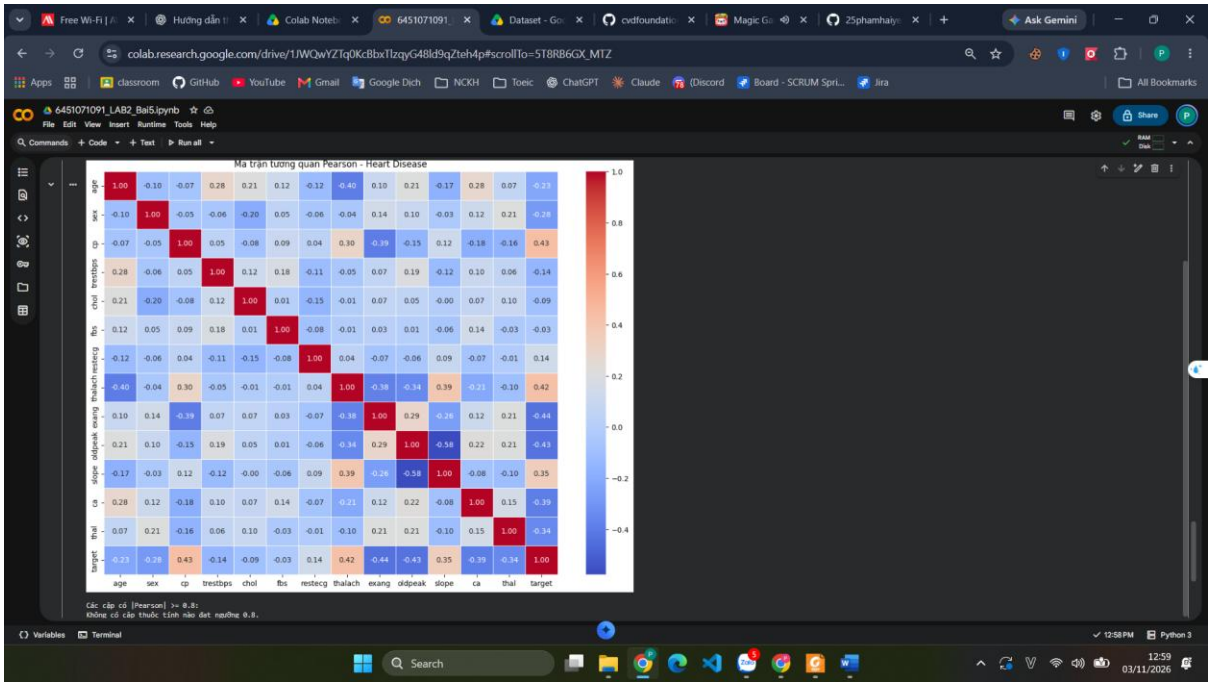
```
        if abs(value) >= threshold:
            high_corr.append({
                "Thuộc tính 1": corr_matrix.columns[i],
                "Thuộc tính 2": corr_matrix.columns[j],
                "Hệ số tương quan": value
            })

print("\nCác cặp có |Pearson| >= 0.8:")

if high_corr:
    display(pd.DataFrame(high_corr))
else:
    print("Không có cặp thuộc tính nào đạt ngưỡng 0.8.")
```

//Chụp hình kết quả





Câu 5:

//dán mã nguồn

```
# BÀI 5 - CÂU 5

# TÁCH X VÀ y


print("Họ tên: Phạm Hải Yến")

print("MSSV: 6451071091")

print("Bài 5 - Câu 5")

print("-" * 50)


# X: các thuộc tính đầu vào

X = heart_df.drop(columns=["target"])


# y: thuộc tính quyết định

y = heart_df["target"]


print("Kích thước X:", X.shape)

print("Kích thước y:", y.shape)

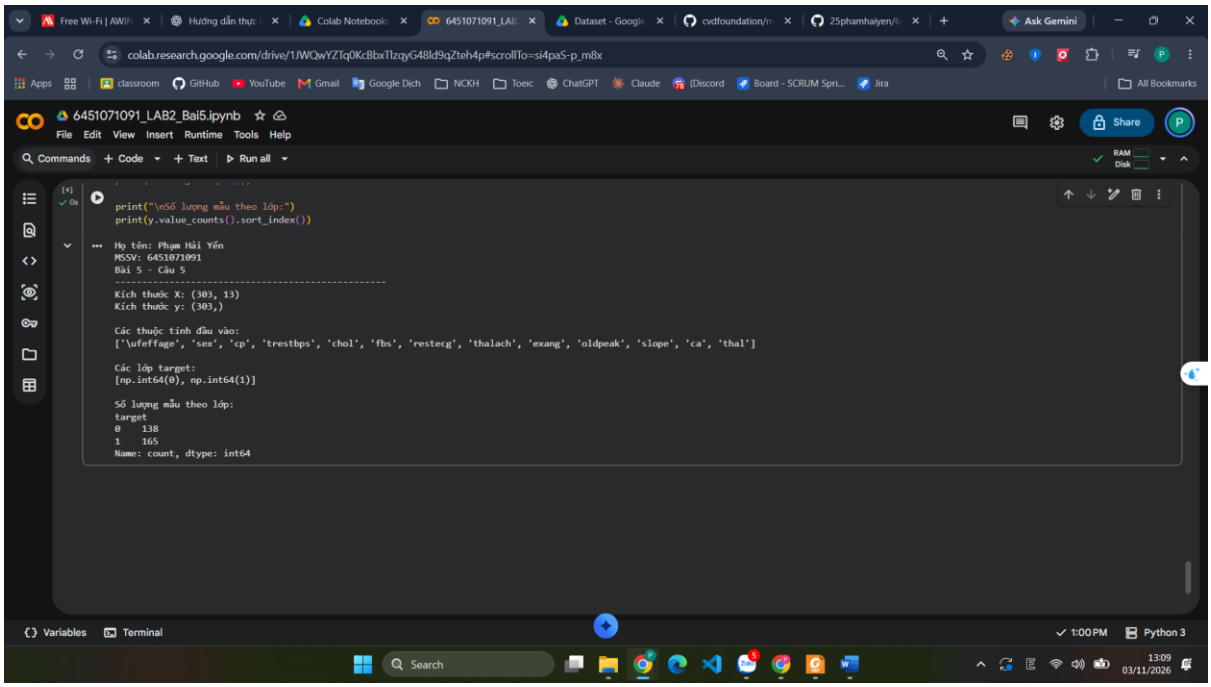

print("\nCác thuộc tính đầu vào:")
```

```
print(X.columns.tolist())


print("\nCác lớp target:")
print(sorted(y.unique()))


print("\nSố lượng mẫu theo lớp:")
print(y.value_counts().sort_index())
```

//Chụp hình kết quả



Câu 6:

//dán mã nguồn

```
# BÀI 5 - CÂU 6
# CHIA DỮ LIỆU TRAIN VÀ TEST


from sklearn.model_selection import train_test_split


print("Họ tên: Phạm Hải Yến")
print("MSSV: 6451071091")
print("Bài 5 - Câu 6")
print("-" * 50)
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.3,
    random_state=42,
    stratify=y
)

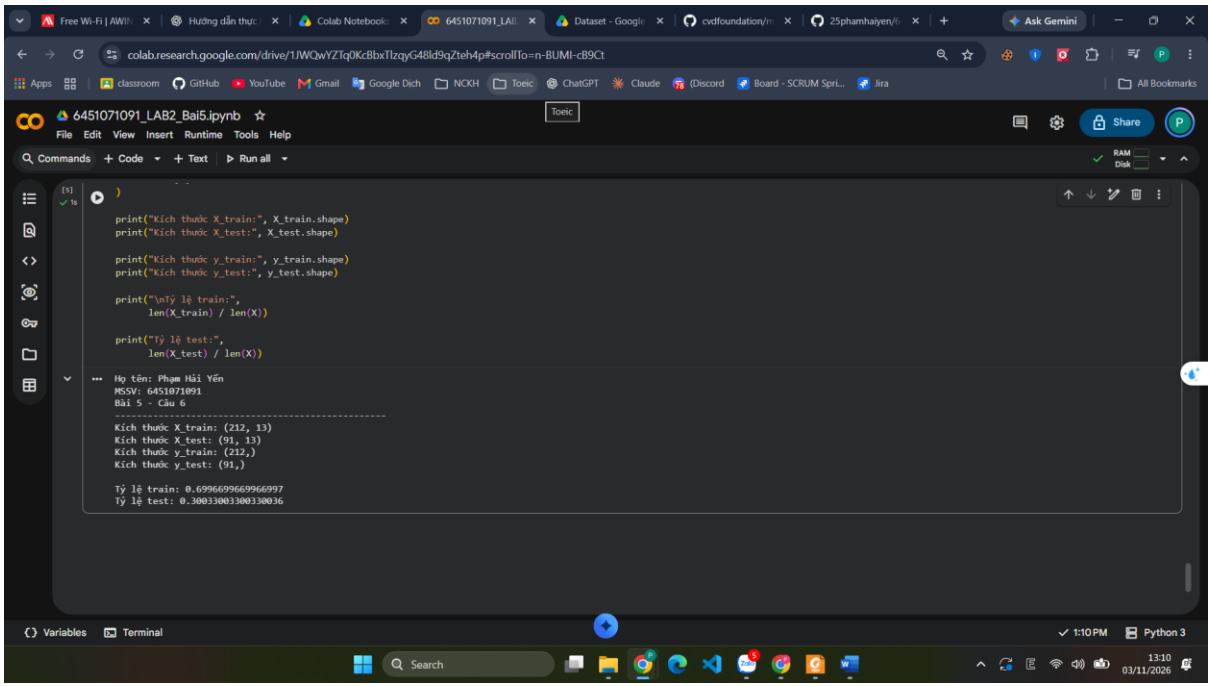
print("Kích thước X_train:", X_train.shape)
print("Kích thước X_test:", X_test.shape)

print("Kích thước y_train:", y_train.shape)
print("Kích thước y_test:", y_test.shape)

print("\nTỷ lệ train:",
      len(X_train) / len(X))

print("Tỷ lệ test:",
      len(X_test) / len(X))
```

//Chụp hình kết quả



Câu 7:

```
//dán mã nguồn
# BÀI 5 - CÂU 7
# XÂY DỰNG CÂY QUYẾT ĐỊNH ID3

import matplotlib.pyplot as plt

from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report
)

print("Họ tên: Phạm Hải Yến")
print("MSSV: 6451071091")
print("Bài 5 - Câu 7 - ID3")
print("-" * 50)

# Khởi tạo mô hình
id3_model = DecisionTreeClassifier(
    criterion="entropy",
    random_state=42
)

# Huấn luyện
id3_model.fit(X_train, y_train)

# Dự đoán
y_pred_id3 = id3_model.predict(X_test)

# Đánh giá
accuracy_id3 = accuracy_score(
```

```
    y_test,
    y_pred_id3
)

print("Độ chính xác ID3:", accuracy_id3)

print("\nMa trận nhầm lẫn:")
print(confusion_matrix(y_test, y_pred_id3))

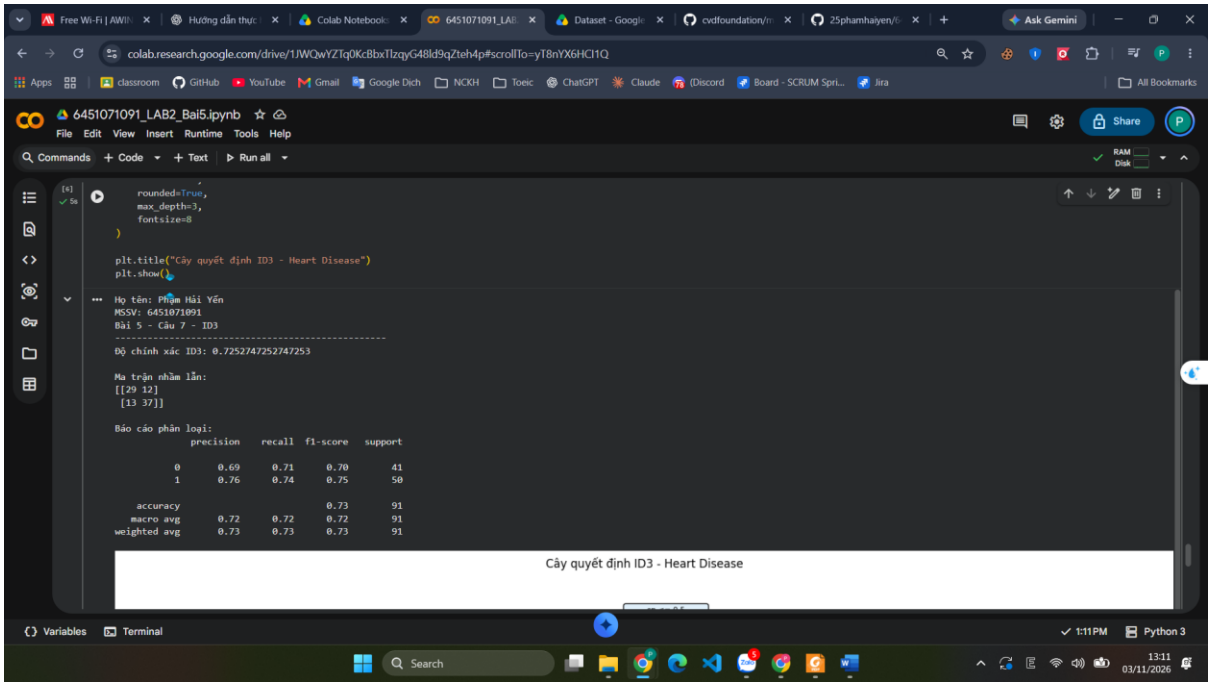
print("\nBáo cáo phân loại:")
print(classification_report(
    y_test,
    y_pred_id3,
    zero_division=0
))

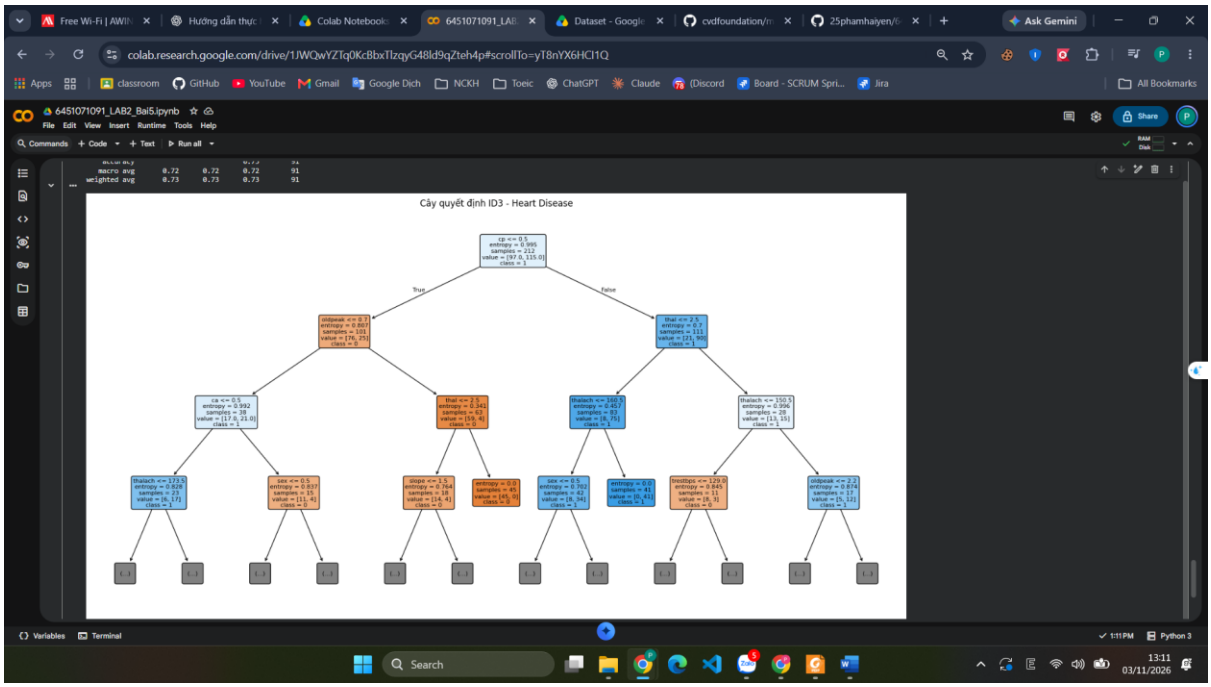
# Vẽ cây
plt.figure(figsize=(20, 10))

plot_tree(
    id3_model,
    feature_names=X.columns,
    class_names=[str(c) for c in id3_model.classes_],
    filled=True,
    rounded=True,
    max_depth=3,
    fontsize=8
)

plt.title("Cây quyết định ID3 - Heart Disease")
plt.show()
```

//Chụp hình kết quả





Câu 8:

//dán mã nguồn

```
# BÀI 5 - CÂU 8

# XÂY DỰNG CÂY QUYẾT ĐỊNH CART


import matplotlib.pyplot as plt


from sklearn.tree import DecisionTreeClassifier, plot_tree
```

```
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report
)

print("Họ tên: Phạm Hải Yến")
print("MSSV: 6451071091")
print("Bài 5 - Câu 8 - CART")
print("-" * 50)

# Khởi tạo mô hình CART
cart_model = DecisionTreeClassifier(
    criterion="gini",
    random_state=42
)

# Huấn luyện
cart_model.fit(X_train, y_train)

# Dự đoán
y_pred_cart = cart_model.predict(X_test)

# Đánh giá
accuracy_cart = accuracy_score(
    y_test,
    y_pred_cart
)

print("Độ chính xác CART:", accuracy_cart)

print("\nMa trận nhầm lẫn:")
print(confusion_matrix(y_test, y_pred_cart))
```

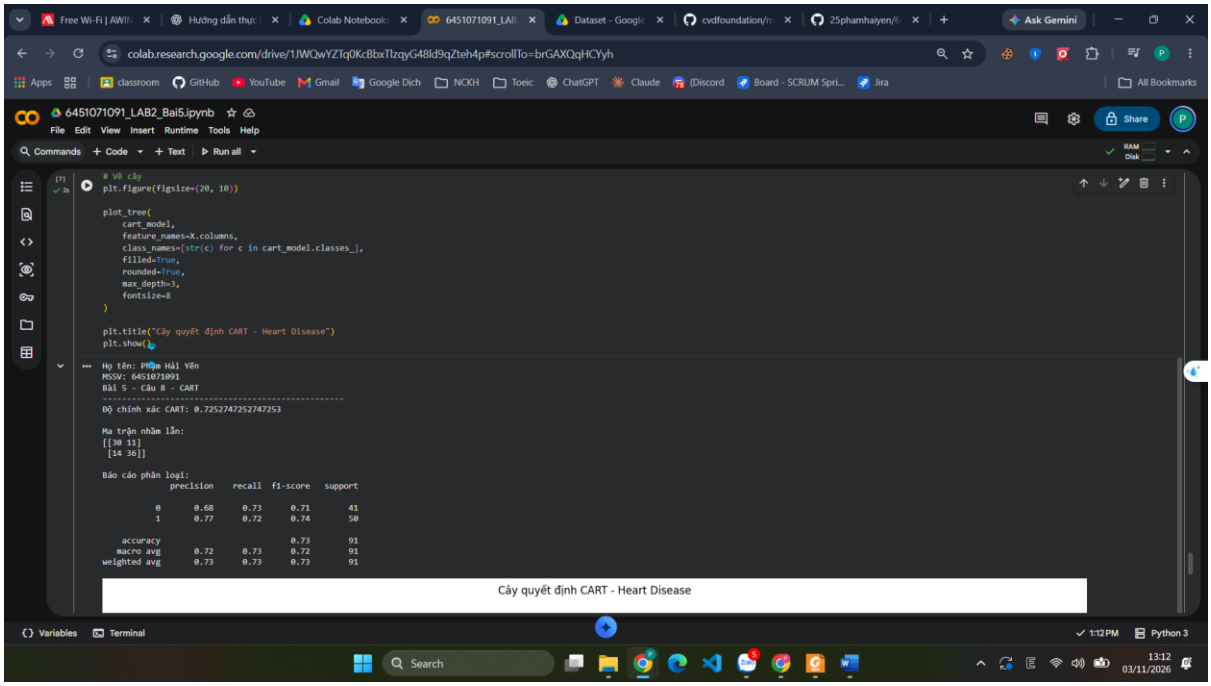
```
print("\nBáo cáo phân loại:")
print(classification_report(
    y_test,
    y_pred_cart,
    zero_division=0
))

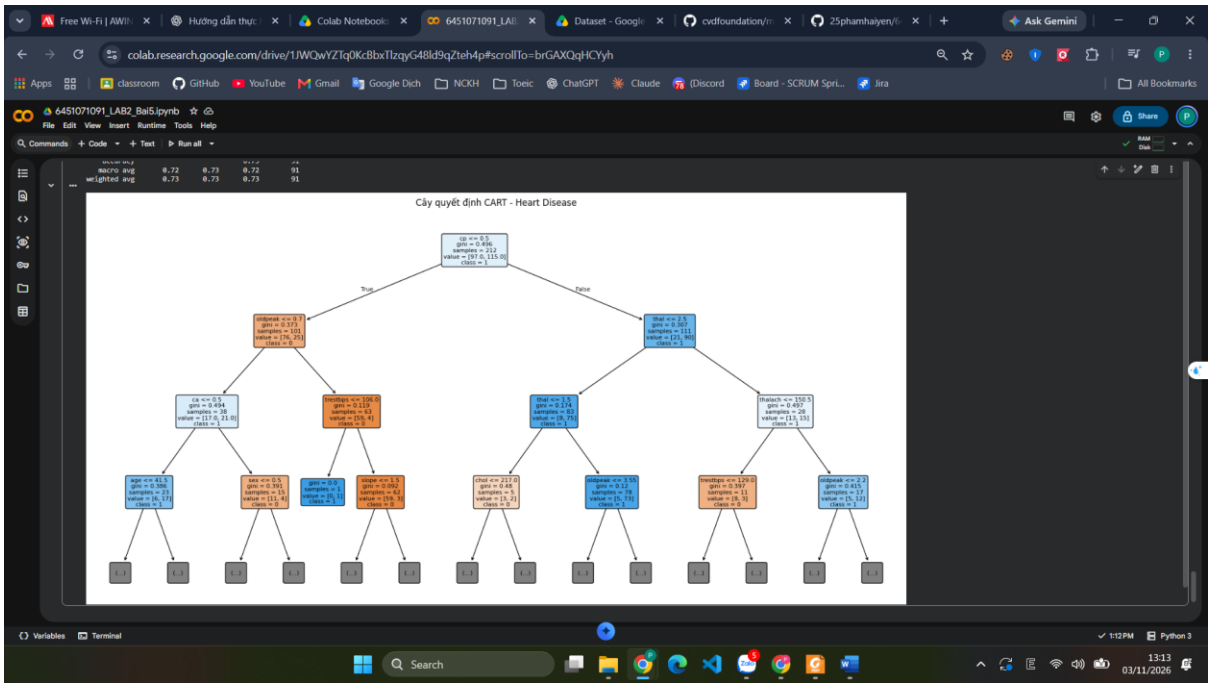
# Vẽ cây
plt.figure(figsize=(20, 10))

plot_tree(
    cart_model,
    feature_names=X.columns,
    class_names=[str(c) for c in cart_model.classes_],
    filled=True,
    rounded=True,
    max_depth=3,
    fontsize=8
)

plt.title("Cây quyết định CART - Heart Disease")
plt.show()
```

//Chụp hình kết quả





Câu 9:

//dán mã nguồn

```
# BÀI 5 - CÂU 9

# XÂY DỰNG MÔ HÌNH NAIVE BAYES


import matplotlib.pyplot as plt


from sklearn.naive_bayes import GaussianNB
```

```
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report,
    ConfusionMatrixDisplay
)

print("Họ tên: Phạm Hải Yến")
print("MSSV: 6451071091")
print("Bài 5 - Câu 9 - Naive Bayes")
print("-" * 50)

# Khởi tạo mô hình
nb_model = GaussianNB()

# Huấn luyện
nb_model.fit(X_train, y_train)

# Dự đoán
y_pred_nb = nb_model.predict(X_test)

# Đánh giá
accuracy_nb = accuracy_score(
    y_test,
    y_pred_nb
)

print("Độ chính xác Naive Bayes:", accuracy_nb)

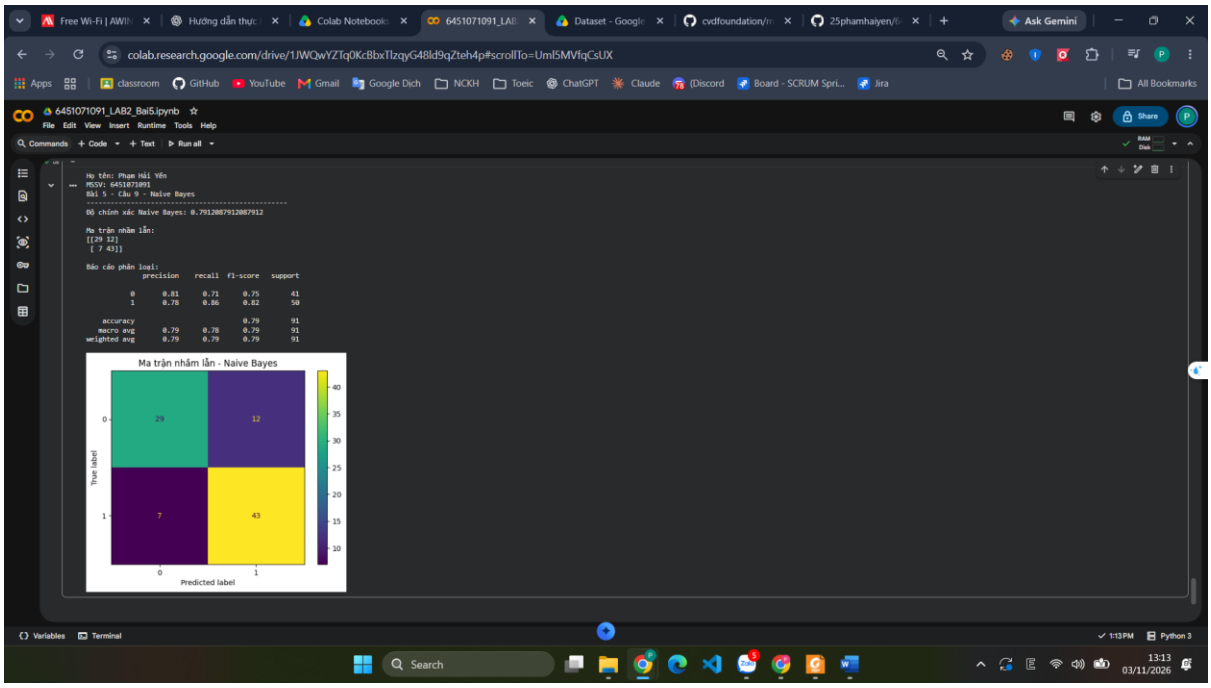
print("\nMa trận nhầm lẫn:")
cm_nb = confusion_matrix(y_test, y_pred_nb)
print(cm_nb)
```

```
print("\nBáo cáo phân loại:")
print(classification_report(
    y_test,
    y_pred_nb,
    zero_division=0
))

# Vẽ ma trận nhầm lẫn
disp = ConfusionMatrixDisplay(
    confusion_matrix=cm_nb,
    display_labels=nb_model.classes_
)

disp.plot()
plt.title("Ma trận nhầm lẫn - Naive Bayes")
plt.show()
```

//Chụp hình kết quả



Câu 10:

//dán mã nguồn

```
# BÀI 5 - CÂU 10
```

```
# SO SÁNH ID3, CART VÀ NAIVE BAYES

import pandas as pd
import matplotlib.pyplot as plt

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score
)

print("Họ tên: Phạm Hải Yến")
print("MSSV: 6451071091")
print("Bài 5 - Câu 10")
print("-" * 50)

# Các mô hình và kết quả dự đoán
models = {
    "ID3": y_pred_id3,
    "CART": y_pred_cart,
    "Naive Bayes": y_pred_nb
}

# Tính các chỉ số
results = []

for model_name, y_pred in models.items():
    results.append({
        "Mô hình": model_name,
        "Accuracy": accuracy_score(y_test, y_pred),
        "Precision": precision_score(
            y_test,
```

```
            y_pred,
            average="weighted",
            zero_division=0
        ),
        "Recall": recall_score(
            y_test,
            y_pred,
            average="weighted",
            zero_division=0
        ),
        "F1-score": f1_score(
            y_test,
            y_pred,
            average="weighted",
            zero_division=0
        )
    })

comparison = pd.DataFrame(results)

print("Bảng so sánh:")
display(comparison)

# Vẽ biểu đồ Accuracy
plt.figure(figsize=(8, 5))

plt.bar(
    comparison["Mô hình"],
    comparison["Accuracy"]
)

plt.title("So sánh Accuracy - Heart Disease")
plt.xlabel("Mô hình")
```

```
plt.ylabel("Accuracy")
plt.ylim(0, 1)

for i, value in enumerate(comparison["Accuracy"]):
    plt.text(
        i,
        value + 0.01,
        f"{value:.4f}",
        ha="center"
    )

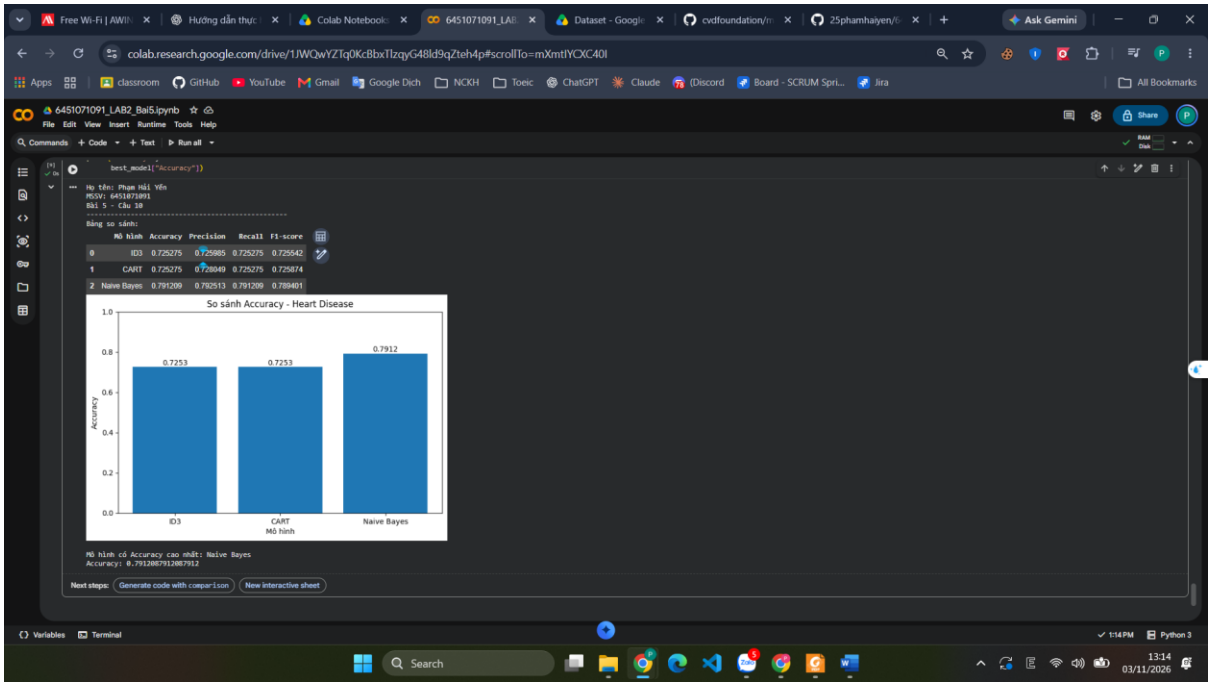
plt.show()

# Tìm mô hình có Accuracy cao nhất
best_model = comparison.loc[
    comparison["Accuracy"].idxmax()
]

print("\nMô hình có Accuracy cao nhất:",
      best_model["Mô hình"])

print("Accuracy:",
      best_model["Accuracy"])
```

//Chụp hình kết quả



Bài 6 (Trang 27)

Câu 4:

//dán mã nguồn

```
# BÀI 6 - CÂU 4
# KHẢO SÁT DỮ LIỆU MUSHROOM VÀ TƯƠNG QUAN

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

print("Họ tên: Phạm Hải Yến")
print("MSSV: 6451071091")
print("Bài 6 - Câu 4")
print("-" * 50)

# Kiểm tra dữ liệu
print("Kích thước dữ liệu:", mushroom_df.shape)

print("\nSố lượng dữ liệu thiếu:")
```

```
print(mushroom_df.isnull().sum())


print("\nSố lượng mẫu theo class:")
print(mushroom_df["class"].value_counts())


# Mã hóa dữ liệu phân loại để khảo sát tương quan
mushroom_encoded = mushroom_df.copy()


for col in mushroom_encoded.columns:
    mushroom_encoded[col] = (
        mushroom_encoded[col]
        .astype("category")
        .cat.codes
    )


# Tính ma trận tương quan
corr_matrix = mushroom_encoded.corr(method="pearson")

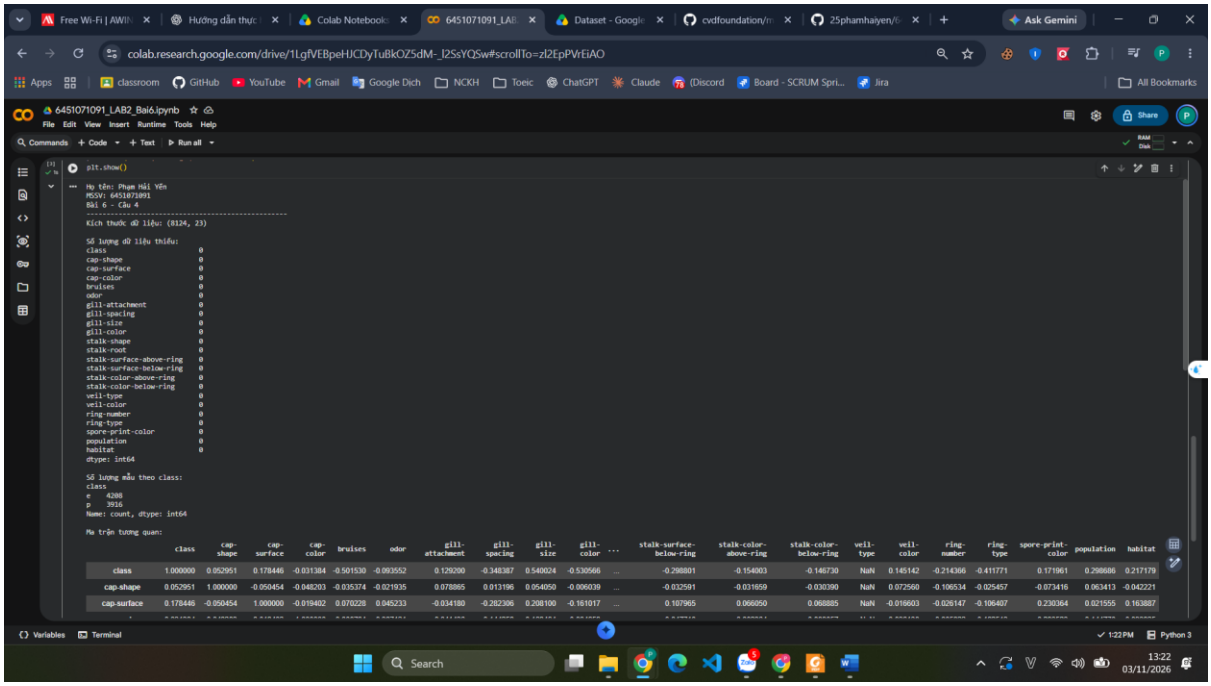

print("\nMa trận tương quan:")
display(corr_matrix)

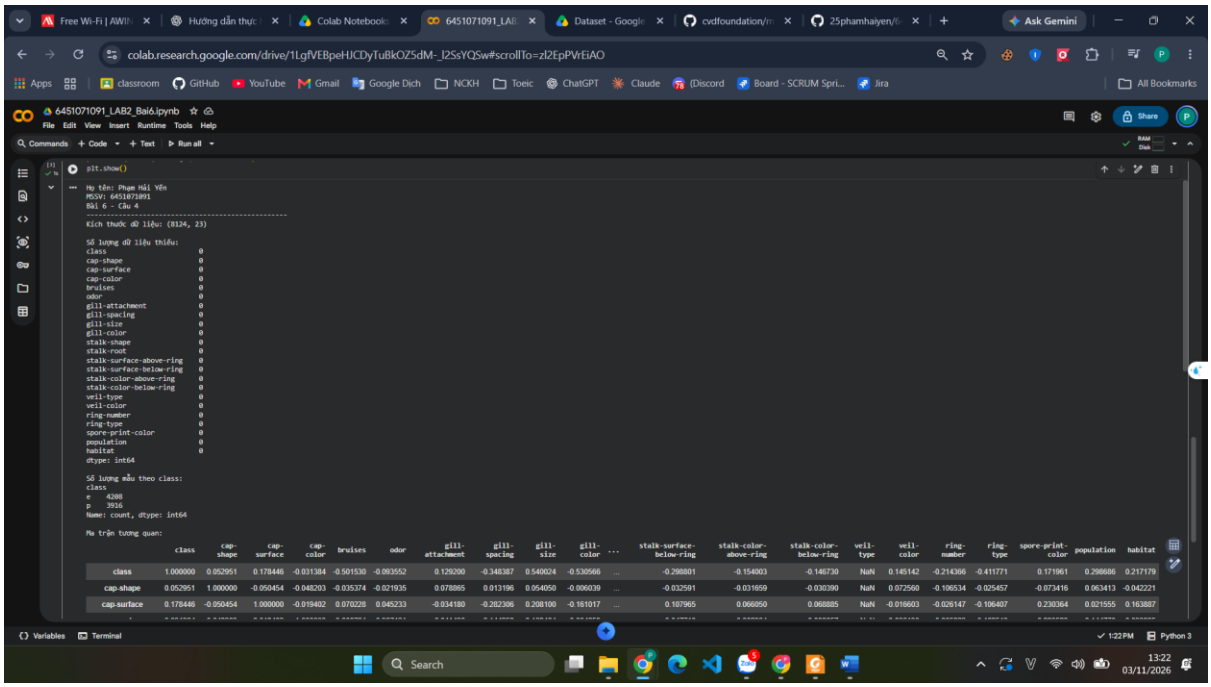

# Vẽ heatmap
plt.figure(figsize=(16, 12))

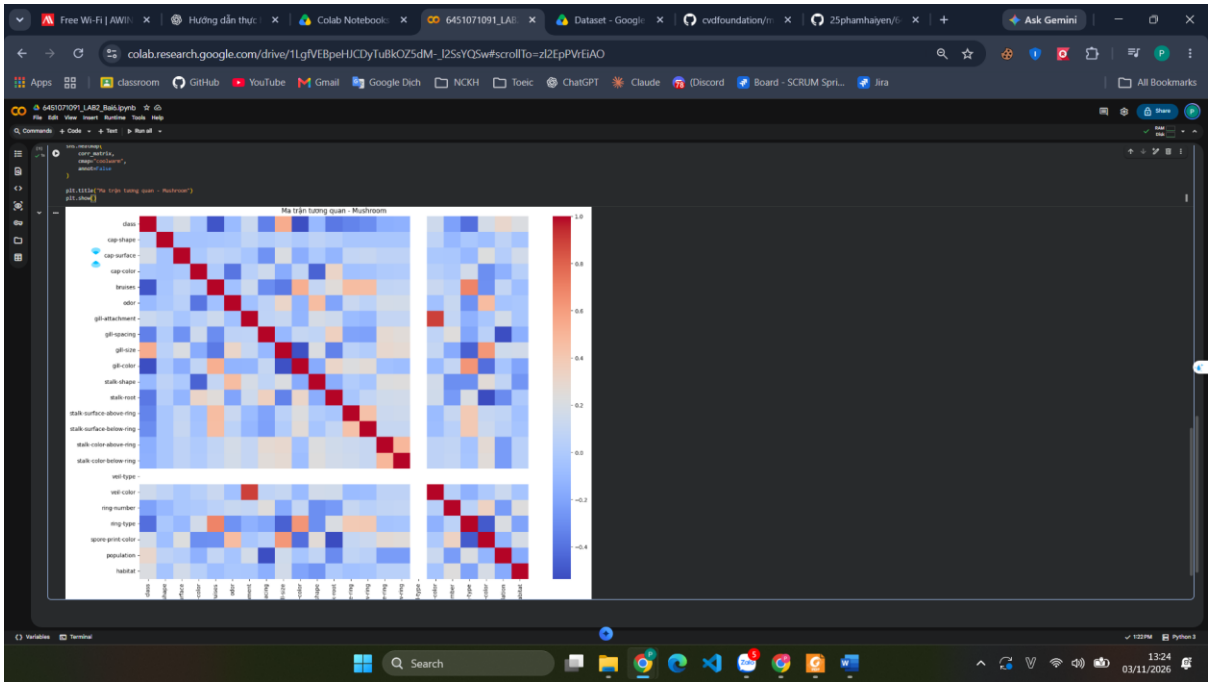

sns.heatmap(
    corr_matrix,
    cmap="coolwarm",
    annot=False
)


plt.title("Ma trận tương quan - Mushroom")
plt.show()
```

//Chụp hình kết quả


colab.research.google.com/drive/1LgfVEBpeHJCDyTuBkOZ5dM-_I2SsYQSw#scrollTo=zI2EpPVrEiAO
6451071091_LAB2_Bai6.ipynb
plt.show()
Họ tên: Phạm Hải Yến
MSV: 6451071091
Bài 6 - Câu 4
Kích thước dữ liệu: (8124, 23)
Số lượng dữ liệu thiếu:
Số lượng mẫu theo class:
Ma trận tương quan:


colab.research.google.com/drive/1LgfVEBpeHJCDyTuBkOZ5dM-_I2SsYQSw#scrollTo=zI2EpPVrEiAO
6451071091_LAB2_Bai6.ipynb
plt.show()
Họ tên: Phạm Hải Yến
MSV: 6451071091
Bài 6 - Câu 4
Kích thước dữ liệu: (8124, 23)
Số lượng dữ liệu thiếu:
Số lượng mẫu theo class:
Ma trận tương quan:



Câu 5:

//dán mã nguồn

```
# BÀI 6 - CÂU 5

# TÁCH X VÀ y


print("Họ tên: Phạm Hải Yến")

print("MSSV: 6451071091")

print("Bài 6 - Câu 5")

print("-" * 50)


# X: các thuộc tính đầu vào

X = mushroom_df.drop(columns=["class"])


# y: thuộc tính quyết định

y = mushroom_df["class"]


print("Kích thước X:", X.shape)

print("Kích thước y:", y.shape)

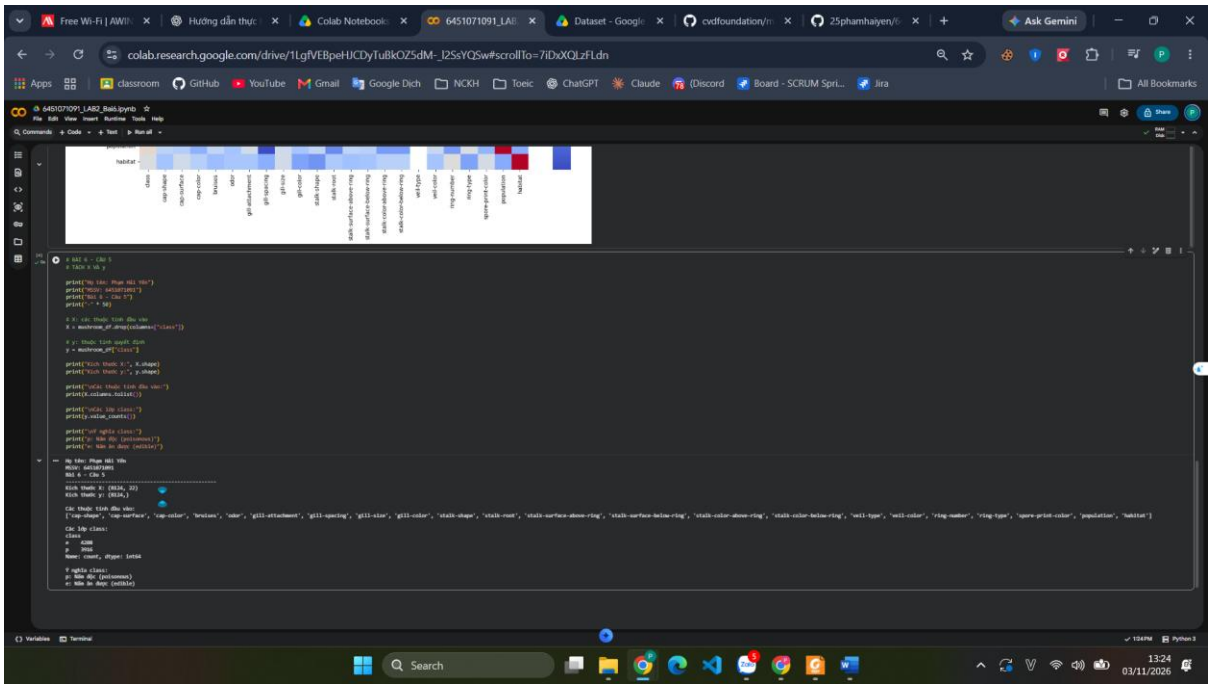

print("\nCác thuộc tính đầu vào:")
```

```
print(X.columns.tolist())


print("\nCác lớp class:")
print(y.value_counts())


print("\nÝ nghĩa class:")
print("p: Nấm độc (poisonous)")
print("e: Nấm ăn được (edible)")
```

//Chụp hình kết quả



Câu 6:

//dán mã nguồn

```
# BÀI 6 - CÂU 6
# CHIA DỮ LIỆU TRAIN VÀ TEST


from sklearn.model_selection import train_test_split


print("Họ tên: Phạm Hải Yến")
print("MSSV: 6451071091")
print("Bài 6 - Câu 6")
print("-" * 50)
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.3,
    random_state=42
)

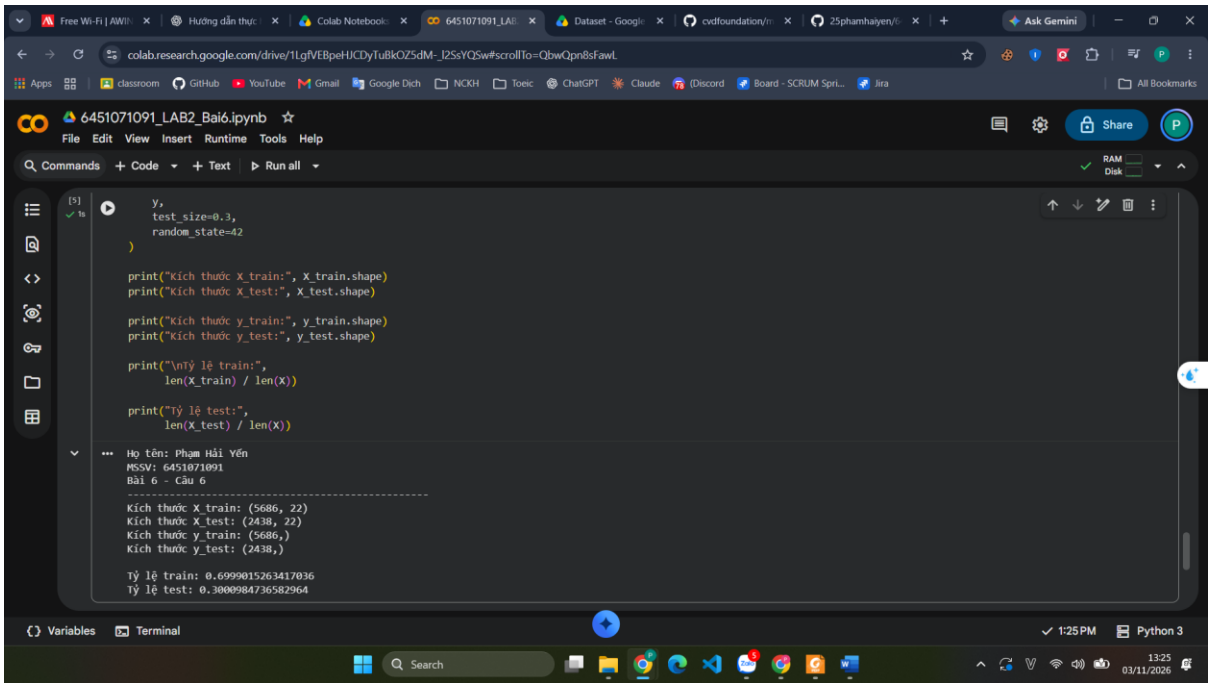
print("Kích thước X_train:", X_train.shape)
print("Kích thước X_test:", X_test.shape)

print("Kích thước y_train:", y_train.shape)
print("Kích thước y_test:", y_test.shape)

print("\nTỷ lệ train:",
      len(X_train) / len(X))

print("Tỷ lệ test:",
      len(X_test) / len(X))
```

//Chụp hình kết quả



Câu 7:

//dán mã nguồn

```
# BÀI 6 - CÂU 7
# XÂY DỰNG CÂY QUYẾT ĐỊNH ID3


import pandas as pd
import matplotlib.pyplot as plt


from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report
)


print("Họ tên: Phạm Hải Yến")
print("MSSV: 6451071091")
print("Bài 6 - Câu 7 - ID3")
print("-" * 50)


# Mã hóa thuộc tính phân loại
X_train_encoded = pd.get_dummies(
    X_train,
    dtype=int
)


X_test_encoded = pd.get_dummies(
    X_test,
    dtype=int
)


# Đồng bộ cột train và test
X_train_encoded, X_test_encoded = X_train_encoded.align(
```

```
    X_test_encoded,
    join="outer",
    axis=1,
    fill_value=0
)

# Xây dựng mô hình ID3 theo tiêu chí Entropy
id3_model = DecisionTreeClassifier(
    criterion="entropy",
    random_state=42
)

# Huấn luyện
id3_model.fit(X_train_encoded, y_train)

# Dự đoán
y_pred_id3 = id3_model.predict(X_test_encoded)

# Đánh giá
accuracy_id3 = accuracy_score(
    y_test,
    y_pred_id3
)

print("Độ chính xác ID3:", accuracy_id3)

print("\nMa trận nhầm lẫn:")
print(confusion_matrix(y_test, y_pred_id3))

print("\nBáo cáo phân loại:")
print(classification_report(
    y_test,
    y_pred_id3,
```

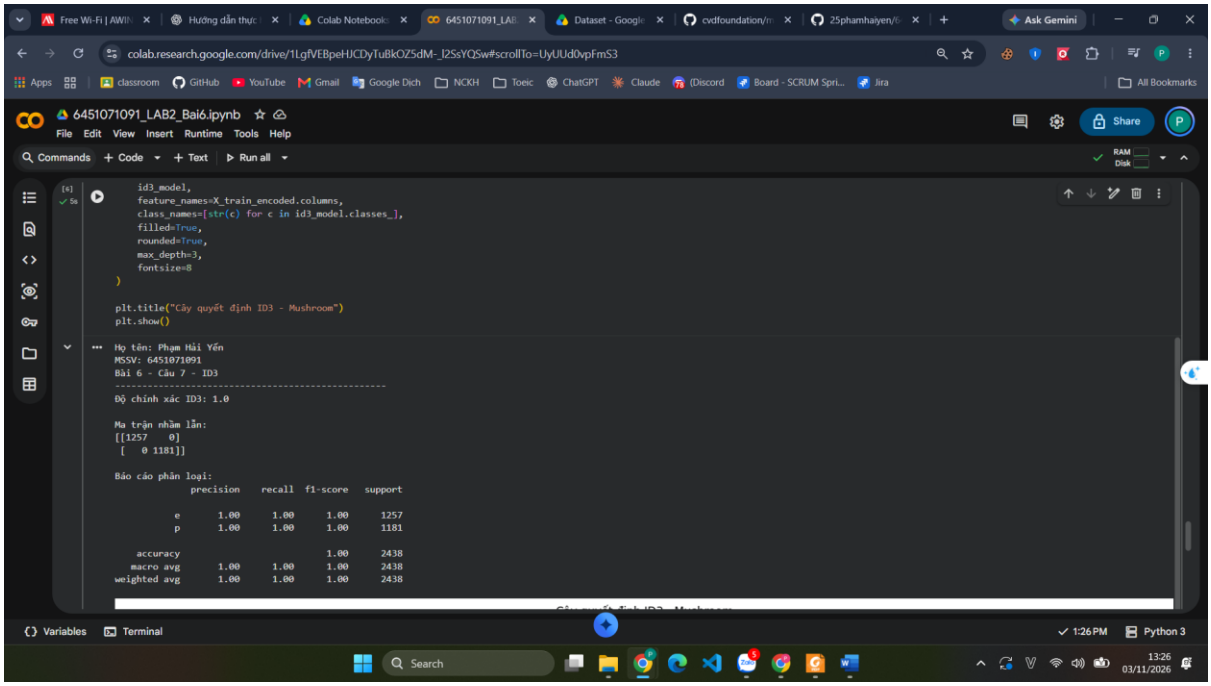
```
    zero_division=0
))

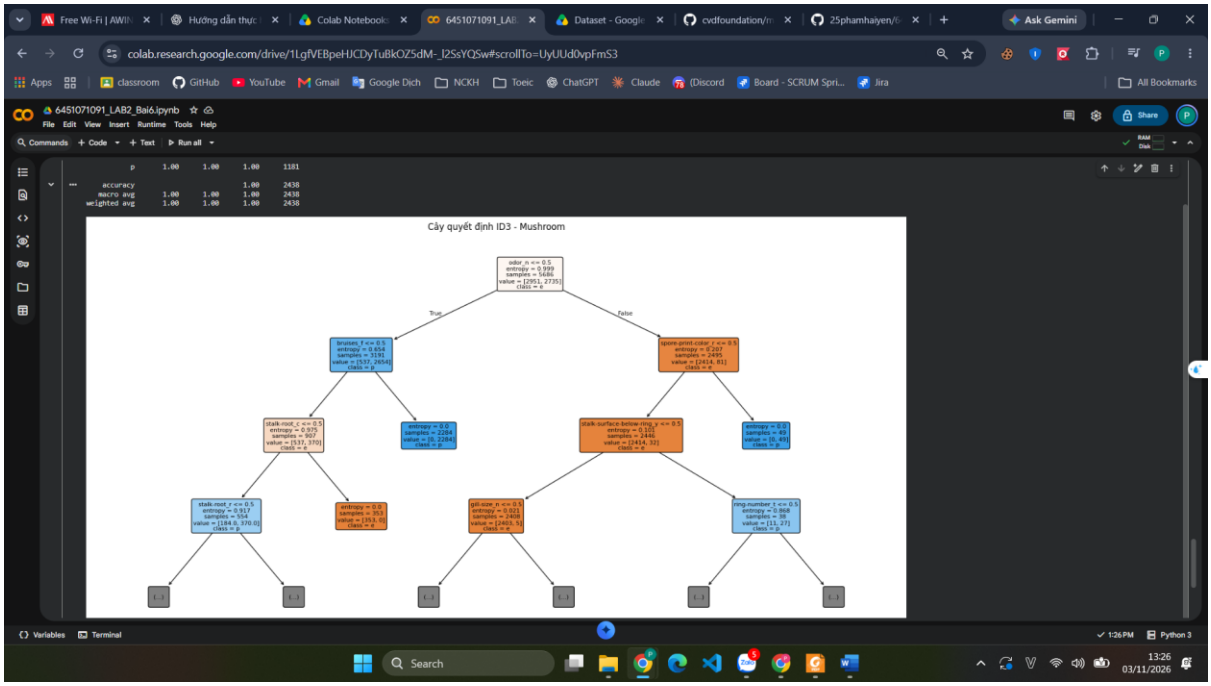
# Vẽ cây quyết định
plt.figure(figsize=(20, 10))

plot_tree(
    id3_model,
    feature_names=X_train_encoded.columns,
    class_names=[str(c) for c in id3_model.classes_],
    filled=True,
    rounded=True,
    max_depth=3,
    fontsize=8
)

plt.title("Cây quyết định ID3 - Mushroom")
plt.show()
```

//Chụp hình kết quả





Câu 8:

//dán mã nguồn

```
# BÀI 6 - CÂU 8
# XÂY DỰNG CÂY QUYẾT ĐỊNH CART

import matplotlib.pyplot as plt

from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report
)

print("Họ tên: Phạm Hải Yến")
print("MSSV: 6451071091")
print("Bài 6 - Câu 8 - CART")
print("-" * 50)

# Khởi tạo mô hình CART
```

```
cart_model = DecisionTreeClassifier(
    criterion="gini",
    random_state=42
)

# Huấn luyện
cart_model.fit(X_train_encoded, y_train)

# Dự đoán
y_pred_cart = cart_model.predict(X_test_encoded)

# Đánh giá
accuracy_cart = accuracy_score(
    y_test,
    y_pred_cart
)

print("Độ chính xác CART:", accuracy_cart)

print("\nMa trận nhầm lẫn:")
print(confusion_matrix(y_test, y_pred_cart))

print("\nBáo cáo phân loại:")
print(classification_report(
    y_test,
    y_pred_cart,
    zero_division=0
))

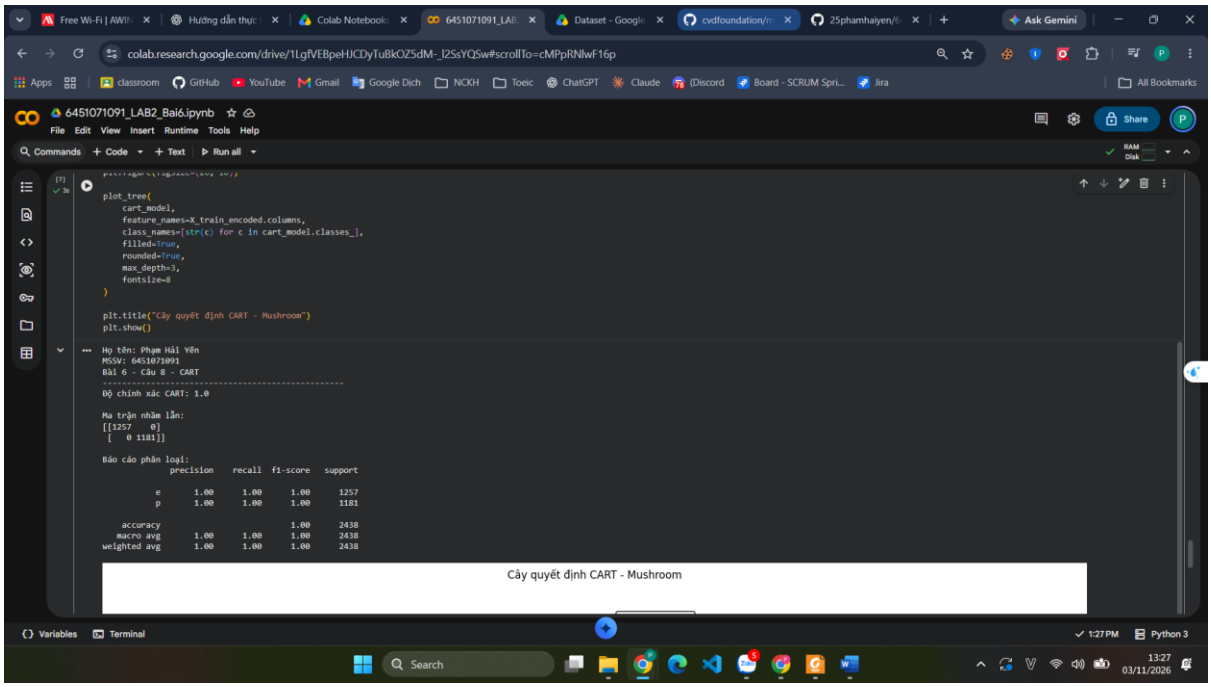
# Vẽ cây quyết định
plt.figure(figsize=(20, 10))

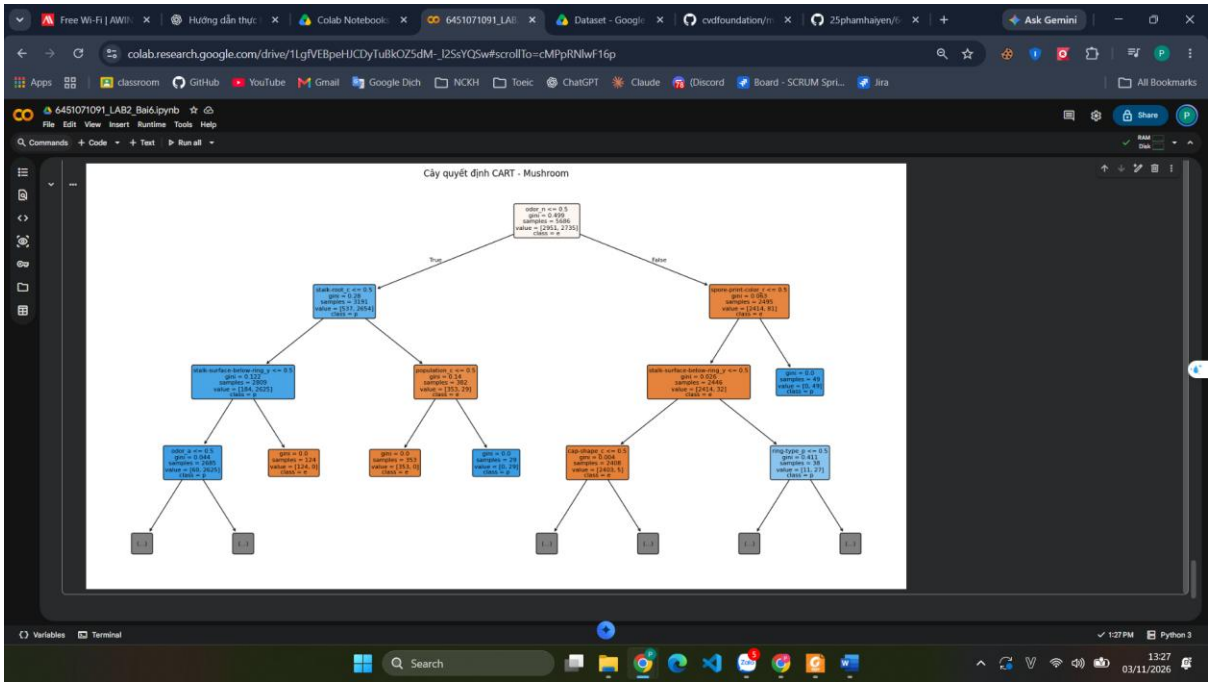
plot_tree(
```

```
    cart_model,
    feature_names=X_train_encoded.columns,
    class_names=[str(c) for c in cart_model.classes_],
    filled=True,
    rounded=True,
    max_depth=3,
    fontsize=8
)

plt.title("Cây quyết định CART - Mushroom")
plt.show()
```

//Chụp hình kết quả





Câu 9:

//dán mã nguồn

```
# BÀI 6 - CÂU 9
# XÂY DỰNG MÔ HÌNH NAIVE BAYES

import matplotlib.pyplot as plt

from sklearn.naive_bayes import BernoulliNB
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report,
    ConfusionMatrixDisplay
)

print("Họ tên: Phạm Hải Yến")
print("MSSV: 6451071091")
print("Bài 6 - Câu 9 - Naive Bayes")
print("-" * 50)
```

```
# Khởi tạo mô hình Bernoulli Naive Bayes
nb_model = BernoulliNB()

# Huấn luyện
nb_model.fit(X_train_encoded, y_train)

# Dự đoán
y_pred_nb = nb_model.predict(X_test_encoded)

# Đánh giá
accuracy_nb = accuracy_score(
    y_test,
    y_pred_nb
)

print("Độ chính xác Naive Bayes:", accuracy_nb)

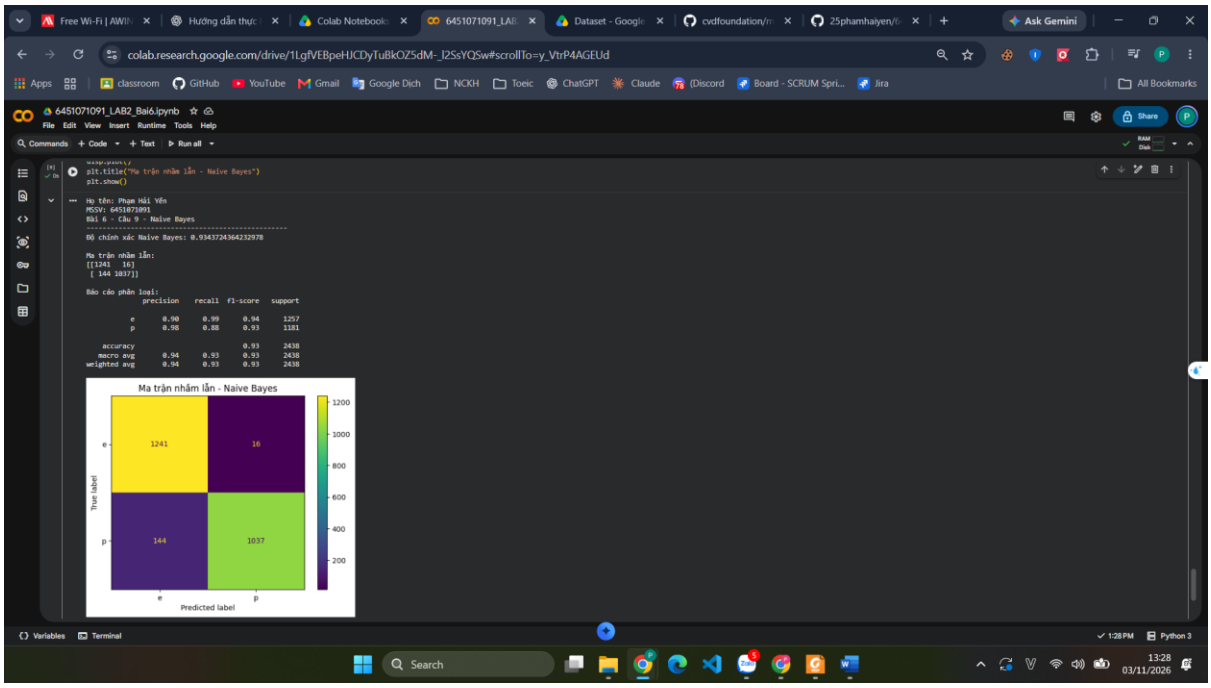
print("\nMa trận nhầm lẫn:")
cm_nb = confusion_matrix(y_test, y_pred_nb)
print(cm_nb)

print("\nBáo cáo phân loại:")
print(classification_report(
    y_test,
    y_pred_nb,
    zero_division=0
))

# Vẽ ma trận nhầm lẫn
disp = ConfusionMatrixDisplay(
    confusion_matrix=cm_nb,
    display_labels=nb_model.classes_
)
```

```
disp.plot()
plt.title("Ma trận nhầm lẫn - Naive Bayes")
plt.show()
```

//Chụp hình kết quả



Câu 10:

//dán mã nguồn

```
# BÀI 6 - CÂU 10
# SO SÁNH ID3, CART VÀ NAIVE BAYES

import pandas as pd
import matplotlib.pyplot as plt

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score
)
```

```
print("Họ tên: Phạm Hải Yến")
print("MSSV: 6451071091")
print("Bài 6 - Câu 10")
print("-" * 50)

# Các mô hình và kết quả dự đoán
models = {
    "ID3": y_pred_id3,
    "CART": y_pred_cart,
    "Naive Bayes": y_pred_nb
}

# Tính các chỉ số đánh giá
results = []

for model_name, y_pred in models.items():
    results.append({
        "Mô hình": model_name,
        "Accuracy": accuracy_score(y_test, y_pred),
        "Precision": precision_score(
            y_test,
            y_pred,
            average="weighted",
            zero_division=0
        ),
        "Recall": recall_score(
            y_test,
            y_pred,
            average="weighted",
            zero_division=0
        ),
        "F1-score": f1_score(
            y_test,
```

```
            y_pred,
            average="weighted",
            zero_division=0
        )
    })

comparison = pd.DataFrame(results)

print("Bảng so sánh kết quả:")
display(comparison)

# Vẽ biểu đồ so sánh Accuracy
plt.figure(figsize=(8, 5))

plt.bar(
    comparison["Mô hình"],
    comparison["Accuracy"]
)

plt.title("So sánh Accuracy - Mushroom")
plt.xlabel("Mô hình")
plt.ylabel("Accuracy")
plt.ylim(0, 1)

for i, value in enumerate(comparison["Accuracy"]):
    plt.text(
        i,
        value + 0.01,
        f"{value:.4f}",
        ha="center"
    )

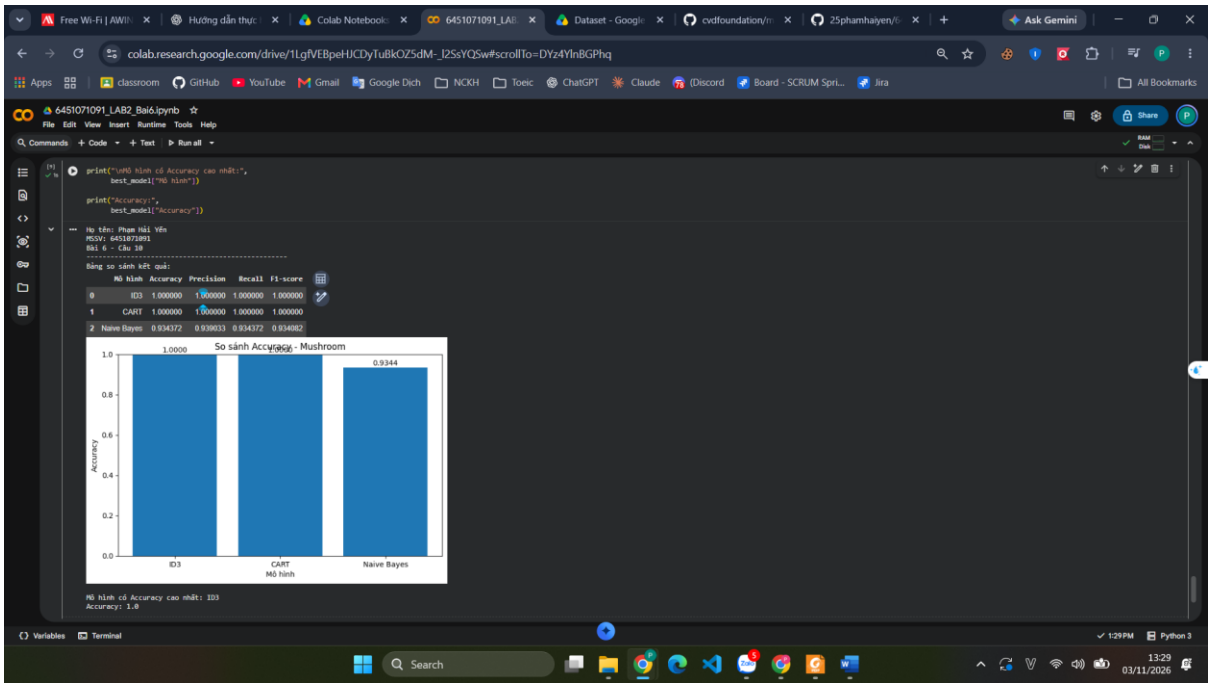
plt.show()
```

```
# Tìm mô hình có Accuracy cao nhất
best_model = comparison.loc[
    comparison["Accuracy"].idxmax()
]

print("\nMô hình có Accuracy cao nhất:",
      best_model["Mô hình"])

print("Accuracy:",
      best_model["Accuracy"])
```

//Chụp hình kết quả



Bài 7 (Trang 27 - 31)

//SV copy-paste mã nguồn và hình ảnh theo như trình bày trong tài liệu thực hành

```
# Bài 7 - Nhận dạng chữ số viết tay MNIST
# Sinh viên: Phạm Hải Yến
# MSSV: 6451071091

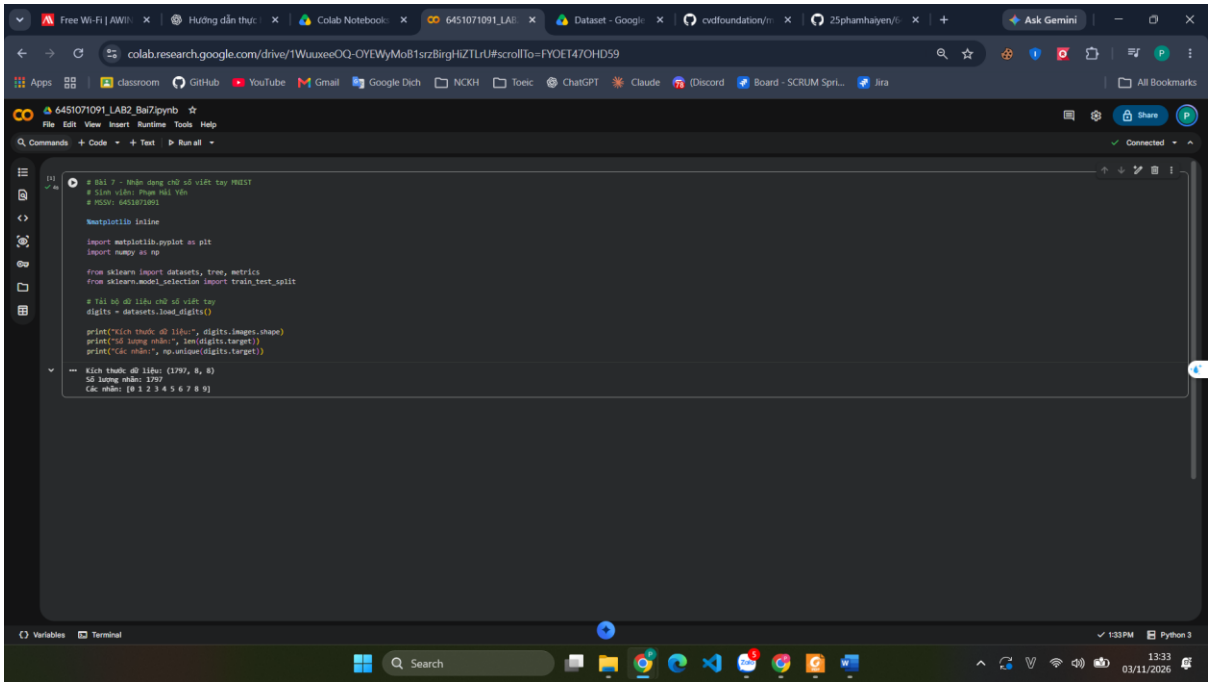
%matplotlib inline
```

```
import matplotlib.pyplot as plt
import numpy as np

from sklearn import datasets, tree, metrics
from sklearn.model_selection import train_test_split

# Tải bộ dữ liệu chữ số viết tay
digits = datasets.load_digits()

print("Kích thước dữ liệu:", digits.images.shape)
print("Số lượng nhãn:", len(digits.target))
print("Các nhãn:", np.unique(digits.target))
```




```
# Biểu diễn 4 mẫu dữ liệu huấn luyện tương ứng với 4 lớp 0, 1, 2, 3

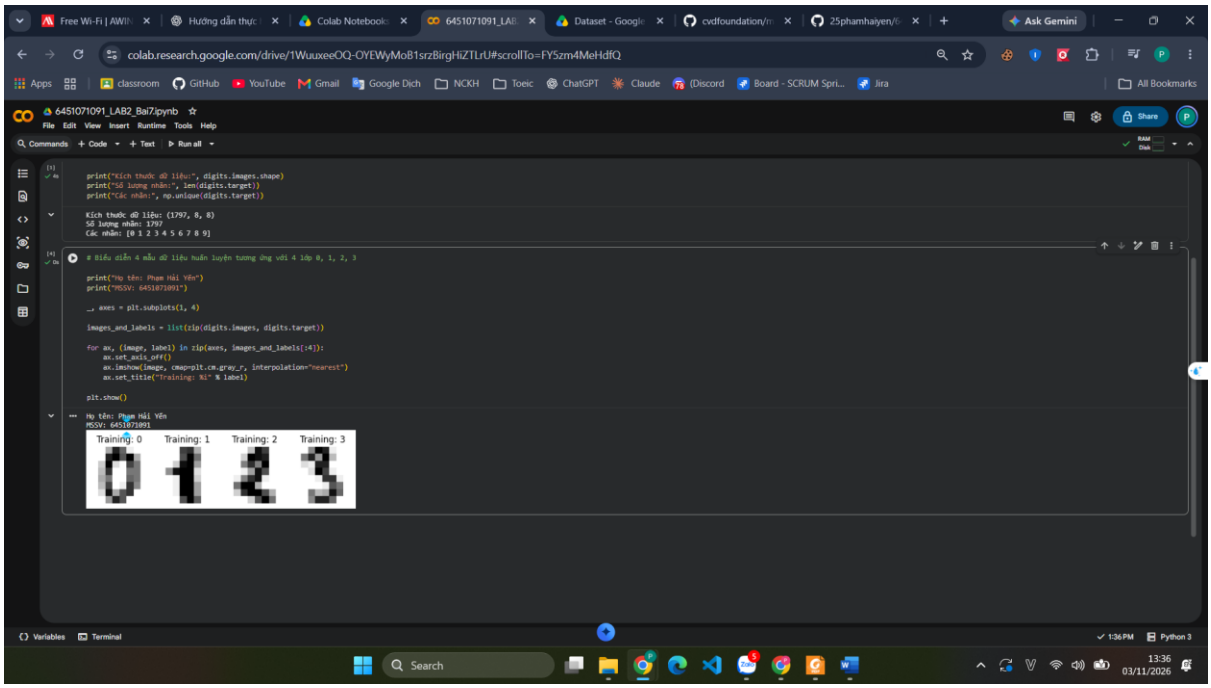
print("Họ tên: Phạm Hải Yến")
print("MSSV: 6451071091")

_, axes = plt.subplots(1, 4)
```

```
images_and_labels = list(zip(digits.images, digits.target))


for ax, (image, label) in zip(axes, images_and_labels[:4]):
    ax.set_axis_off()
    ax.imshow(image, cmap=plt.cm.gray_r, interpolation="nearest")
    ax.set_title("Training: %i" % label)


plt.show()
```



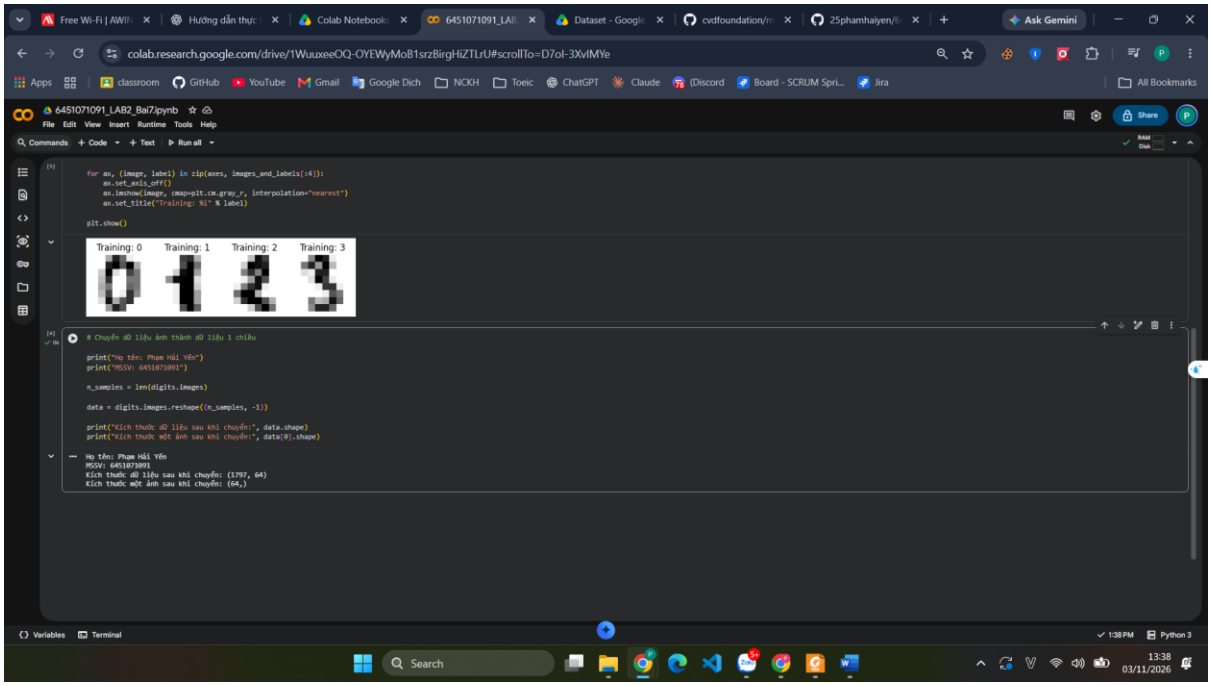

```
# Chuyển dữ liệu ảnh thành dữ liệu 1 chiều


print("Họ tên: Phạm Hải Yến")
print("MSSV: 6451071091")


n_samples = len(digits.images)


data = digits.images.reshape((n_samples, -1))


print("Kích thước dữ liệu sau khi chuyển:", data.shape)
print("Kích thước một ảnh sau khi chuyển:", data[0].shape)
```




```
# Tạo bộ phân loại cây quyết định ID3


print("Họ tên: Phạm Hải Yến")

print("MSSV: 6451071091")


classifier = tree.DecisionTreeClassifier(

    criterion="entropy",

    random_state=33

)
```

```
# Chia dữ liệu thành tập huấn luyện và tập kiểm thử


print("Họ tên: Phạm Hải Yến")

print("MSSV: 6451071091")


X_train, X_test, y_train, y_test = train_test_split(

    data,

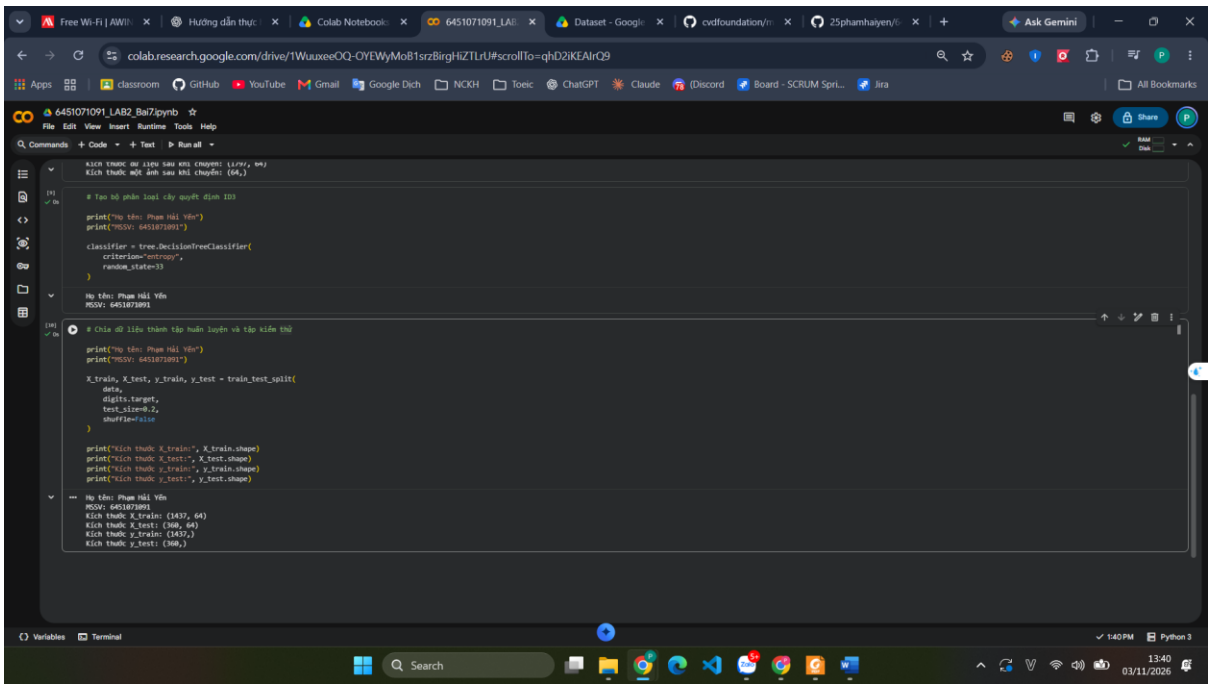
    digits.target,

    test_size=0.2,

    shuffle=False
```

```
)


print("Kích thước X_train:", X_train.shape)
print("Kích thước X_test:", X_test.shape)
print("Kích thước y_train:", y_train.shape)
print("Kích thước y_test:", y_test.shape)
```




```
# Huấn luyện cây quyết định trên tập huấn luyện


print("Họ tên: Phạm Hải Yến")
print("MSSV: 6451071091")


classifier.fit(X_train, y_train)
```

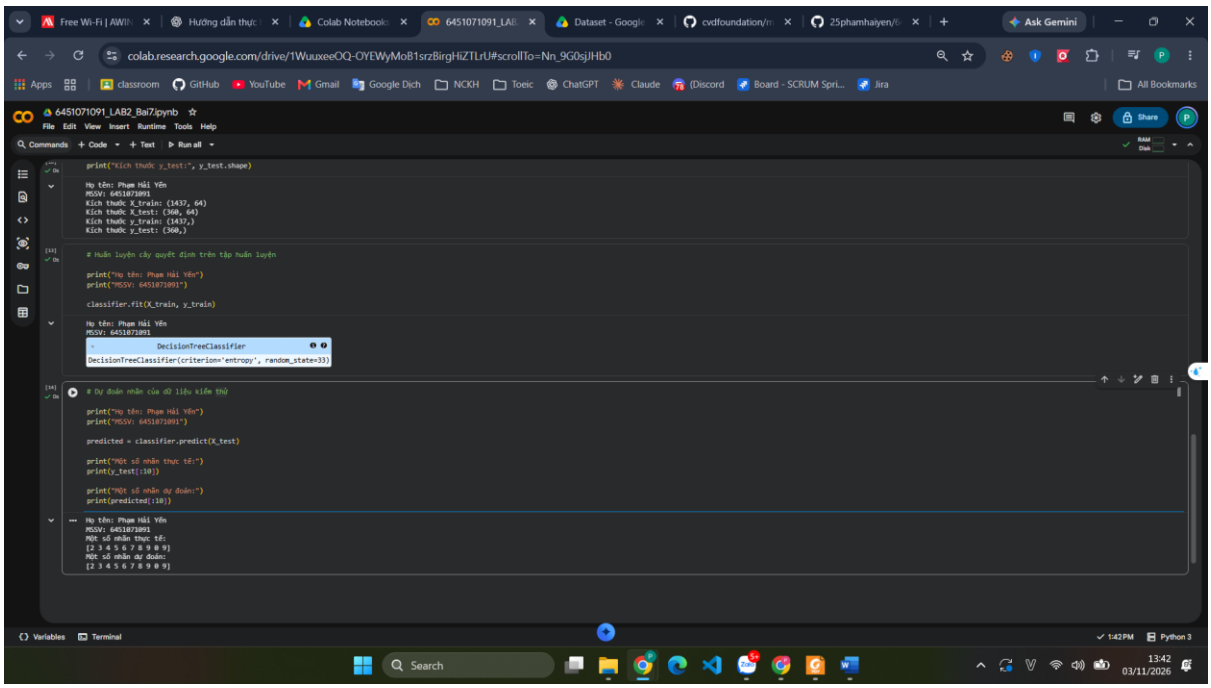
```
# Dự đoán nhãn của dữ liệu kiểm thử


print("Họ tên: Phạm Hải Yến")
print("MSSV: 6451071091")


predicted = classifier.predict(X_test)
```

```
print("Một số nhãn thực tế:")
print(y_test[:10])

print("Một số nhãn dự đoán:")
print(predicted[:10])
```



```
# Hiển thị 4 ảnh và nhãn dự đoán

print("Họ tên: Phạm Hải Yến")
print("MSSV: 6451071091")

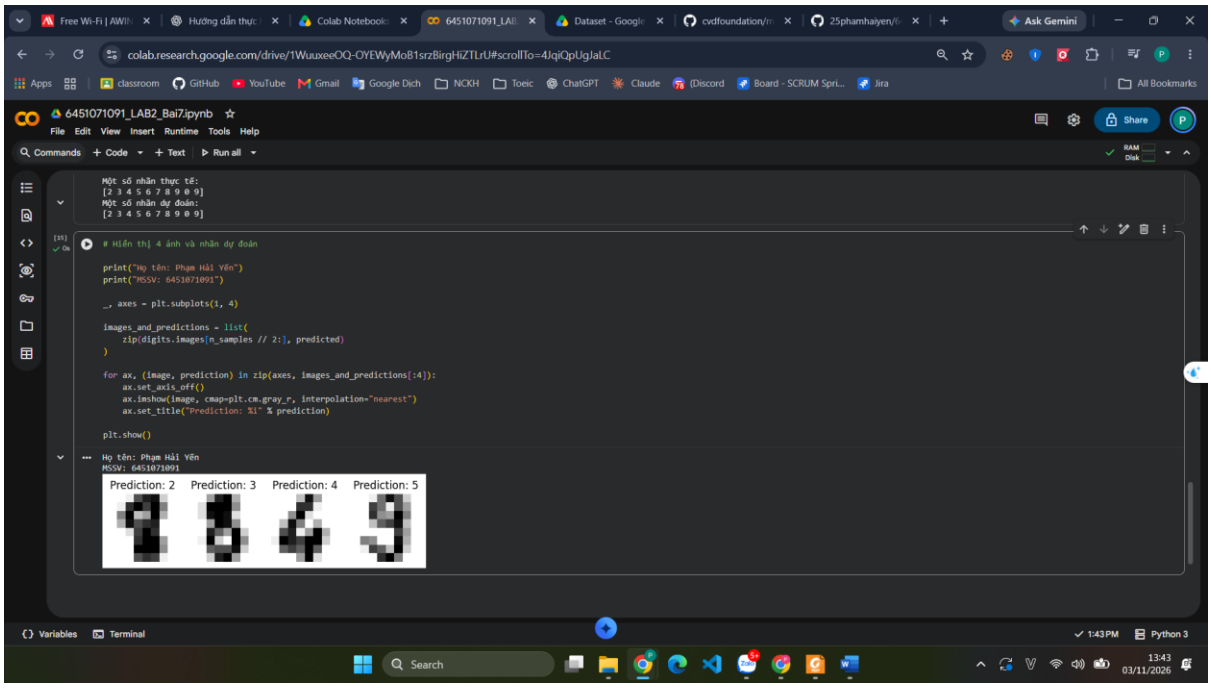
_, axes = plt.subplots(1, 4)

images_and_predictions = list(
    zip(digits.images[n_samples // 2:], predicted)
)

for ax, (image, prediction) in zip(axes, images_and_predictions[:4]):
    ax.set_axis_off()
```

```
    ax.imshow(image, cmap=plt.cm.gray_r, interpolation="nearest")
    ax.set_title("Prediction: %i" % prediction)

plt.show()
```




```
# In báo cáo phân loại

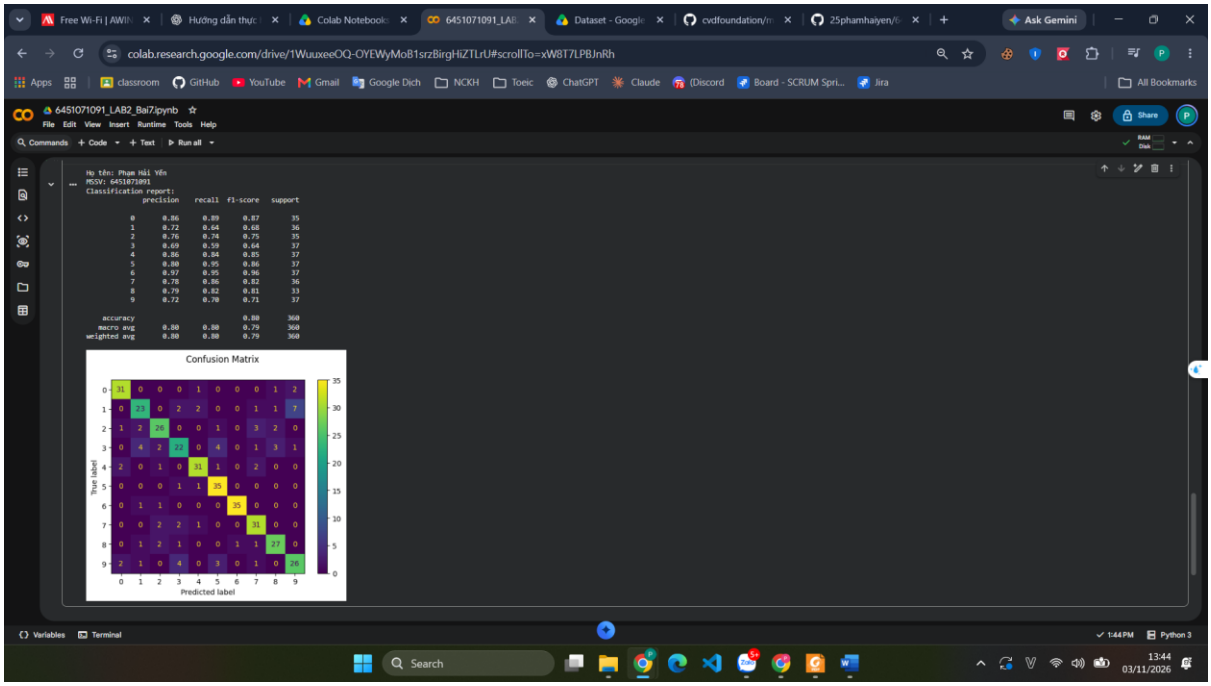
print("Họ tên: Phạm Hải Yến")
print("MSSV: 6451071091")

print("Classification report:")
print(metrics.classification_report(y_test, predicted))

# Vẽ ma trận nhầm lẫn
disp = metrics.ConfusionMatrixDisplay.from_predictions(
    y_test,
    predicted
)

disp.figure_.suptitle("Confusion Matrix")
```

```
plt.show()
```




```
from google.colab import files
from PIL import Image, ImageOps
import numpy as np
import matplotlib.pyplot as plt

print("Họ tên: Phạm Hải Yến")
print("MSSV: 6451071091")

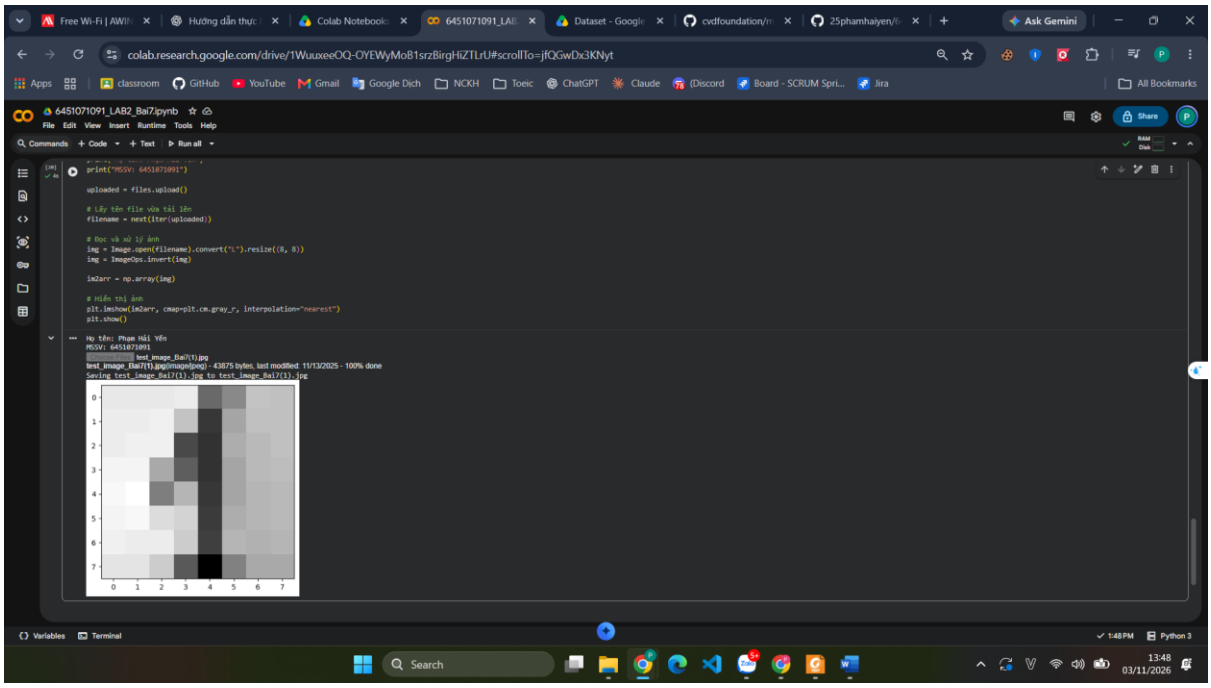
uploaded = files.upload()

# Lấy tên file vừa tải lên
filename = next(iter(uploaded))

# Đọc và xử lý ảnh
img = Image.open(filename).convert("L").resize((8, 8))
img = ImageOps.invert(img)

im2arr = np.array(img)
```

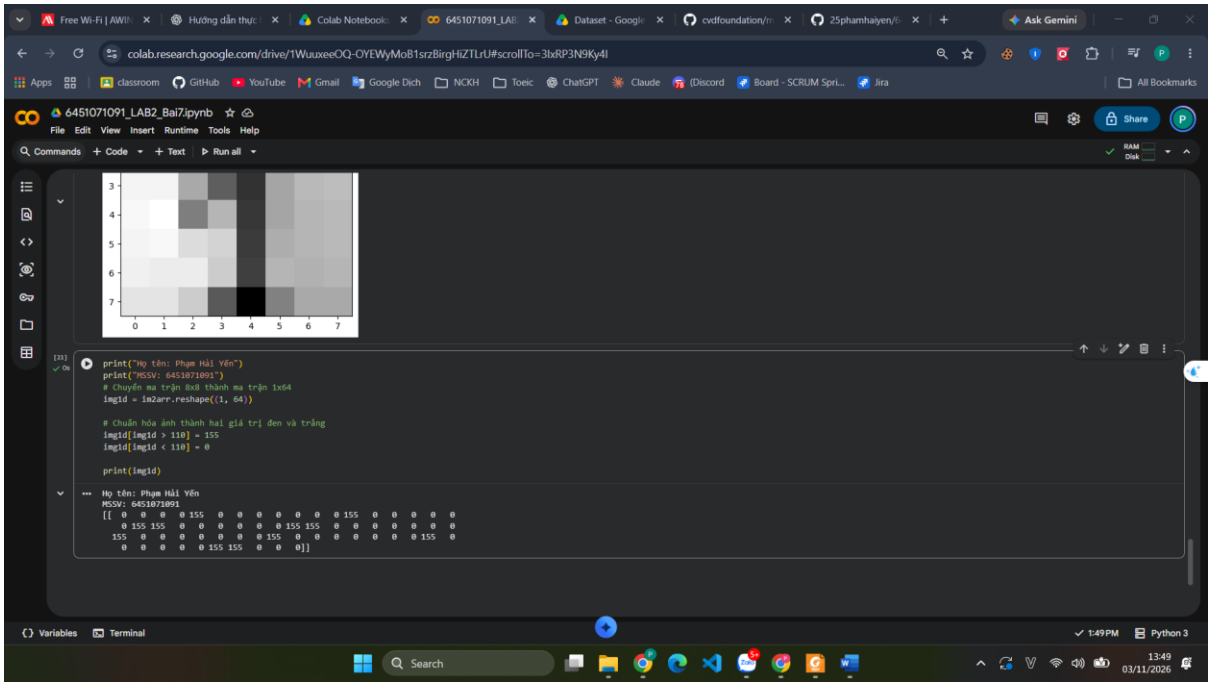
```
# Hiển thị ảnh
plt.imshow(im2arr, cmap=plt.cm.gray_r, interpolation="nearest")
plt.show()
```




```
print("Họ tên: Phạm Hải Yến")
print("MSSV: 6451071091")
# Chuyển ma trận 8x8 thành ma trận 1x64
img1d = im2arr.reshape((1, 64))

# Chuẩn hóa ảnh thành hai giá trị đen và trắng
img1d[img1d > 110] = 155
img1d[img1d < 110] = 0

print(img1d)
```




```
# Hiển thị ảnh sau khi chuẩn hóa


print("Họ tên: Phạm Hải Yến")

print("MSSV: 6451071091")

plt.imshow(

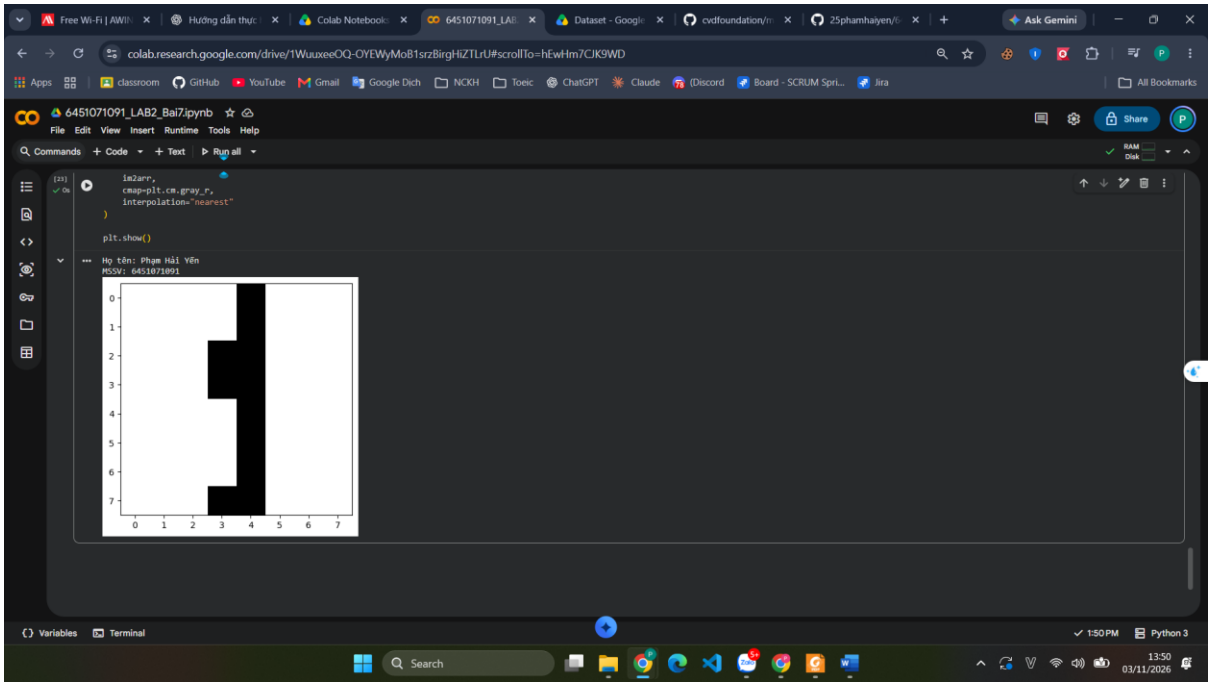
    im2arr,

    cmap=plt.cm.gray_r,

    interpolation="nearest"

)


plt.show()
```



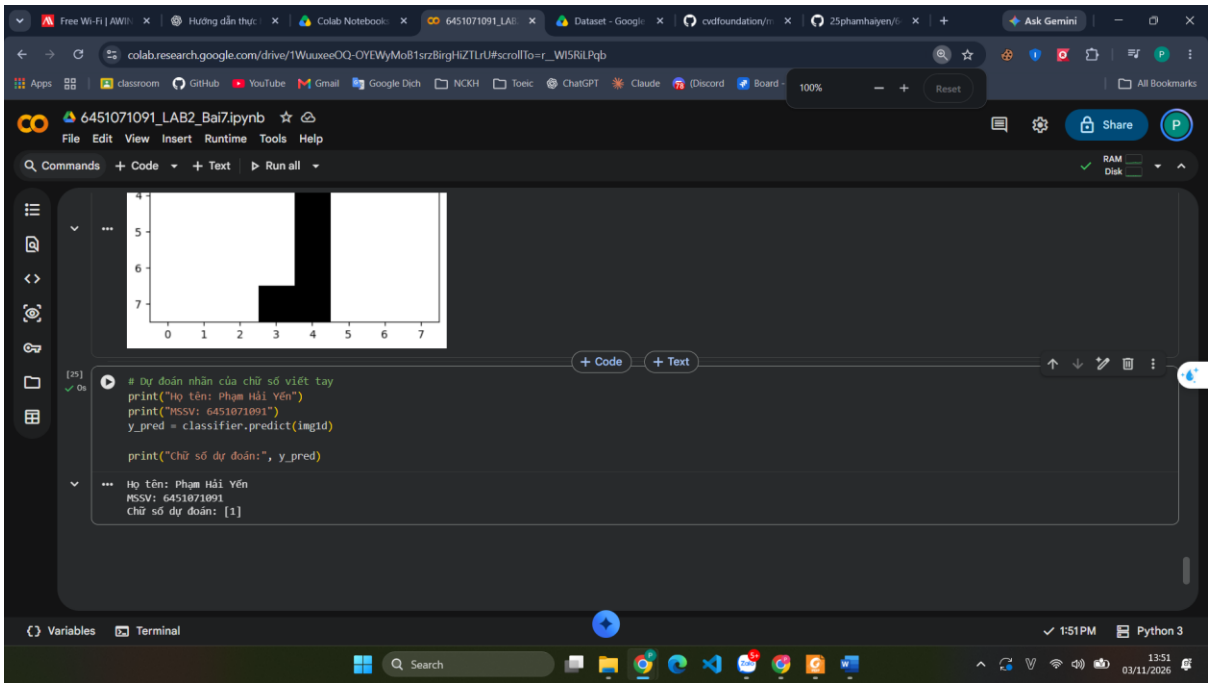

```
# Dự đoán nhãn của chữ số viết tay

print("Họ tên: Phạm Hải Yến")

print("MSSV: 6451071091")

y_pred = classifier.predict(img1d)


print("Chữ số dự đoán:", y_pred)
```



Bài 2 (Trang 32)

```
# BÀI 1 - DỰ ĐOÁN DOANH SỐ ĐỒ CHƠI
```

```
# Họ tên: Phạm Hải Yến
# MSSV: 6451071091

import pandas as pd
import numpy as np

from sklearn.preprocessing import OrdinalEncoder
from sklearn.tree import DecisionTreeClassifier
from sklearn.naive_bayes import CategoricalNB
from sklearn.metrics import (
    confusion_matrix,
    precision_score,
    recall_score,
    f1_score,
    classification_report
)

# 1. Nhập dữ liệu huấn luyện theo bảng trang 24
data1 = {
    "Loai": [
        "Điều khiển", "Xếp hình", "Xếp hình",
        "Điều khiển", "Búp bê", "Điều khiển",
        "Búp bê", "Điều khiển", "Xếp hình",
        "Xếp hình", "Búp bê", "Xếp hình",
        "Điều khiển", "Búp bê", "Búp bê"
    ],
    "So_mau": [3, 5, 7, 5, 3, 5, 5, 7, 7, 3, 3, 3, 5, 5, 7],
    "Kich_thuoc": [
        "Nhỏ", "Vừa", "To", "Nhỏ", "Vừa",
        "Vừa", "To", "Vừa", "To", "To",
        "Nhỏ", "Nhỏ", "To", "Vừa", "To"
    ],
    "Chat_lieu": [
```

```
        "Nhựa PP", "Cao su", "Nhựa PP", "Cao su", "Nhựa PP",
        "Nhựa PP", "Nhựa PP", "Cao su", "Cao su", "Nhựa PP",
        "Cao su", "Nhựa PP", "Nhựa PP", "Nhựa PP", "Nhựa PP"
    ],
    "Doanh_so": [
        "Cao", "Thấp", "Thấp", "Thấp", "Thấp",
        "Cao", "Cao", "Thấp", "Cao", "Thấp",
        "Thấp", "Cao", "Thấp", "Cao", "Cao"
    ]
}

df1 = pd.DataFrame(data1)

print("Họ tên: Phạm Hải Yến")
print("MSSV: 6451071091")
print("\nDữ liệu huấn luyện:")
print(df1)

# 2. Tách thuộc tính và nhãn
X1 = df1.drop("Doanh_so", axis=1)
y1 = df1["Doanh_so"]

# 3. Mã hóa thuộc tính dạng chữ
encoder1 = OrdinalEncoder()
X1_encoded = encoder1.fit_transform(X1)

# 4. Khởi tạo và huấn luyện mô hình
id3_1 = DecisionTreeClassifier(
    criterion="entropy",
    random_state=42
)

cart_1 = DecisionTreeClassifier(
```

```
    criterion="gini",
    random_state=42
)

nb_1 = CategoricalNB()

id3_1.fit(X1_encoded, y1)
cart_1.fit(X1_encoded, y1)
nb_1.fit(X1_encoded, y1)

# 5. Nhập các mẫu cần dự đoán theo đề
test1 = pd.DataFrame({
    "Loai": ["Búp bê", "Xếp hình", "Điều khiển"],
    "So_mau": [3, 5, 3],
    "Kich_thuoc": ["To", "To", "Vừa"],
    "Chat_lieu": ["Cao su", "Nhựa PP", "Cao su"]
})

# Doanh số thực tế theo yêu cầu e
y1_test = ["Thấp", "Thấp", "Cao"]

X1_test = encoder1.transform(test1)

# 6. Hàm đánh giá mô hình
def danh_gia(y_true, y_pred, ten_mo_hinh):
    print("\n==========", ten_mo_hinh, "==========")

    print("Nhãn thực tế:", y_true)
    print("Nhãn dự đoán:", y_pred)

    print("\nConfusion Matrix:")
    print(confusion_matrix(
        y_true,
```

```
    y_pred,
    labels=["Thấp", "Cao"]
))

precision = precision_score(
    y_true, y_pred,
    average="weighted",
    zero_division=0
)

recall = recall_score(
    y_true, y_pred,
    average="weighted",
    zero_division=0
)

f_measure = f1_score(
    y_true, y_pred,
    average="weighted",
    zero_division=0
)

print("Precision:", round(precision, 4))
print("Recall:", round(recall, 4))
print("F-measure:", round(f_measure, 4))

print("\nClassification Report:")
print(classification_report(
    y_true,
    y_pred,
    zero_division=0
))
```

```
# 7. Dự đoán và tính F-measure

pred_id3_1 = id3_1.predict(X1_test)

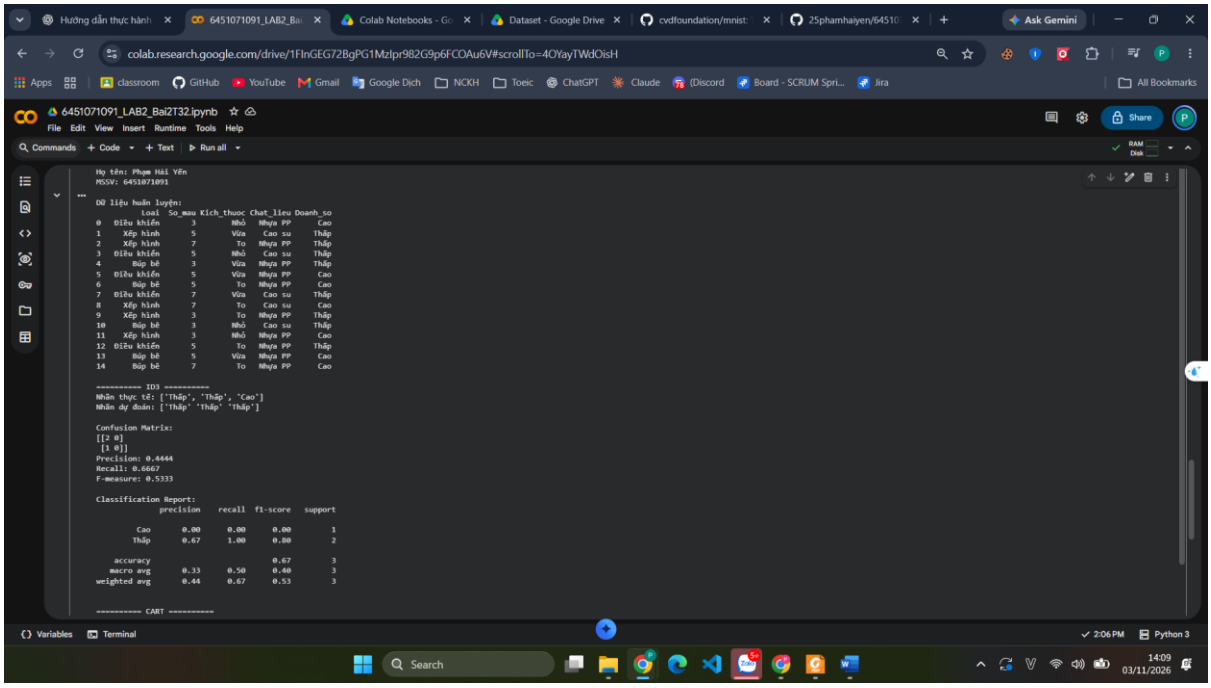
pred_cart_1 = cart_1.predict(X1_test)

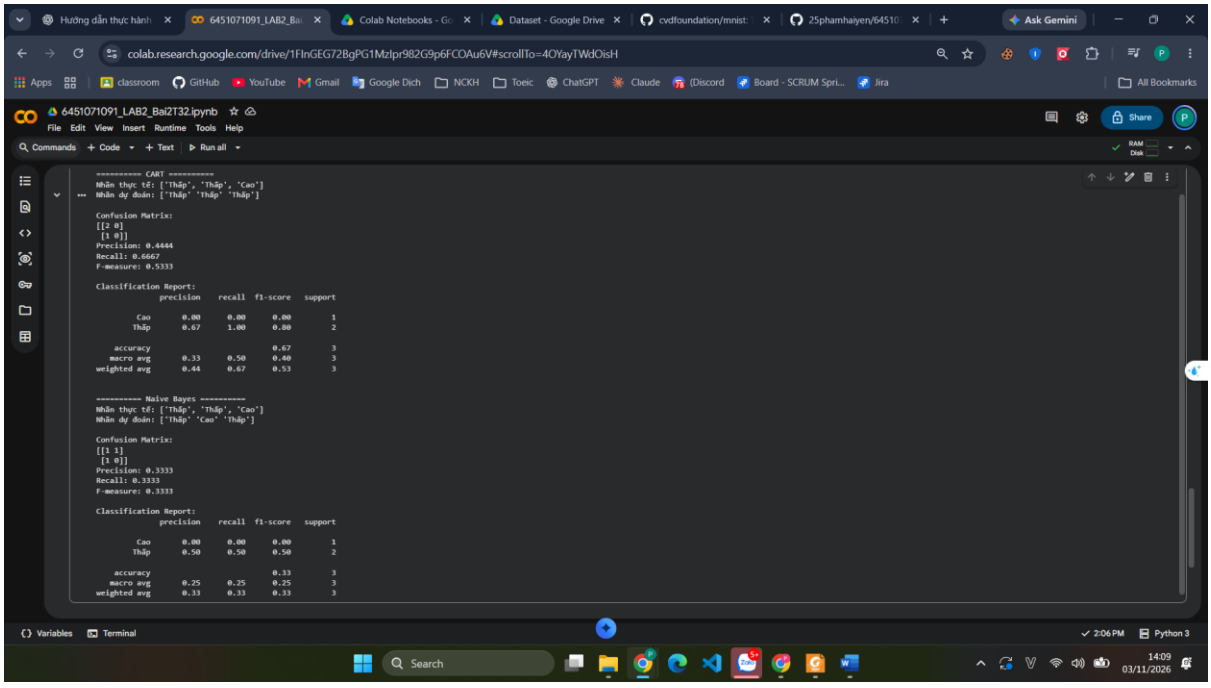
pred_nb_1 = nb_1.predict(X1_test)


danh_gia(y1_test, pred_id3_1, "ID3")

danh_gia(y1_test, pred_cart_1, "CART")

danh_gia(y1_test, pred_nb_1, "Naive Bayes")
```





```
# BÀI 2 - PHÂN TÍCH CẢM XÚC
# Họ tên: Phạm Hải Yến
# MSSV: 6451071091

import pandas as pd
import numpy as np

from sklearn.preprocessing import OrdinalEncoder
from sklearn.tree import DecisionTreeClassifier
from sklearn.naive_bayes import CategoricalNB
from sklearn.metrics import (
    confusion_matrix,
    precision_score,
    recall_score,
    f1_score,
    classification_report
)

# 1. Nhập dữ liệu theo bảng trang 25
data2 = {
    "giam": [
        "0..5", "11..20", "6..10", ">20",
        "0..5", "0..5", "0..5", "11..20",
        "0..5", "11..20", "11..20", ">20",
        "0..5", "6..10", "11..20"
    ],
    "nguoi": [
        "11..20", "6..10", "0..5", "0..5",
        ">20", "11..20", "6..10", "0..5",
        "0..5", "0..5", "0..5", "6..10",
        ">20", "11..20", "11..20"
    ],
    "chuyen": [
```

```
        ">20", "6..10", "6..10", "11..20",
        "11..20", "0..5", "0..5", "11..20",
        "0..5", "6..10", "6..10", "0..5",
        "11..20", "6..10", ">20"
    ],
    "yeu": [
        "11..20", "0..5", "11..20", "6..10",
        "0..5", "0..5", "11..20", "0..5",
        "11..20", "6..10", "6..10", "0..5",
        "11..20", ">20", "0..5"
    ],
    "vua": [
        ">20", "11..20", "0..5", "6..10",
        "0..5", "6..10", "11..20", "0..5",
        "0..5", "0..5", "0..5", "11..20",
        "11..20", "0..5", "0..5"
    ],
    "di": [
        "0..5", "11..20", "6..10", ">20",
        "0..5", "0..5", "11..20", "0..5",
        "6..10", "0..5", "11..20", ">20",
        "0..5", "6..10", "0..5"
    ],
    "Cam_xuc": [
        "tốt", "tốt", "xấu", "bình thường",
        "xấu", "xấu", "xấu", "tốt",
        "bình thường", "tốt", "tốt", "xấu",
        "bình thường", "bình thường", "xấu"
    ]
}

df2 = pd.DataFrame(data2)
```

```
print("Họ tên: Phạm Hải Yến")
print("MSSV: 6451071091")
print("\nDữ liệu huấn luyện:")
print(df2)

# 2. Tách thuộc tính và nhãn
X2 = df2.drop("Cam_xuc", axis=1)
y2 = df2["Cam_xuc"]

# 3. Mã hóa dữ liệu
encoder2 = OrdinalEncoder()
X2_encoded = encoder2.fit_transform(X2)

# 4. Khởi tạo và huấn luyện mô hình
id3_2 = DecisionTreeClassifier(
    criterion="entropy",
    random_state=42
)

cart_2 = DecisionTreeClassifier(
    criterion="gini",
    random_state=42
)

nb_2 = CategoricalNB()

id3_2.fit(X2_encoded, y2)
cart_2.fit(X2_encoded, y2)
nb_2.fit(X2_encoded, y2)

# 5. Nhập các mẫu cần dự đoán theo trang 26
test2 = pd.DataFrame({
    "giam": ["0..5", "0..5", "6..10", "6..10"],
```

```
    "nguoi": ["6..10", "0..5", "0..5", "11..20"],
    "chuyen": ["0..5", "6..10", "11..20", "6..10"],
    "yeu": ["11..20", "0..5", "6..10", "6..10"],
    "vua": ["6..10", "11..20", "0..5", ">20"],
    "di": ["0..5", ">20", "6..10", "0..5"]
})

# Cảm xúc thực tế theo đề
y2_test = [
    "xấu",
    "tốt",
    "bình thường",
    "tốt"
]

X2_test = encoder2.transform(test2)

# 6. Hàm đánh giá mô hình
def danh_gia(y_true, y_pred, ten_mo_hinh):
    print("\n==========", ten_mo_hinh, "==========")

    print("Nhãn thực tế:", y_true)
    print("Nhãn dự đoán:", y_pred)

    print("\nConfusion Matrix:")
    print(confusion_matrix(
        y_true,
        y_pred,
        labels=["xấu", "tốt", "bình thường"]
    ))

    precision = precision_score(
        y_true, y_pred,
```

```
        average="weighted",
        zero_division=0
    )

    recall = recall_score(
        y_true, y_pred,
        average="weighted",
        zero_division=0
    )

    f_measure = f1_score(
        y_true, y_pred,
        average="weighted",
        zero_division=0
    )

    print("Precision:", round(precision, 4))
    print("Recall:", round(recall, 4))
    print("F-measure:", round(f_measure, 4))

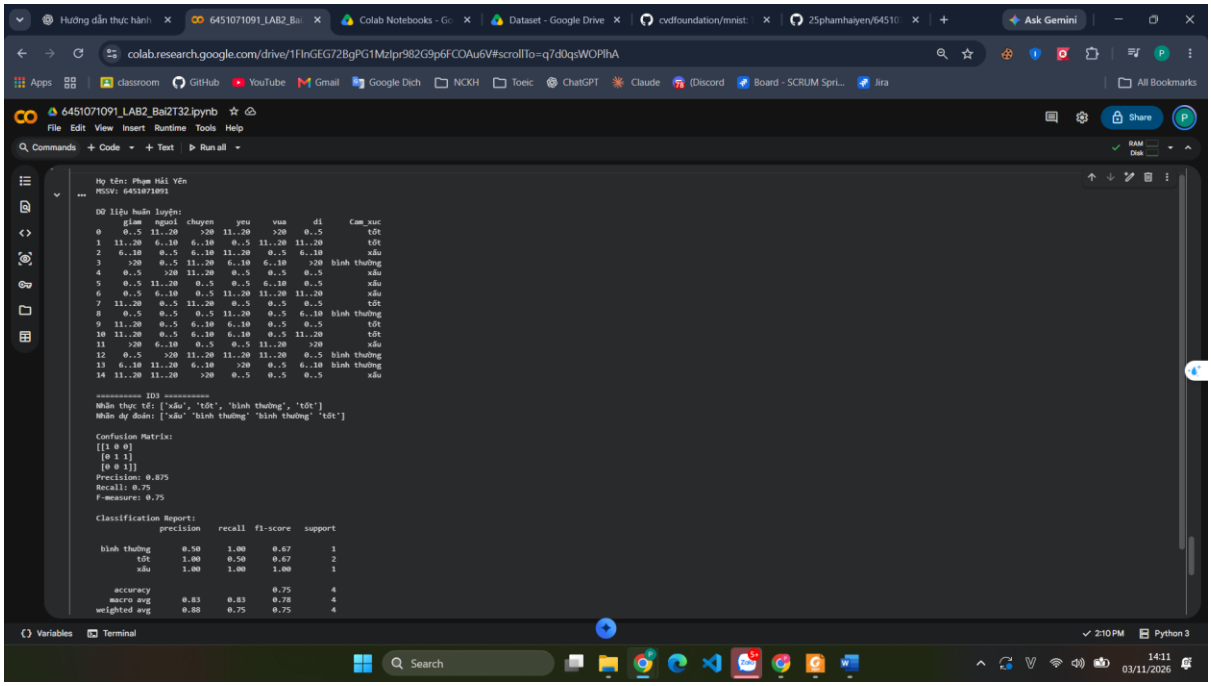
    print("\nClassification Report:")
    print(classification_report(
        y_true,
        y_pred,
        zero_division=0
    ))

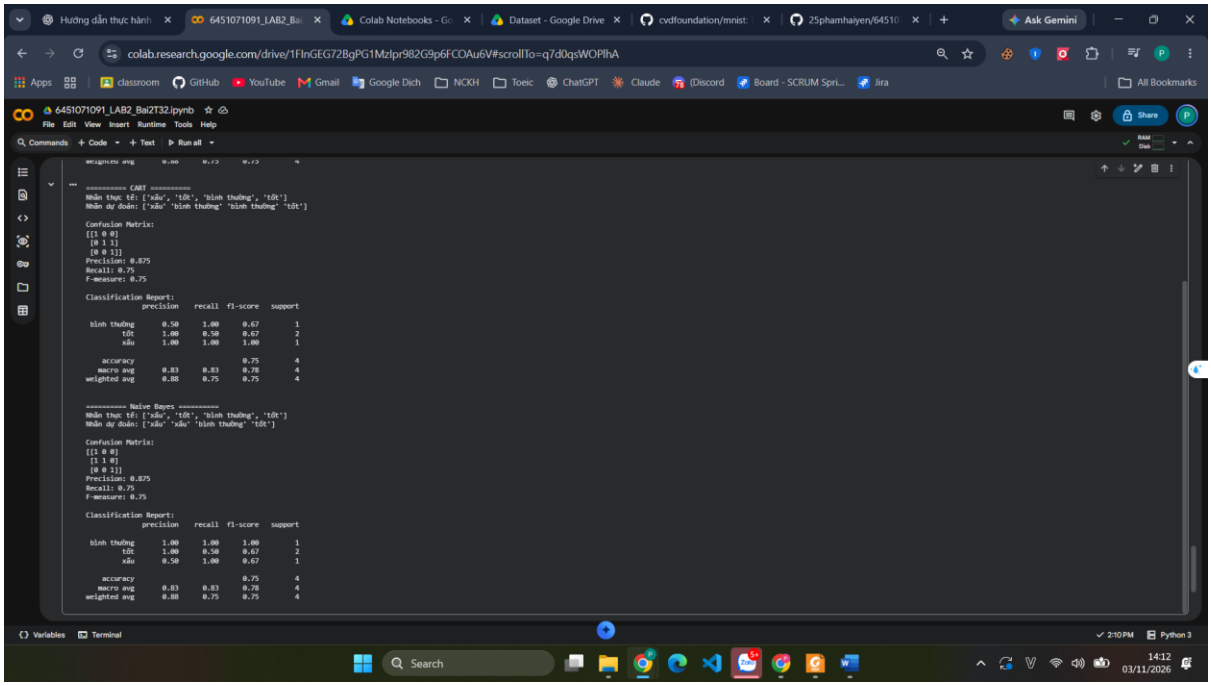
# 7. Dự đoán và tính F-measure
pred_id3_2 = id3_2.predict(X2_test)
pred_cart_2 = cart_2.predict(X2_test)
pred_nb_2 = nb_2.predict(X2_test)

danh_gia(y2_test, pred_id3_2, "ID3")
```

```
danh_gia(y2_test, pred_cart_2, "CART")

danh_gia(y2_test, pred_nb_2, "Naive Bayes")
```





--- HẾT---